Course End Project 1

Simulated Web Application Penetration Testing

Nidhi Sharma

Problem Statement

Real-time scenario:

A FinTech company plans to launch a web application for managing personal finances. Before the launch, ethical hackers test a simulated version of the app to identify vulnerabilities that could expose sensitive data. Hidden flags represent critical information, ensuring no real data is at risk during testing. This provides a safe environment to assess potential security flaws.

Using tools like Burp Suite, testers uncover issues like exposed debug directories, SQL injection vulnerabilities, and misconfigured login forms. These weaknesses highlight significant risks that could allow unauthorized access to sensitive information.

The hackers document their findings and recommend fixes to the development team. These include parameter sanitization, input validation, and improved access controls. After implementing these changes, the app undergoes further testing to ensure its security.

Industry Relevance

The following tools used in this project serve specific purposes within the industry:

- Kali Linux: Widely used for penetration testing and ethical hacking due to its comprehensive toolset
- Burp Suite: Industry standard for web application security testing, enabling vulnerability assessment and exploitation
- VirtualBox: Essential for creating isolated environments to safely test and simulate vulnerabilities
- dirb/dirbuster: Popular tools for directory brute-forcing, helping identify hidden directories and files in web applications

- Nmap: A critical tool for network scanning and reconnaissance, widely used in vulnerability assessment
- OWASP Zap: Frequently employed for automated web application security testing to identify vulnerabilities efficiently

Table of Contents

Project Overview

Goal

Gain hands-on experience by identifying vulnerabilities and capturing hidden flags in a safe, simulated environment.

Scenario

Test a simulated personal finance web app for security flaws before its public launch—no real sensitive data involved.

Tools Used

Kali Linux, Burp Suite, VirtualBox, dirb/dirbuster, Nmap, and OWASP Zap.

Task Breakdown

Import the vulnerable VM

Download and import the vulnerable web application (a Capture-The-Flag box) into VirtualBox.

Configure network settings

Set up the VM’s network and record its IP address.

Reconnaissance

Use tools (like Nmap) to map services and gather system information.

Vulnerability Assessment

Analyze for weaknesses using both manual testing and open-source tools (such as Burp Suite and directory brute-forcing tools).

Report flags and findings

Document all discovered flags and provide recommendations for improving security

Hosting vulnerable site on CentOS VM

We have VirtualBox and CentOS installed, so we will be moving to directly hosting the vulnerable website in CentOS VM.

Running the Vulnerable App Directly as a VM in VirtualBox (Imported OVA)

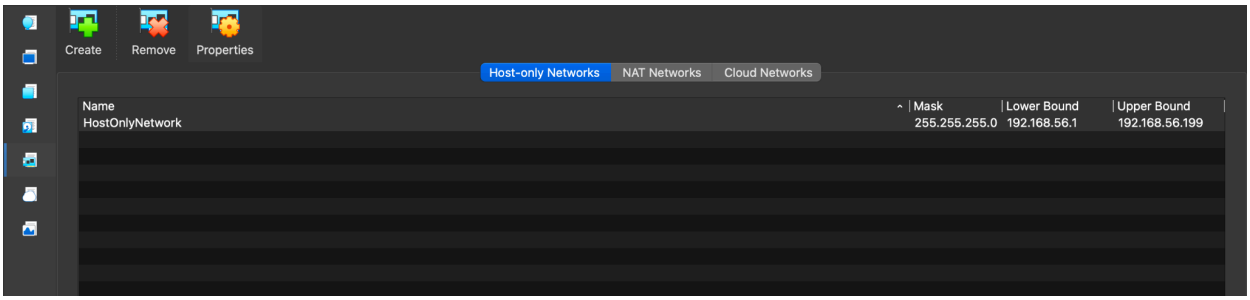
Steps:

- Download the OVA file of the vulnerable app VM.
- Open VirtualBox and use “Import Appliance” to load the OVA.
- Configure the VM as needed (RAM, CPU, network).
- Start the VM to boot into the vulnerable app environment.

Configure Host-Only Network for Virtual Machines (kali and Vulnerable App)

Create a Host-Only Network in VirtualBox

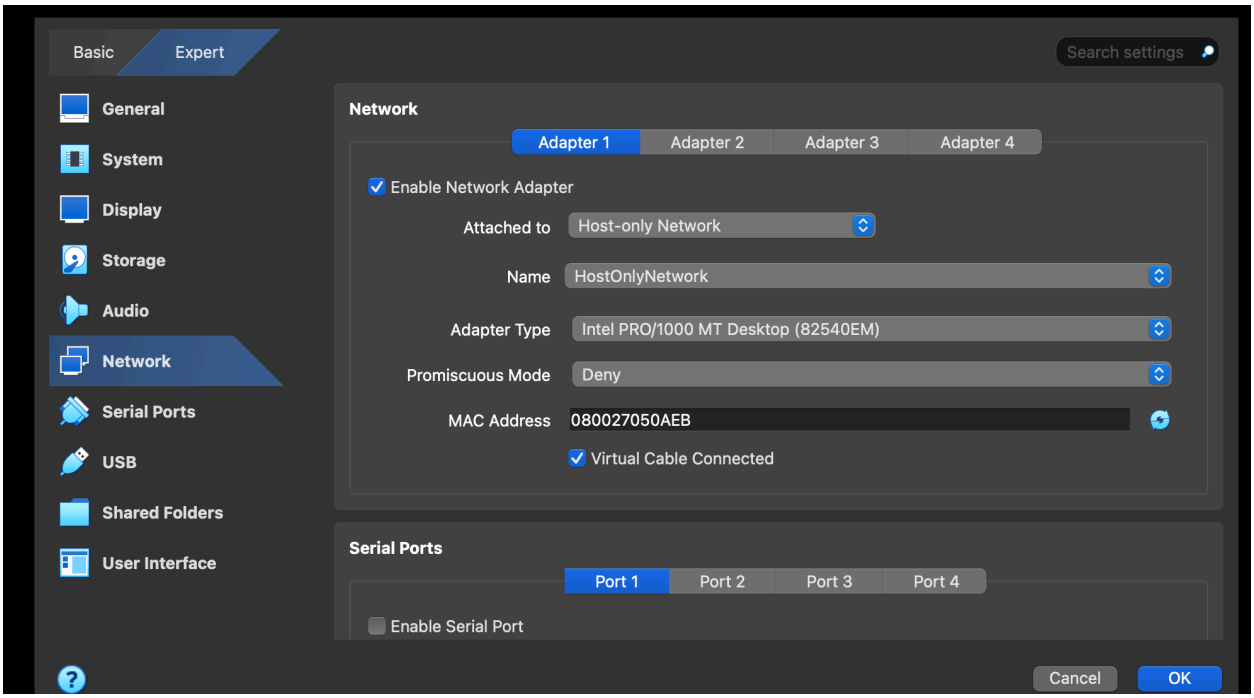
- Open VirtualBox Manager.
- Go to the Left Menu Bar > Network → Oracle VirtualBox Manager
- Check if a host-only network already exists (e.g., vboxnet0). If not, create a new one by clicking on "Create".
- Configure its DHCP settings if needed (usually default works fine).

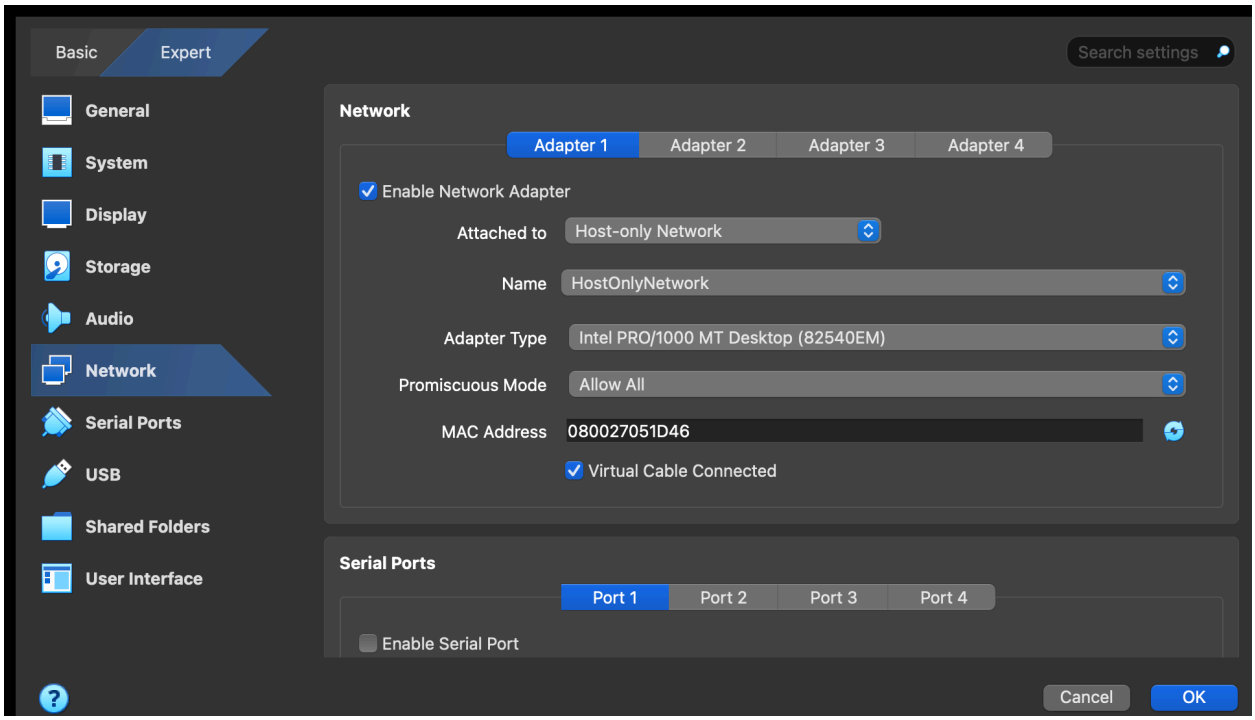


Assign VMs to Host-Only Network

- For both Kali and the vulnerable VM:
  - Open Settings > Network.
  - Attach Adapter 1 to Host-Only Network.
  - From the dropdown, select the created host-only network (e.g., vboxnet0).

Kali Linux screenshot below and Vulnerable VM Screenshots


Basic
Expert
Search settings
General
System
Display
Storage
Audio
Network
Serial Ports
USB
Shared Folders
User Interface
Network
Adapter 1
Adapter 2
Adapter 3
Adapter 4
Enable Network Adapter
Attached to
Host-only Network
Name
HostOnlyNetwork
Adapter Type
Intel PRO/1000 MT Desktop (82540EM)
Promiscuous Mode
Deny
MAC Address
080027050AEB
Virtual Cable Connected
Serial Ports
Port 1
Port 2
Port 3
Port 4
Enable Serial Port
Cancel
OK


Basic
Expert
Search settings
General
System
Display
Storage
Audio
Network
Serial Ports
USB
Shared Folders
User Interface
Network
Adapter 1
Adapter 2
Adapter 3
Adapter 4
Enable Network Adapter
Attached to
Host-only Network
Name
HostOnlyNetwork
Adapter Type
Intel PRO/1000 MT Desktop (82540EM)
Promiscuous Mode
Allow All
MAC Address
080027051D46
Virtual Cable Connected
Serial Ports
Port 1
Port 2
Port 3
Port 4
Enable Serial Port
Cancel
OK

Start both VMs

They will get IP addresses from the host-only network and can communicate with each other.

Verify Connectivity

- Check IP on both VMs.
- Ping each other to confirm network access.

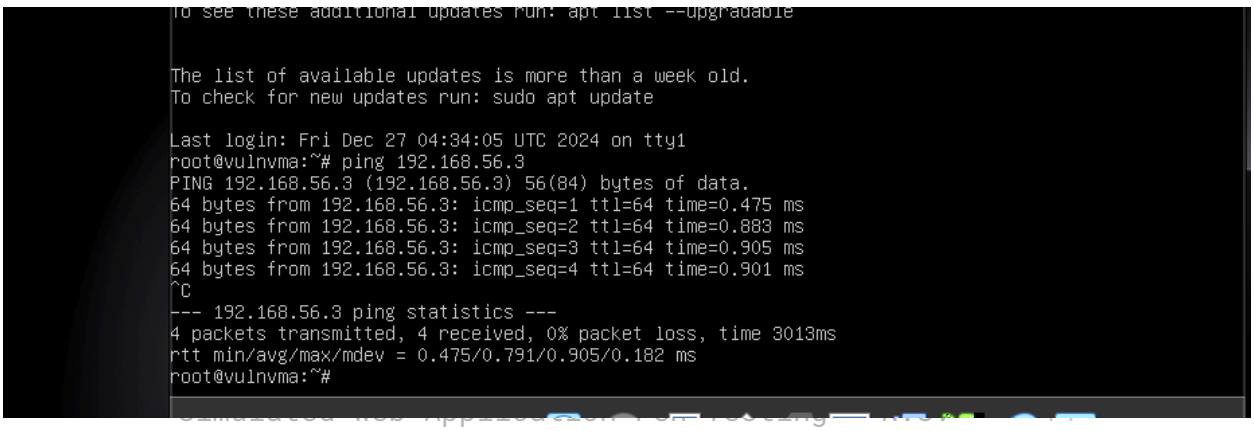
Check IP Addresses

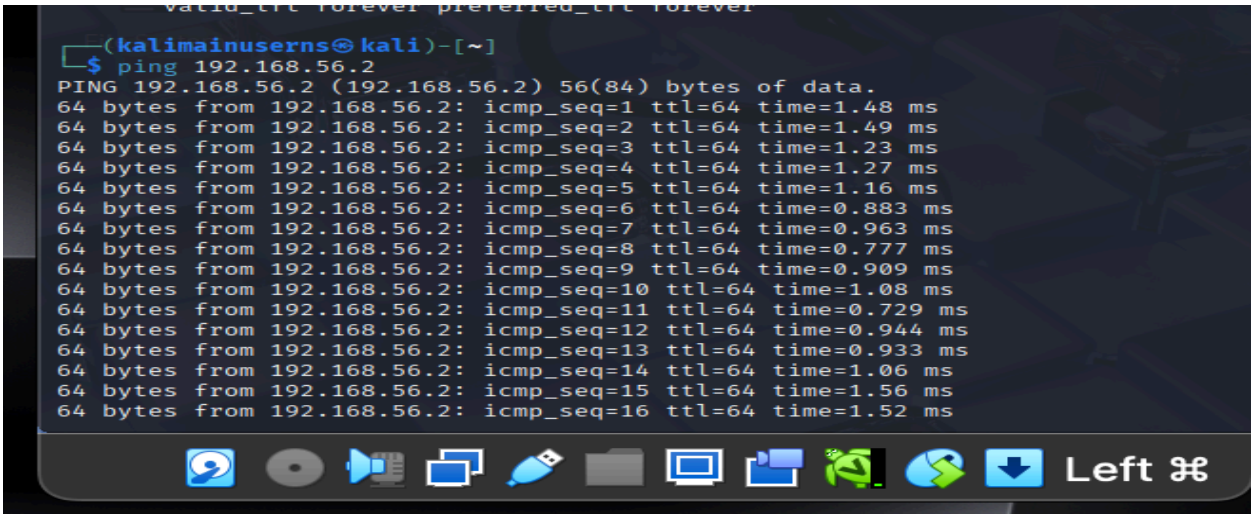
  - In each VM, run ifconfig or ip a to get their IPs on the host-only network / (usually something like 192.168.56.X) or a NAT network. Whatever we choose, we need to choose the same setting for both the VM.

Test Connectivity

  - From Kali, ping the vulnerable VM's IP.
  - We should be able to reach the target VM. Now we can proceed with security scans and testing.
  -

- Now we can see both can ping each other.




(kalimainuserns㉿kali)-[~]
$ ping 192.168.56.2
PING 192.168.56.2 (192.168.56.2) 56(84) bytes of data.
64 bytes from 192.168.56.2: icmp_seq=1 ttl=64 time=1.48 ms
64 bytes from 192.168.56.2: icmp_seq=2 ttl=64 time=1.49 ms
64 bytes from 192.168.56.2: icmp_seq=3 ttl=64 time=1.23 ms
64 bytes from 192.168.56.2: icmp_seq=4 ttl=64 time=1.27 ms
64 bytes from 192.168.56.2: icmp_seq=5 ttl=64 time=1.16 ms
64 bytes from 192.168.56.2: icmp_seq=6 ttl=64 time=0.883 ms
64 bytes from 192.168.56.2: icmp_seq=7 ttl=64 time=0.963 ms
64 bytes from 192.168.56.2: icmp_seq=8 ttl=64 time=0.777 ms
64 bytes from 192.168.56.2: icmp_seq=9 ttl=64 time=0.909 ms
64 bytes from 192.168.56.2: icmp_seq=10 ttl=64 time=1.08 ms
64 bytes from 192.168.56.2: icmp_seq=11 ttl=64 time=0.729 ms
64 bytes from 192.168.56.2: icmp_seq=12 ttl=64 time=0.944 ms
64 bytes from 192.168.56.2: icmp_seq=13 ttl=64 time=0.933 ms
64 bytes from 192.168.56.2: icmp_seq=14 ttl=64 time=1.06 ms
64 bytes from 192.168.56.2: icmp_seq=15 ttl=64 time=1.56 ms
64 bytes from 192.168.56.2: icmp_seq=16 ttl=64 time=1.52 ms
Left ⌘

Reconnaissance & Scanning with Nmap

Purpose: Discover Open ports and Services running on Vulnerable VM

Open a terminal in Kali Linux.

Run the command:

`nmap -sS -sV -A 192.168.56.2`

`-sS` for stealth SYN scan

`-sV` to detect service versions

`-A` to enable OS and version detection, script scanning, and traceroute

Analyze open ports (e.g., 80 for HTTP, 443 for HTTPS) and running services.

Analysing NMap Results

Open Ports and Services

- Summary of open/closed ports:
  - 21/tcp open ftp (vsftpd 3.0.5)
  - 22/tcp open ssh (OpenSSH 8.9p1)
  - 80/tcp open http (Apache 2.4.52)
  - 20/tcp, 443/tcp, 8000/tcp are closed.

Service Version Enumeration

- FTP (vsftpd 3.0.5):

    - Allows anonymous login (ftp-anon: Anonymous FTP login allowed).
    - No bandwidth limit, plain text login – insecure.
    - Attack vector: Try connecting via FTP with username anonymous, look for file upload vulnerabilities, sensitive data disclosure, or misconfigurations.
- SSH (OpenSSH 8.9p1):
    - Secure remote administration.
    - Usually not worth attacking directly unless a weak password, old version, or brute-force has potential.
- HTTP (Apache 2.4.52):
    - Port 80 is open, running Apache under Ubuntu.
    - The web application is likely hosted here.
    - Attack vector: Test for web vulnerabilities (SQLi, XSS, file upload, info disclosure).
    - Shows a “Travel Blog” in the HTTP title; robots.txt allows 1 disallowed entry (could hint at hidden paths).

Recommended Attack Paths

- FTP (Port 21):
    - Use anonymous login to see if sensitive files are accessible/downloadable.
    - Check for directory traversal or misconfigured permissions.
- Web (Port 80):
    - Browse http://192.168.56.2 to review the web application.
    - Run Dirb/Dirbuster for hidden directories.
    - Use Burp Suite, OWASP ZAP for vulnerability scanning (XSS, SQLi, insecure auth, etc.).
- SSH:
    - Only pursue if we can find credentials elsewhere (e.g., from FTP or web).
- Robots.txt:
    - Check http://192.168.56.2/robots.txt for restricted directories that might contain sensitive info.

Planning Next Steps

Document versions:

- Published vulnerabilities or exploits for
  - vsftpd 3.0.5
  - OpenSSH 8.9p1
  - Apache 2.4.52
- Focus on web application pentesting and possible abuses via FTP.
- Less emphasis on brute-forcing SSH unless login/password hints are found from other services.

Nmap output tells us which services to target, what potential weaknesses exist, and lets us plan our next moves – attack FTP with anonymous login, enumerate and attack the web site, investigate any sensitive files or directories, and note technology for future steps with Dirb, Burp, and ZAP.

Directory Brute-Forcing with Dirb or Dirbuster

Purpose

Find hidden directories or files that are not linked visibly on the web app.

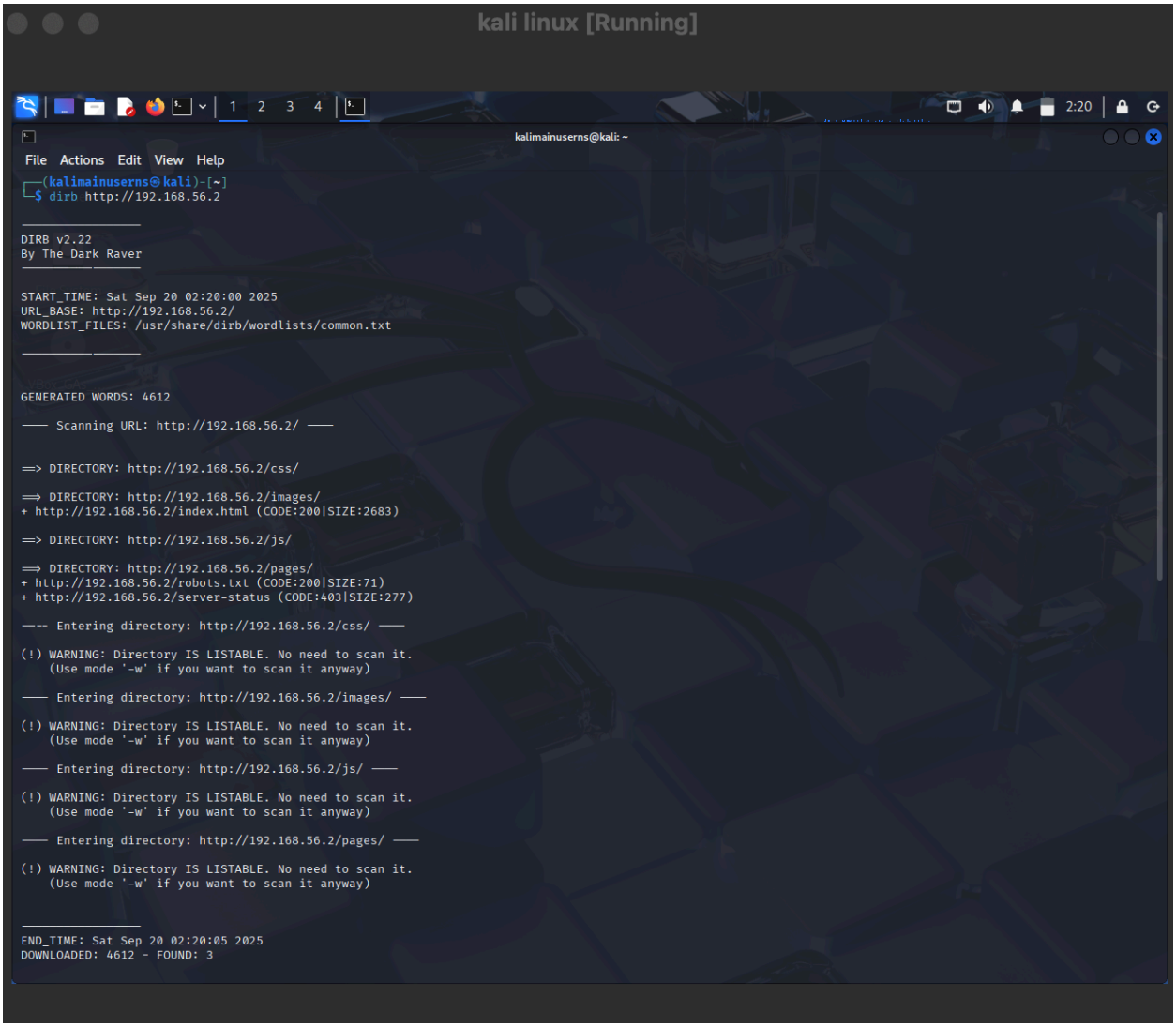
The nmap scan revealed port 80 (HTTP) running Apache 2.4.52 is open, which means a web server is hosting content on the target. The purpose of web-based pentesting is to discover all possible web entry points, including hidden files and directories.
Directory Brute-Forcing (Dirb or Dirbuster) is a natural second step after nmap because:

- Nmap told us that web services are running (port 80, Apache)
- We want to find every possible “hidden” web source, admin page, config file or upload script exposed on the server
- These are prime targets for web vulnerabilities (XSS, authentication bypass, SQLi)
- We could attack FTP after the nmap (anonymous login allowed). Directory brute forcing is often prioritised in CTF (Capture the Flag) and analysis because it reveals web app structure - FTP exploration could be performed in parallel or shortly after a holistic approach.

Using Dirb

Run dirb against web root:

```
dirb http://192.168.56.2
```



Dirb uses its default wordlist (`/usr/share/dirb/wordlists/common.txt`). We may specify other wordlists for deeper scans.

Outputs directories/files found.

Why Directory Brute-Forcing Matters After Nmap

Nmap shows HTTP is running; brute-forcing directories immediately reveals the application's secret structure, giving us more targets for step 3 (manual exploitation, Burp Suite, ZAP testing).

We can often find `/admin`, `/config`, `/uploads`, which may not be linked from the main interface but still exist and could be vulnerable.

Example Workflow

- Nmap: Shows web service is active (port 80 open).
- Dirbuster/Dirb: Find hidden paths; /admin or /uploads.
- Browse new paths: Manual or automated testing with web scanners, attack found directories.
- Parallel FTP attack: Since anonymous login is allowed on port 21, check for sensitive files and potential credential leaks.

We used Dirb/Dirbuster after nmap because nmap confirms an HTTP service; directory brute-forcing systematically reveals the web server's structure for deeper attacks. Wordlists are vital – they're directories and file names that Dirb/Dirbuster tries in HTTP requests. The combo of nmap ("what's running?"), then brute-forcing ("what can I reach?"), is the classic pentest workflow.

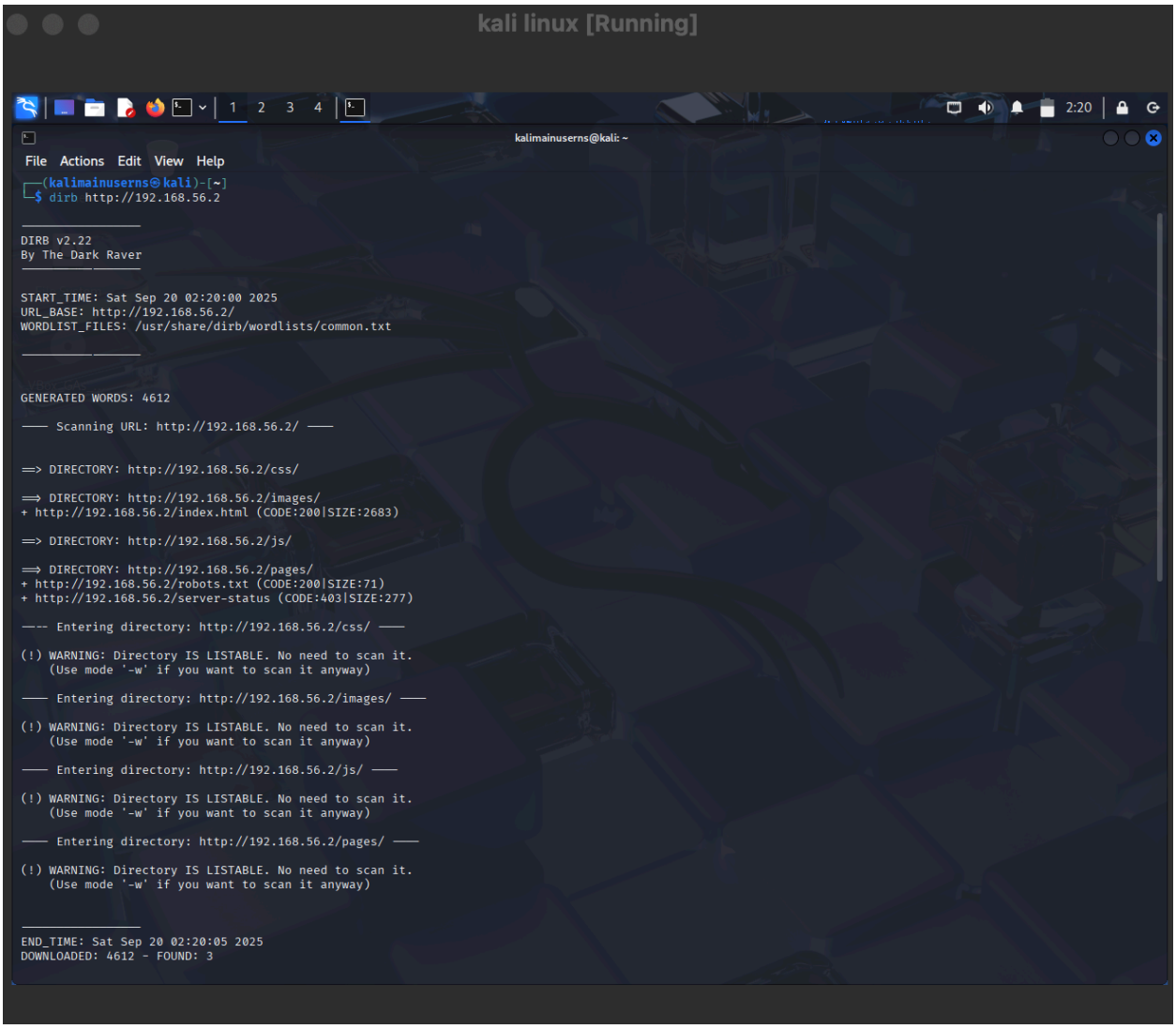
Analysing the result of Dirb

The Dirb scan results indicate several discoverable and "listable" directories on the target web server.

Scanned with wordlist `/usr/share/dirb/wordlists/common.txt` against `http://192.168.56.2/`.

Directly discovered directories/files:

- `/css/`
- `/images/`



- `/js/`
- `/pages/`
- `/index.html (main page)`
- `/robots.txt`
- `/server-status (potentially status or config info)`

LISTABLE Directory Warning:
It means Directories like `/css/, /images/, /js/, /pages/` are “listable” – their contents can be viewed in a browser, showing all files inside. This is often insecure, as attackers can see resources that should be hidden.

Planning Our Next steps

- Dirb shows which web directories and files are exposed. Now, let's visit each of them, enumerate their content, and plan targeted exploitation using Burp Suite, ZAP, or manual browser actions.
- Link findings with what we know from Nmap; e.g., see if any file in /pages/ or similar leaks info that could help us attack FTP, SSH, or escalate privileges.
- This Dirb output shows that our scan of `http://192.168.56.2/` found several directories (`/css/`, `/images/`, `/js/`, `/pages/`) and files (`/index.html`, `/robots.txt`, `/server-status`).
- The "LISTABLE" warning means these directories allow directory listing, so their contents are visible to anyone browsing that path in a browser.

What this result means

- Each found directory could contain files or scripts not normally linked from the main site, which may be poorly secured or provide clues about app logic.
- Directory listing is often a misconfiguration; it may expose sensitive files, scripts, or backups that should not be publicly viewable.
- Files like /robots.txt may point to locations meant to be hidden from search engines, while /server-status can leak server information.

Next steps

- Manually visiting each directory in the browser and looking at the contents.
- Making note of all files revealed — checking for source code, config files, or anything unusual.

- Visiting `/robots.txt` for clues about additional sensitive or hidden areas.
- Visit `/server-status` to see if info about the server or running applications is exposed.
- Use a larger wordlist (e.g., `/usr/share/wordlists/dirb/big.txt`) if we want to discover more obscure directories or files.

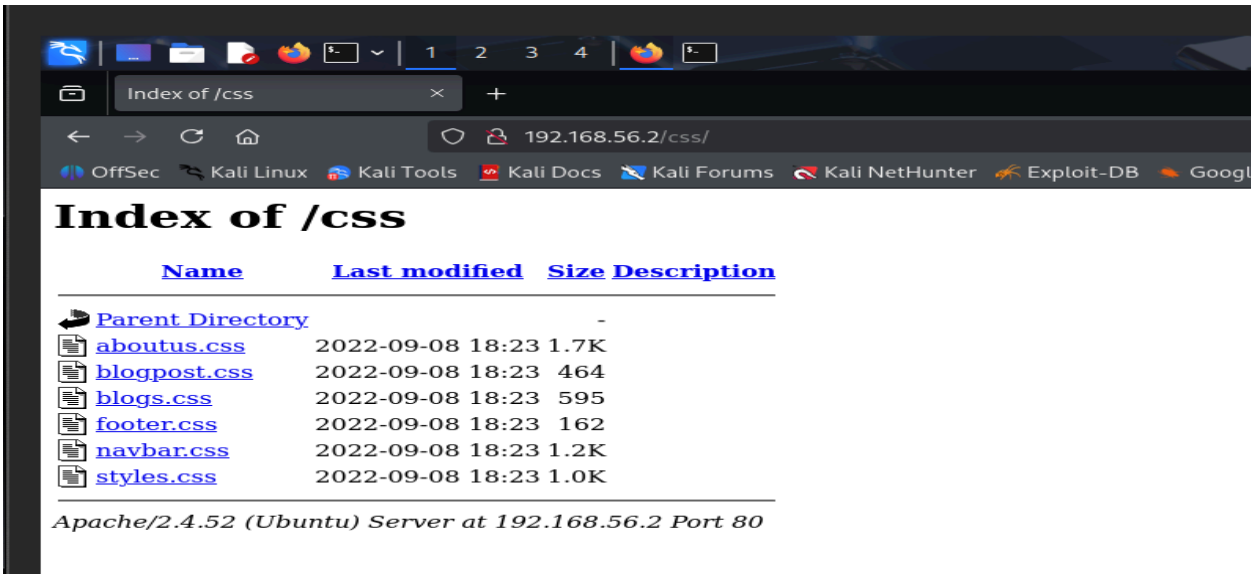
Linking with nmap results

We use Dirb because nmap shows that HTTP (`port 80`) is open; Dirb targets the web service exposed on that port.

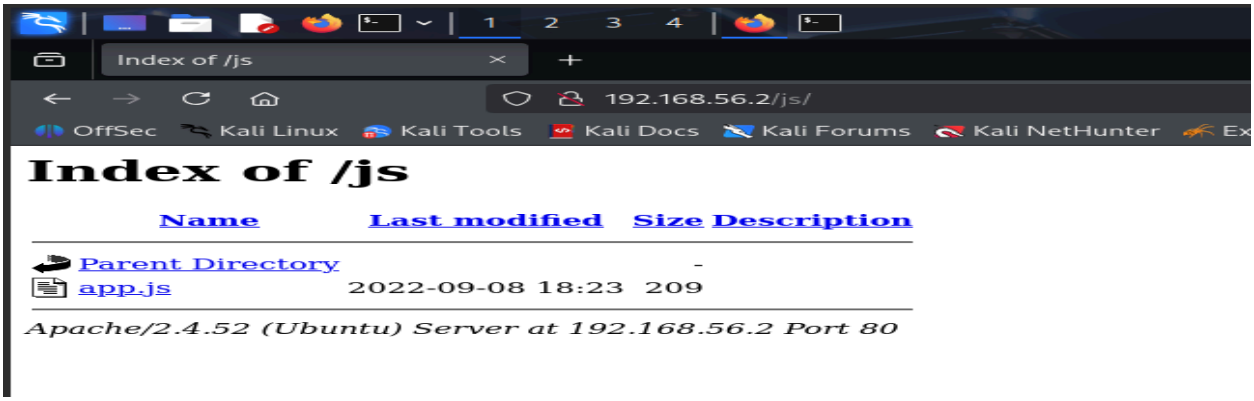
Findings from Dirb may directly influence later attacks with Burp Suite, ZAP, or manual testing—any discovered script or page is a candidate for further investigation for vulnerabilities.

Manual Exploration of Exposed Targets

- http://192.168.56.2/css

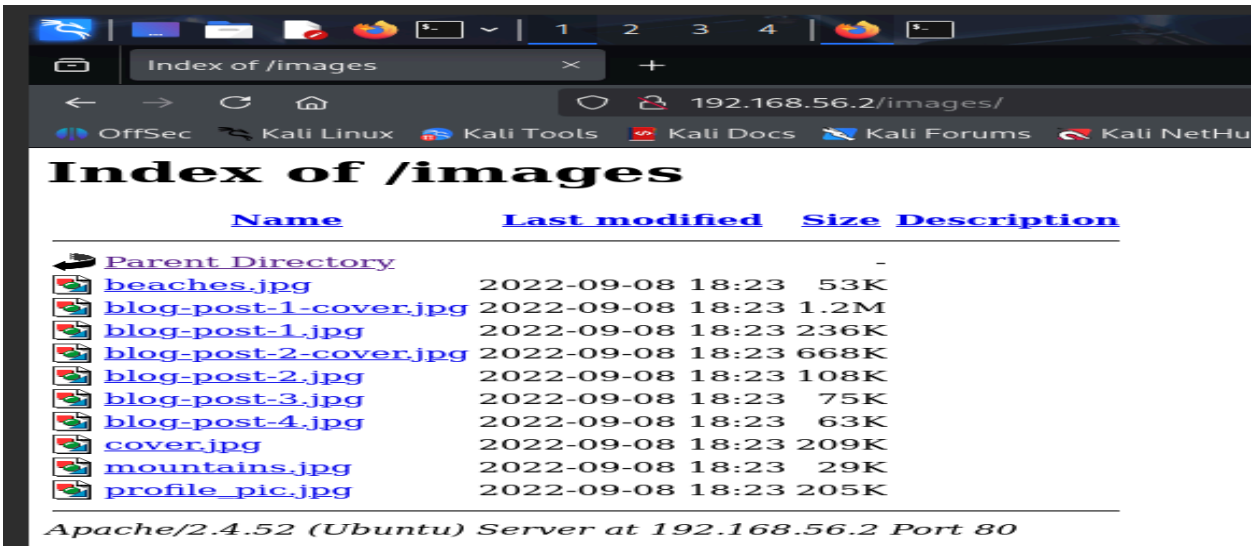


- http://192.168.56.2/js/

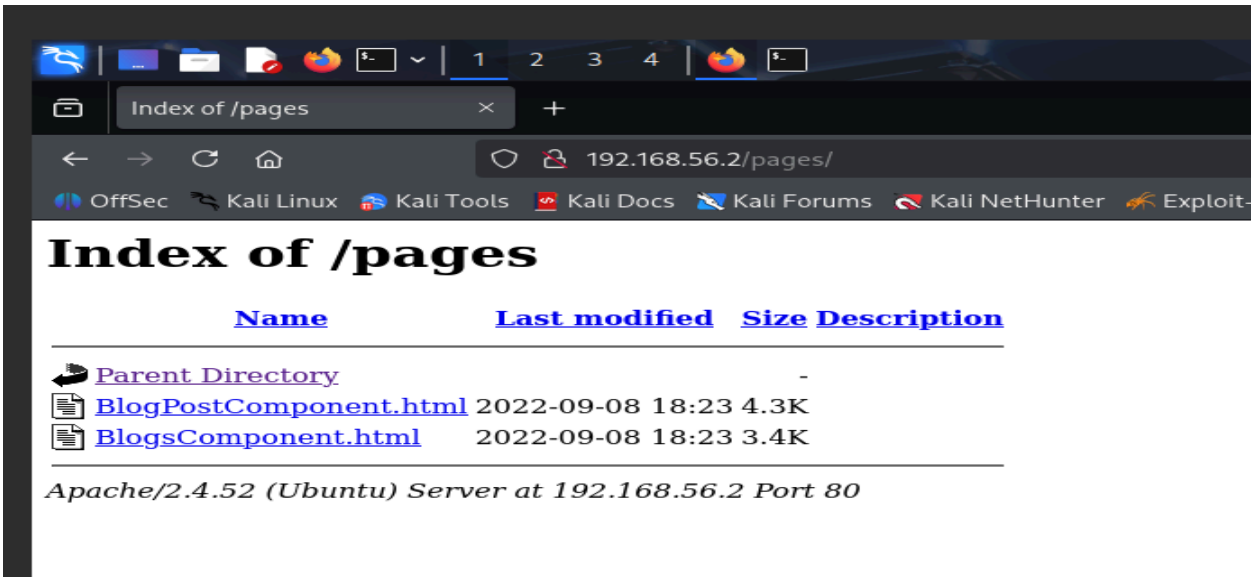


W

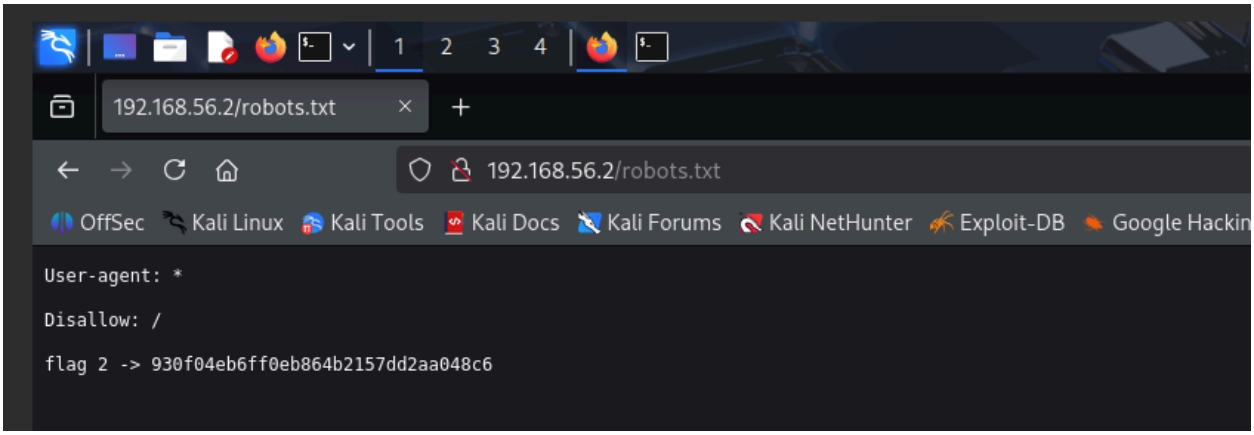
- http://192.168.56.2/images



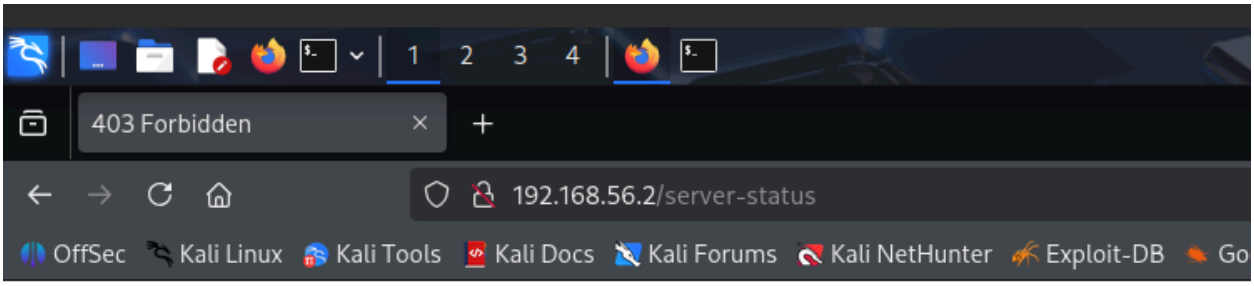
- http://192.168.56.2/pages/



- http://192.168.56.2/robots.txt



- http://192.168.56.2/server-status



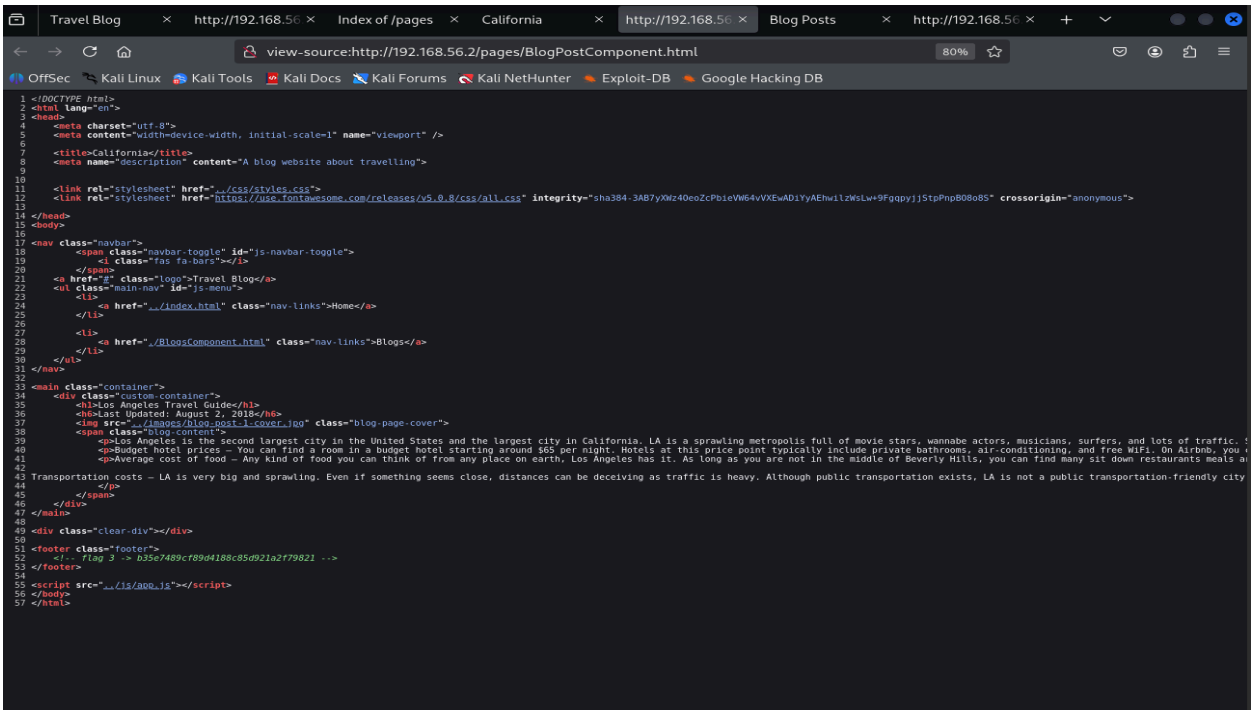
Forbidden

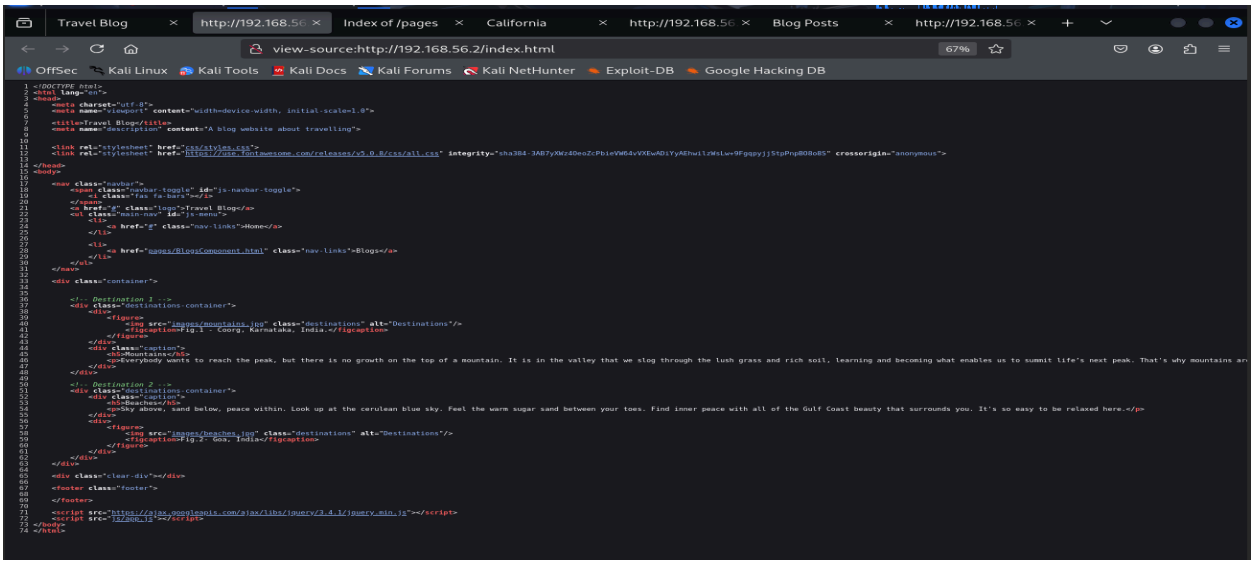
You don't have permission to access this resource.

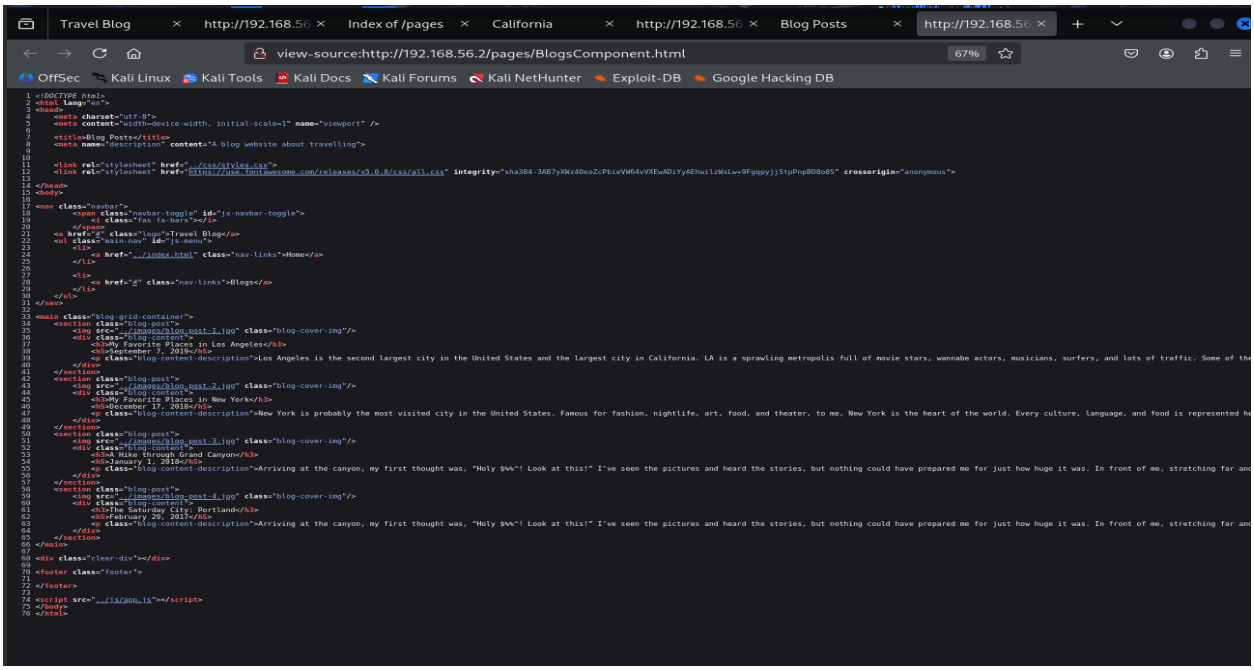
Apache/2.4.52 (Ubuntu) Server at 192.168.56.2 Port 80

Next Steps

Reviewing Source Code







For BlogPostComponent.html, BlogsComponent.html, and app.js, index.html , view their full source (right-click > View Page Source or open in browser) to look for:

- Hidden comments
- Credentials or API keys
- Unlinked endpoints or actions
- Logic that may be exploitable (e.g., hidden parameters)
- Vulnerabilities or weakness

Test for Vulnerabilities

- Using Burp Suite & OWASP ZAP to proxy the browsing while interacting with these components and inputs. Look for:
    - XSS opportunities in blog/comment sections

Parameter tampering

- IDOR (insecure direct object reference) or sensitive file access via URL changes.

Deeper Enumeration

- Exploring if we can POST/GET to new endpoints referenced by app.js.
- Try directly accessing files with similar names (guessing), e.g., `/pages/admin.html`

Our Findings

- Flag

```
Disallow: /

flag 2 -> 930f04eb6ff0eb864b2157dd2aa048c6
```

```
49 <div class="clear-div"></div>
50
51 <footer class="footer">
52     <!-- flag 3 -> b35e7489cf89d4188c85d921a2f79821 -->
53 </footer>
54
55 <script src="../js/app.js"></script>
56 </body>
57 </html>
```

- Files are listable and contents publicly exposed (a critical finding)
- Document 403 on `/server-status` as a sign of improved, but not perfect, hardening

Summary:

- We are mapping actual site content, confirming what Dirb found, revealing sensitive data (flag 2), and preparing for more advanced attacks by analyzing page source and dynamic web requests.
- This is the core of how enumeration moves into vulnerability identification and exploitation

Analysing `/pages/BlogsComponent.html`

Observations

- The code is mostly static HTML with no visible forms, user-input fields, or backend interaction.
- Uses client-side JavaScript only for navbar toggling (from `/js/app.js`).
- No embedded credentials, API keys, or comments with sensitive information.

Potential issues

- Reveals static blog post data—no direct exploit unless JavaScript or other dynamically referenced resources are hidden.

Next Action:

- Check for hidden JavaScript or AJAX calls (not seen here).
- Use Burp/ZAP to monitor any additional requests as we interact with the live page.

Analysing `/pages/BlogPostComponent.html`

Observations:

- As discovered in previous section, this file does contain a hidden HTML comment at the bottom:
  `<!-- flag 3 -> b35e7489fc894b188c85d921a2f79821 -->`
- This is a CTF "flag"—capture and record for our project as a discovered proof.
- No forms, input boxes, or dynamic elements seen.

Potential issues:

- In a real application, hidden comments with sensitive data will become a vulnerability. Here, it's intentional for the exercise.

Action:

- Always look for HTML comments with sensitive info in every page's source.
- Record this flag as proof of access.

Analysing `/index.html`

Observations:

- Basic static front page.
- Links to the blogs component via `href="pages/BlogsComponent.html"`.
- Includes images and a navigation bar, no form fields or dynamic actions in the HTML.
- No plaintext credentials or sensitive comments found.

Potential issues:

- No clear security issue in this static code.
- Ensure to check if any JavaScript referenced loads secondary data (we can review `/js/app.js` again).

Action:

- Look for hidden fields, unlinked content, or dynamically loaded scripts.

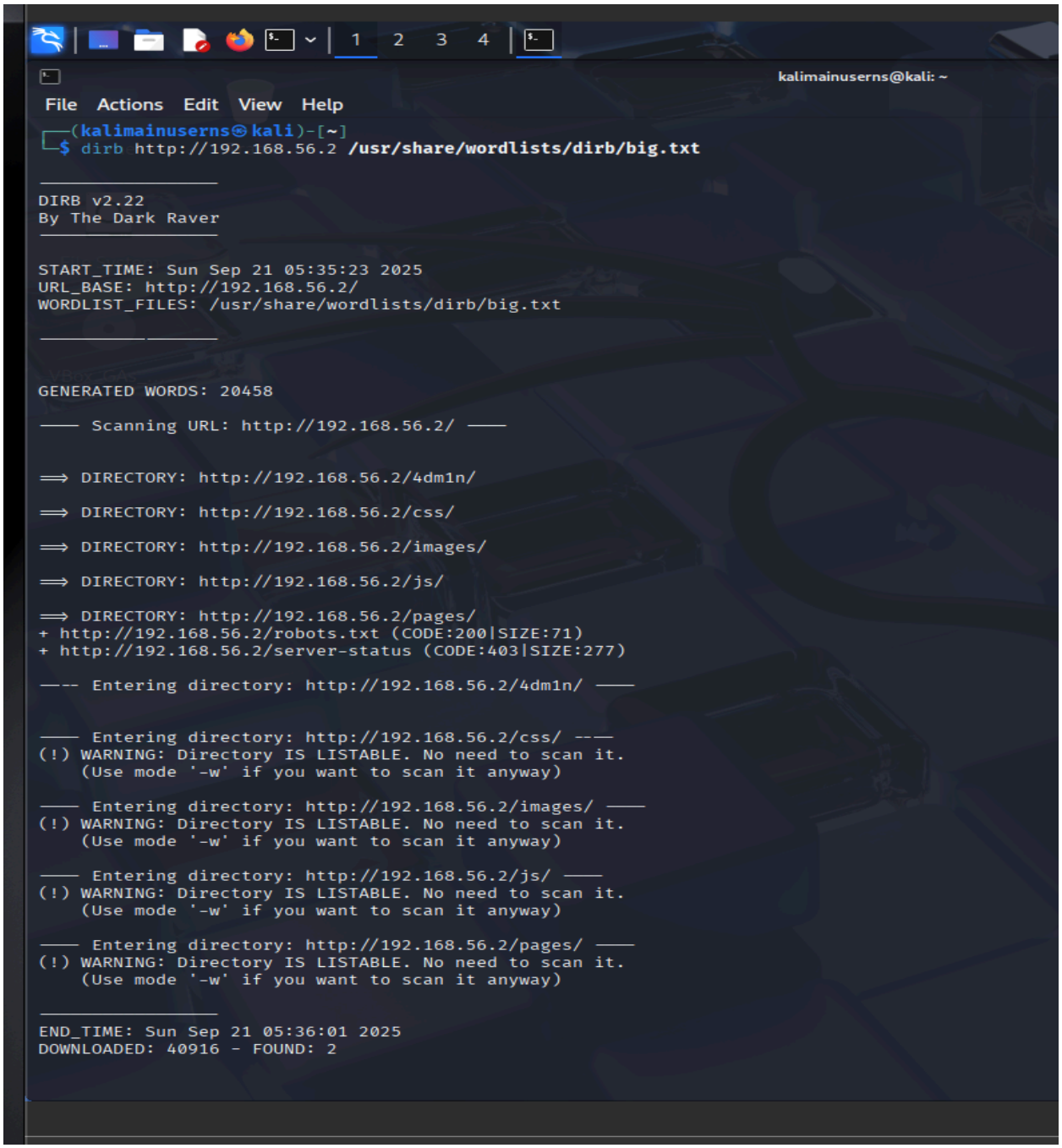
General Next Steps

- No credentials or secrets are visible other than the intended CTF flag.
- Main potential vulnerabilities would only appear if there were form fields (for XSS, SQLi) or AJAX/dynamic calls in scripts—none detected in the code provided.

For further testing:

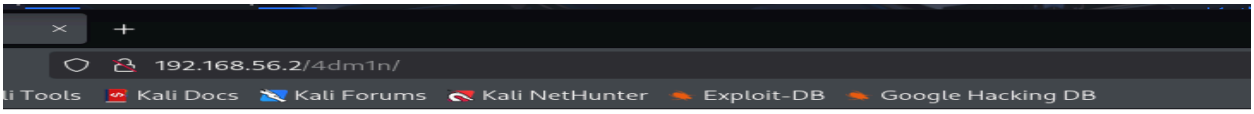
- Use dirb with `big.txt`
- Proxy browser traffic through Burp Suite or OWASP ZAP while browsing these pages to intercept AJAX calls, hidden resources, or API endpoints that don’t appear in the static source.
- Brute-force or guess other file and page names based on patterns we see (e.g., there may be additional Component files or similar hidden admin pages).

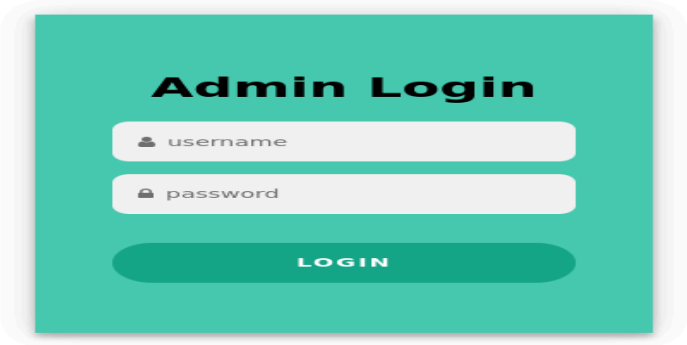
Testing with dirb big.txt



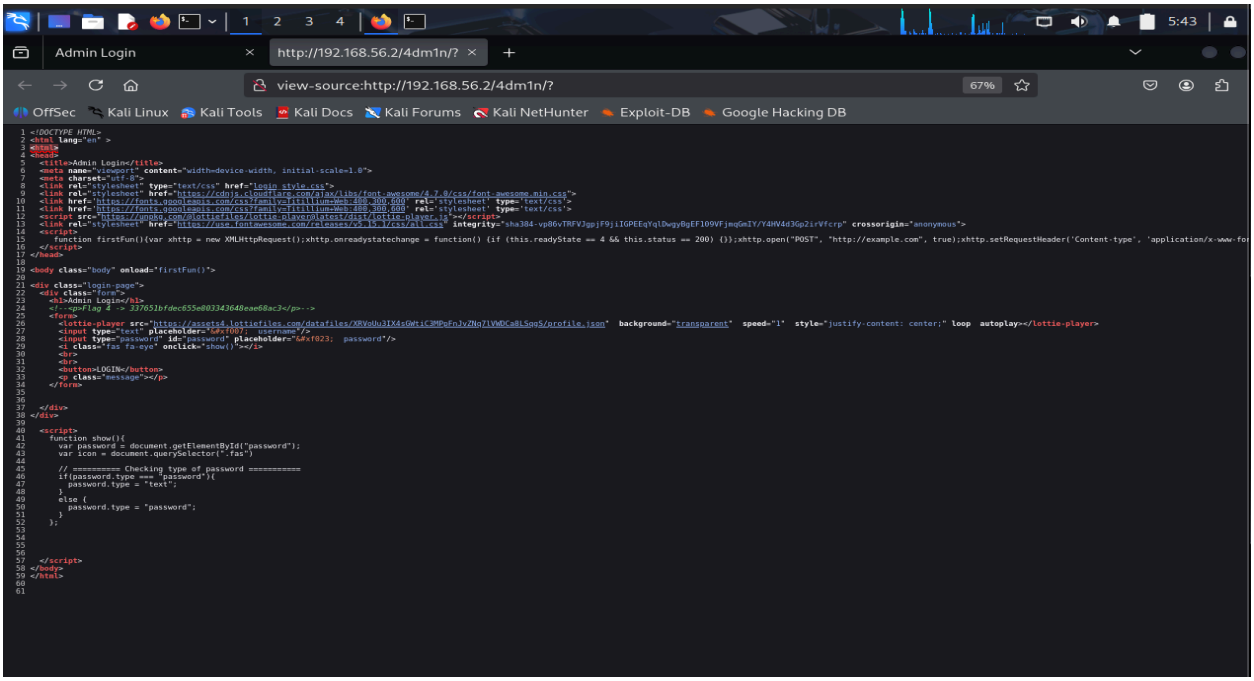
We can see one new entry: `/4dm1n`

Accessing `http://192.168.56.2/4dm1n`





This is the admin page. Next we go to view page source. Screenshot



below.

Analysing above output

Dirb Output (`big.txt` wordlist)

- With a larger wordlist, Dirb discovered /4dm1n/ as a new directory not found before.
- This suggests the admin login page is somewhat "hidden-by-obscurity," as a basic wordlist wouldn't uncover it.
- This is a common way to hide sensitive functionality, but not secure against determined attackers.

`/4dm1n/` Page and Source

- The page is a basic "Admin Login" form for username and password.
- Source Code Review:
- There's a flag:
  `<!-- flag 4 -> 3f257b16cf655c0033d643e68ae68ac3 -->` in an HTML comment.
- No credentials are visible, but the login form is present.
- Simple client-side JS lets us show/hide passwords, but there's no security vulnerability in that itself.

Next Steps

- Record the new flag discovered in the HTML comment:
  flag 4 -> 3f257b16cf655c0033d643e68ae68ac3

```
20
21 <div class="login-page">
22   <div class="form">
23     <h1>Admin Login</h1>
24     <!--<p>Flag 4 -> 337651bfdec655e803343648eae68ac3</p>-->
25     <form>
26       <lottie-player src="https://assets4.lottiefiles.com/datafiles/XRVoUu3IX4sGWtiC
27       <input type="text" placeholder="&#xf007;  username"/>
28       <input type="password" id="password" placeholder="&#xf023;  password"/>
29       <i class="fas fa-eye" onclick="show()"></i>
```

- Use ***Burp Suite or ZAP (proxies) to intercept our login attempts*** and test for:

    - Error messages
    - Weak or default credential acceptance
    - SQL injection responses (try simple payloads)
    - Parameter tampering
- If the server returns different status codes or error pages, follow up with further enumeration and payloads.
- If ***brute-forcing or injection*** allows admin login, document the process and what data, pages, or admin power is gained.
- ***Try common credentials*** (admin:admin, admin:password, etc.) or payload-based bypasses.

Summary:

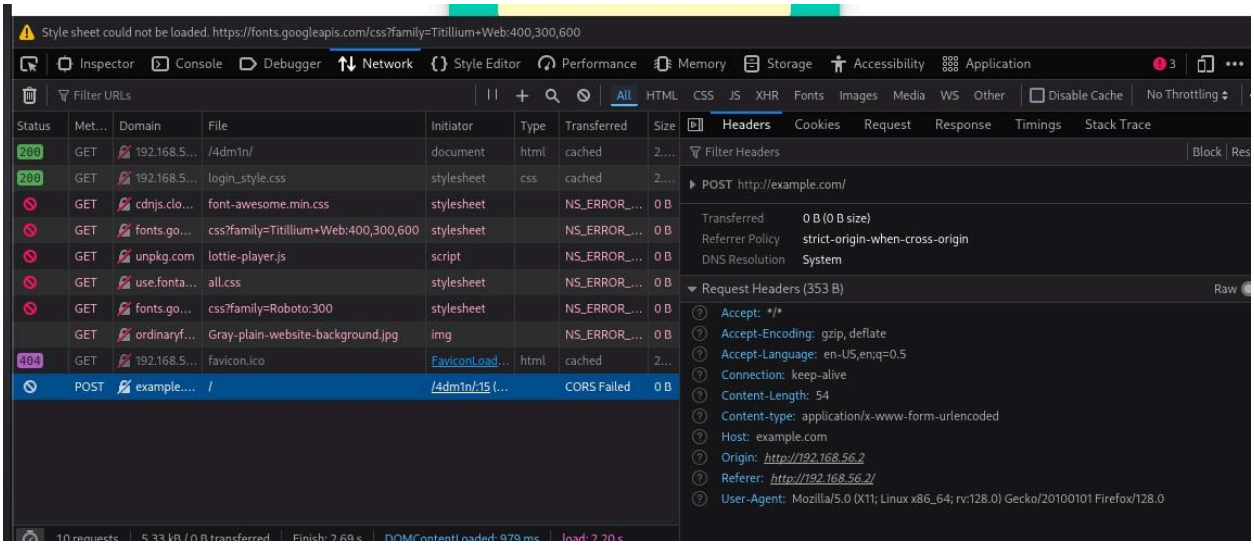
- If we can access the backend (for example, with Burp Suite), we can:
    - Try ***brute-forcing credentials*** since the admin page is now found.
    - Test for ***SQL injection***—try submitting ' OR '1'='1 or similar in either username or password to see if we can bypass authentication.
    - **Inspect the login** functionality using ***Burp Suite/ZAP*** to analyze requests, parameters, and responses for hints of vulnerabilities.

Brute Force Attack on admin login page

Confirm Form Details

From the HTML code:

- *Login URL*: `http://192.168.56.2/4dm1n/`
- *Fields*: username and password
- *Method*: Most typical login forms use POST. (The code shows an HTML <form> without an explicit method, but most browsers default to POST unless specified.)
- Manually test the login with an incorrect username/password. Note the error or response text (e.g., “Invalid username or password”, “Login failed”).
- This string will help Hydra know when a login attempt fails.



This is the screenshot from the developer tools → network tab after entering username and password.

Analysing the Network response after entering the login credentials

The screenshot of the browser's network tab shows that when the login fails, there isn't any error message returned to the web page, and the POST request goes to the dummy endpoint `http://example.com/` due to placeholder JavaScript code.

Key Points from the Developer Tools

- The actual login form (when submitted) triggers a POST to `http://example.com/` (seen in the request headers).
- This is not the real backend endpoint—it's hardcoded as a placeholder in the site's JavaScript for demonstration/testing.
- No useful response, no error message, no indicator of "success" or "failure" from the web server.

What This Means for Hydra

- Hydra needs to check the real backend login endpoint—not just this dummy request.
- Since the login form's backend doesn't appear to be implemented, brute-forcing with Hydra on `http://192.168.56.2/4dm1n/` will likely not work unless the actual server app processes POST requests there and returns meaningful responses.

Actions to Take

Confirm Backend Login Logic

- Lets try submitting the form with other values: Does the page ever change, reload, or display a message?
- Or try submitting while inspecting the network tab—do any requests go to `/4dm1n/` or another endpoint on the actual VM IP?

If /4dm1n/ Accepts POST:

- Hydra can be set up to brute-force. In the absence of a failure string, use HTTP status codes by setting the "fail" string to something generic or use Hydra's option to check for differing lengths/responses.

Example command:

`As hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.56.2 http-post-form "/4dm1n/:username=^USER^&password=^PASS^:S=200"`
(Here, S=200 just looks for a successful HTTP response (change if needed).

Run and Monitor

- Run Hydra and watch for successful logins.
- If successful, Hydra will display the username and password.
- Log in manually to confirm access and search for new flags, sensitive data, or admin functionality.

Summary:

- Use Hydra with the admin login URL, fields, and ready dictionary.
- Adjust "F=Login failed" with the actual failure response.
- On a successful brute-force, use discovered credentials for further testing.

If Login Not Actually Implemented:

- The login form seems to be intentionally nonfunctional for this exercise.
- Since this is the case, we focus on other vulnerabilities (e.g., exposed flags, directory listings, FTP access, or try for SQL injection if the form has any backend logic

Check Open Port FTP

Step 1: Connect to FTP Server

Run:
`ftp 192.168.56.2`

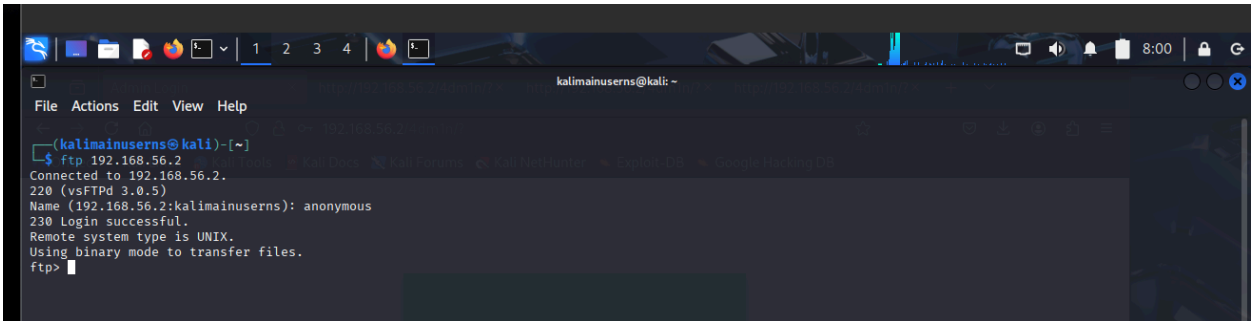
We see a message `Connected to 192.168.56.2`.

Step 2: Login Anonymously

When prompted for username, enter:

`anonymous`
When prompted for password, enter any string or the email address (just press Enter or type anything).

We should see a `login success` message.



Step 3: List Files and Directories

Run:

`ls`

This lists files and directories accessible with anonymous login.
If ls fails, try:

`dir`
Note down all files and directories listed.

Note: Sometimes we see an `Entering Extended Passive Mode` message. It indicates that the FTP client and server are negotiating passive mode for data transfer. In Extended Passive Mode (EPSV), the server tells the client which port to use for the data channel that is used to transfer directory listings and files. EPSV is compatible with both IPv4 and IPv6 and works better through firewalls and NATs compared to traditional active FTP mode. The client then establishes a data connection to the server's specified port rather than the server connecting back to the client.

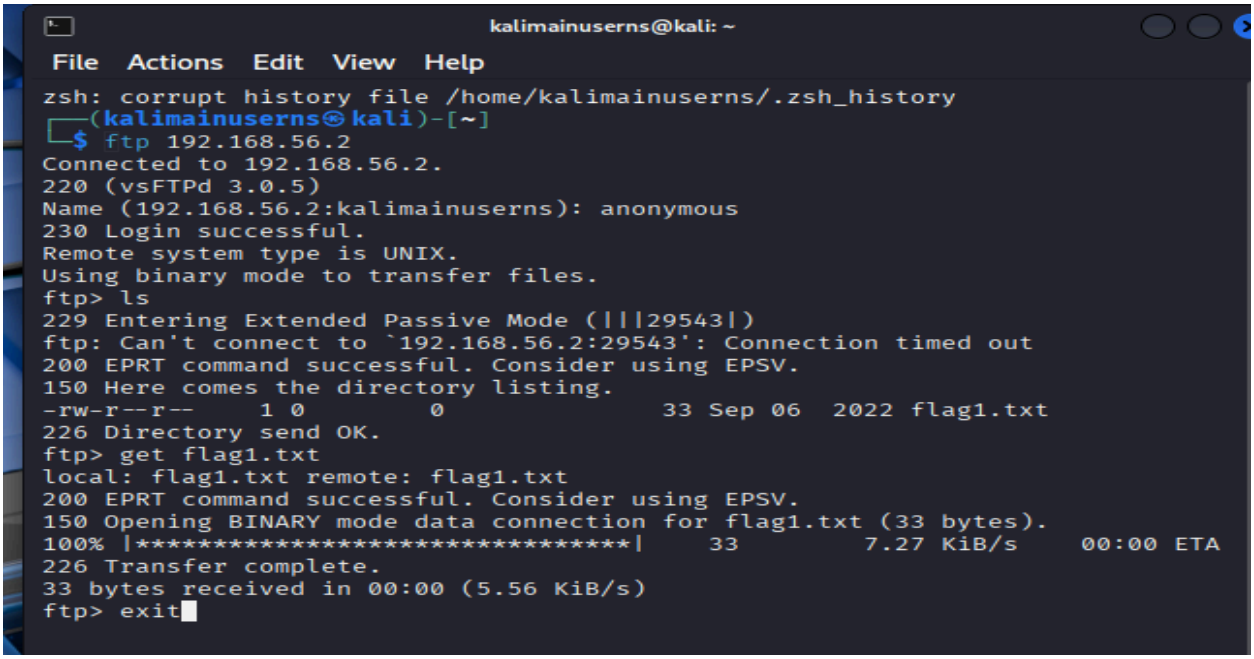
Step 4: Navigate and Download Files

Change directory:

`cd <directory_name>`

Download a file:

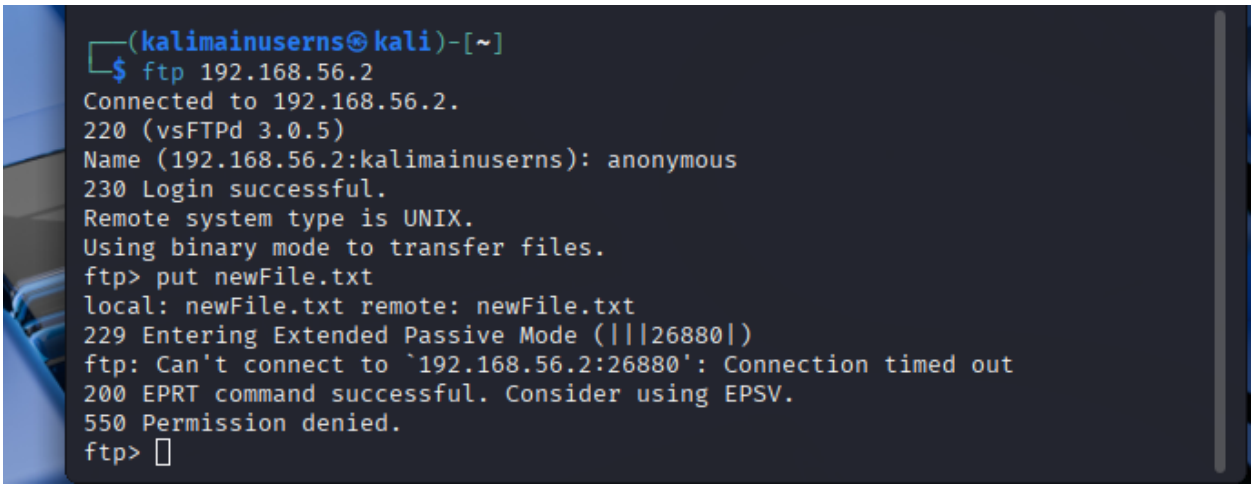
`get <filename>`



Step 5: Upload File (Test Write Permission)

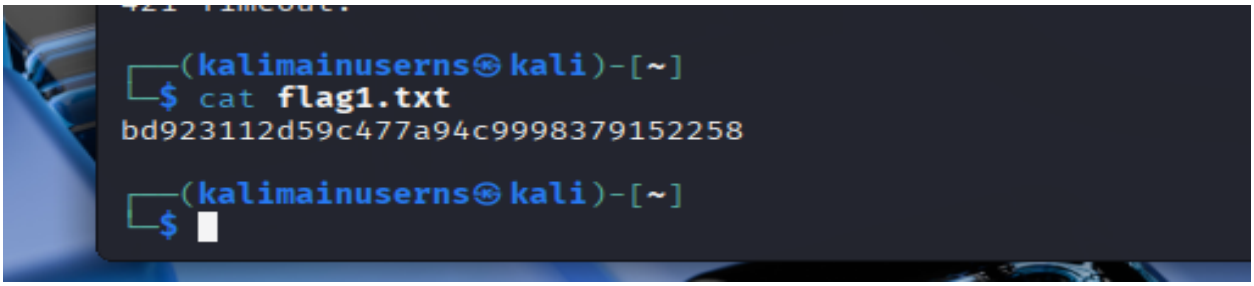
Check if we can upload (write permissions):
`put <local_test_file>`

If upload succeeds, the server is misconfigured and vulnerable.



Step 6: Summary of Vulnerabilities

- *Anonymous login allowed*: Sign of weak FTP password policy.
- *Directory listing enabled:* Allows attackers to gather info on available files.
- *No Upload permission:* Anonymous users do NOT have write/upload permission on this FTP server. The 550 Permission denied error confirms the server is configured to block uploads by users logged in as “anonymous.”
- *Anonymous users DO have read/download permission*, since we were able to fetch and download existing files
- *Downloadable sensitive files*: Lets Look for config files, backups, credentials. After listing all hidden files we didn’t find anything.



We found *flag 1 → bd923112d59c477a94c9998379152258*

Step 7: Exit FTP

Run:
`bye`

To quit the FTP session.

Burpe Suite

Burp Suite Introduction

- Burp Suite is an integrated platform for performing security testing of web applications.
- It is commonly used by pentesters to intercept, inspect, and modify web traffic between their browser and a target web application.
- With Burp Suite, we can find and exploit vulnerabilities like XSS, SQL injection, broken authentication, and more.

Why Burp Suite?

- It acts as an intercepting proxy to capture and modify HTTP/HTTPS traffic.
- Burp Suite is called a proxy because its core function is to act as an HTTP proxy server that sits between the browser and the target website, allowing us to intercept, inspect, and modify the network traffic in real-time. This intermediary role enables security testers to analyze requests and responses, revealing how applications interact with servers and identifying potential vulnerabilities.
- Provides automated vulnerability scanning.
- Allows detailed manual testing of web requests/responses.
- Helps in identifying security flaws by observing web app behavior in real-time

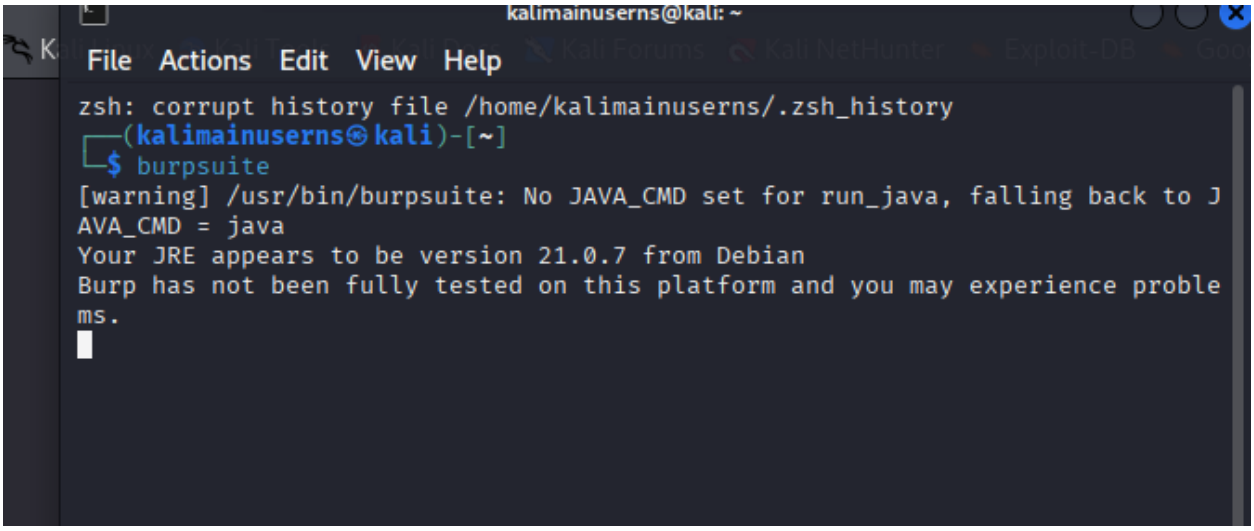
Where is it Used?

- Web application penetration testing engagement.
- Application security assessments.
- Ethical hacking CTF labs and real-world security audits.

Step-by-Step Guide to Set Up Burp Suite for Penetration Testing

Launch Burp Suite on Kali Linux

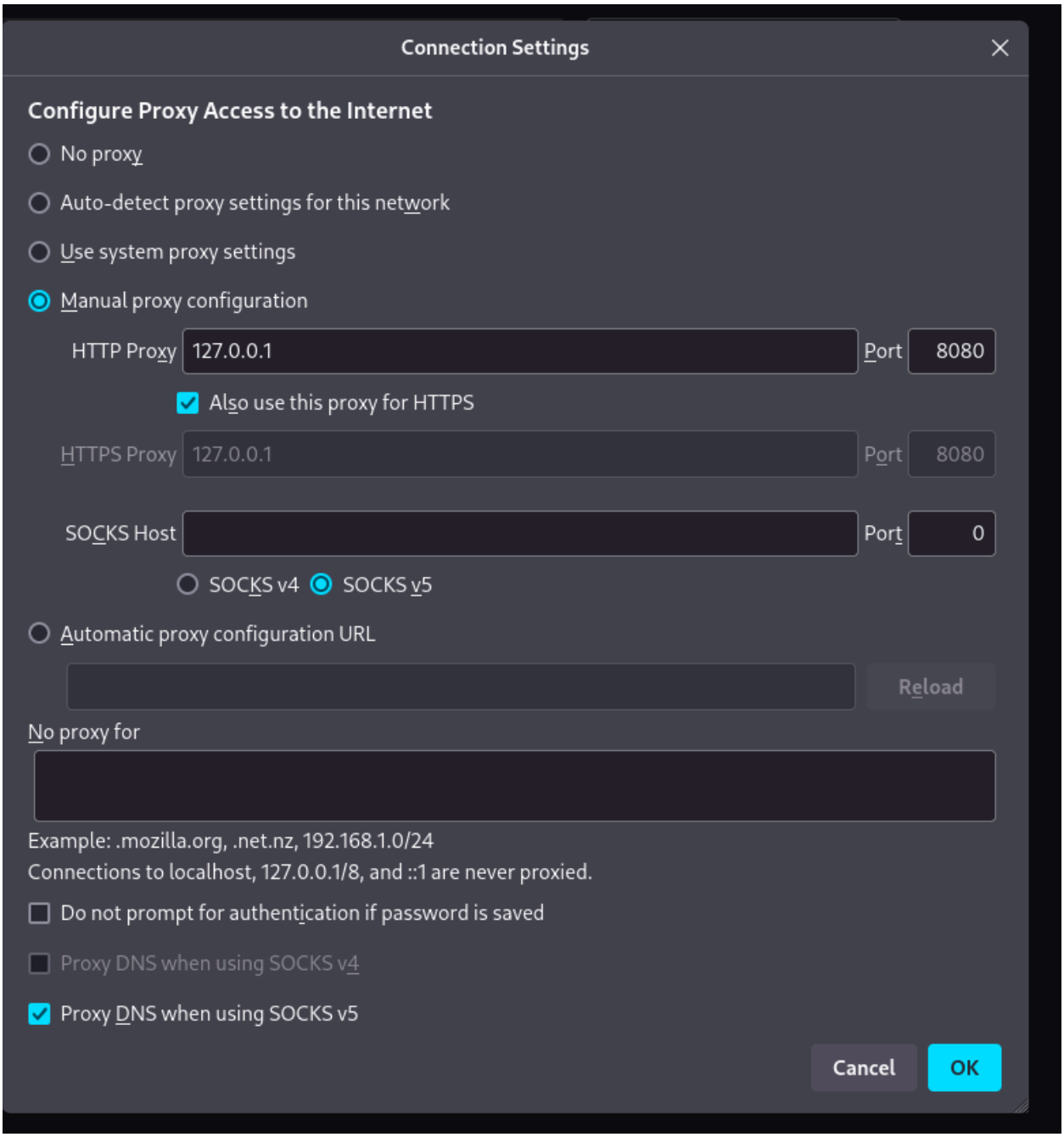
- Open a terminal or use the Kali Linux menu to find and start Burp Suite by typing:
  `burpsuite`
- The Burp Suite application window will open. Choose Temporary Project > Start Burp.



Configure Browser to Use Burp as Proxy

For Firefox (default browser in Kali):

- Open Firefox and go to Preferences.
- Scroll down to Network Settings.
- Click Settings... to open Connection Settings.
- Select Manual proxy configuration.



In HTTP Proxy, enter:

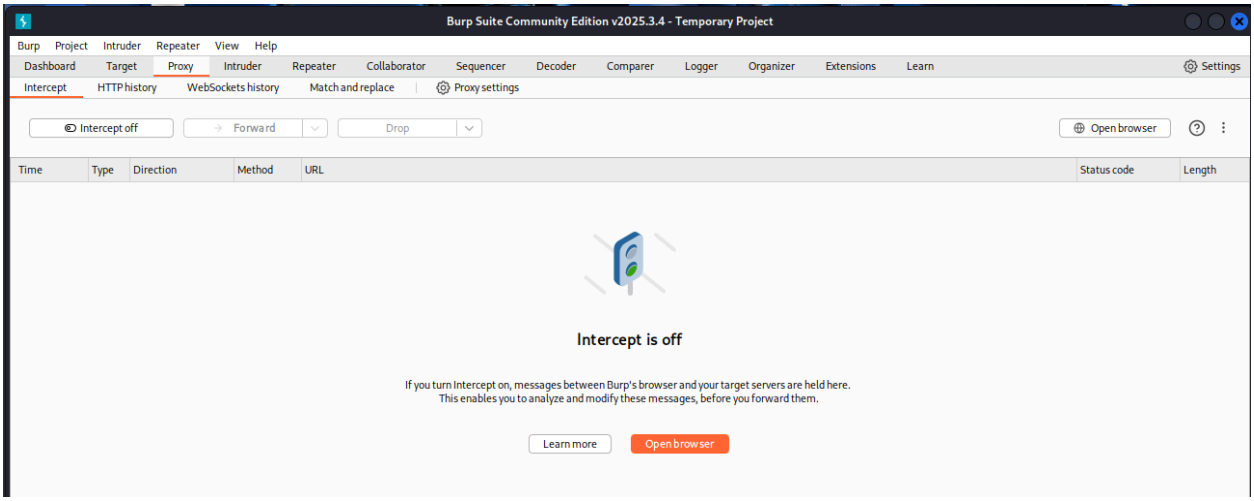
`127.0.0.1`

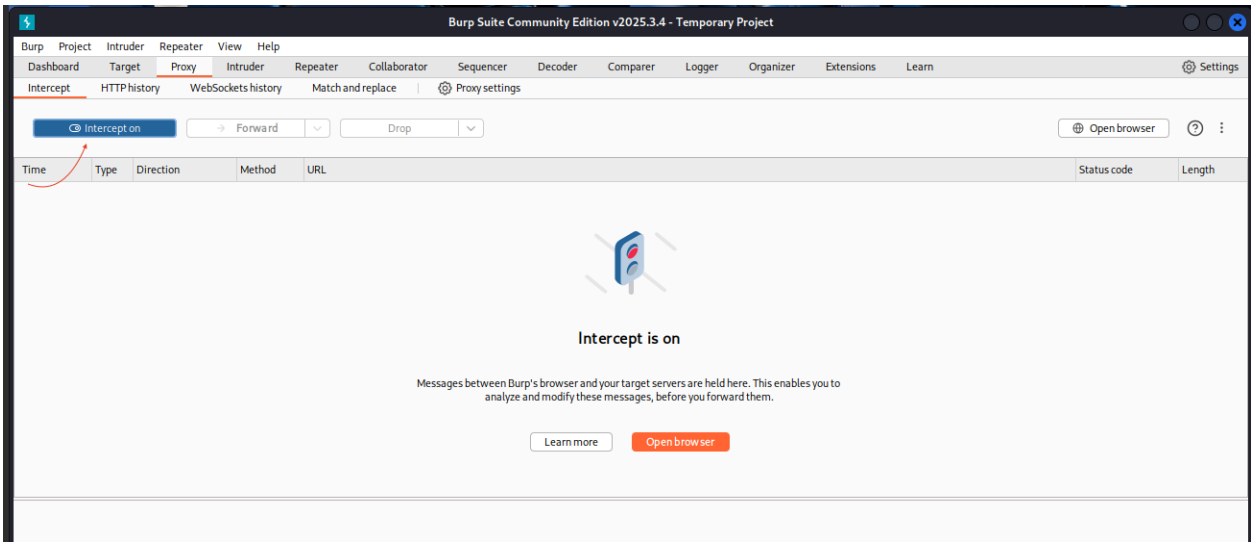
- In the Port field next to HTTP Proxy, enter:
  `8080`
- Check the box Use this proxy server for all protocols.

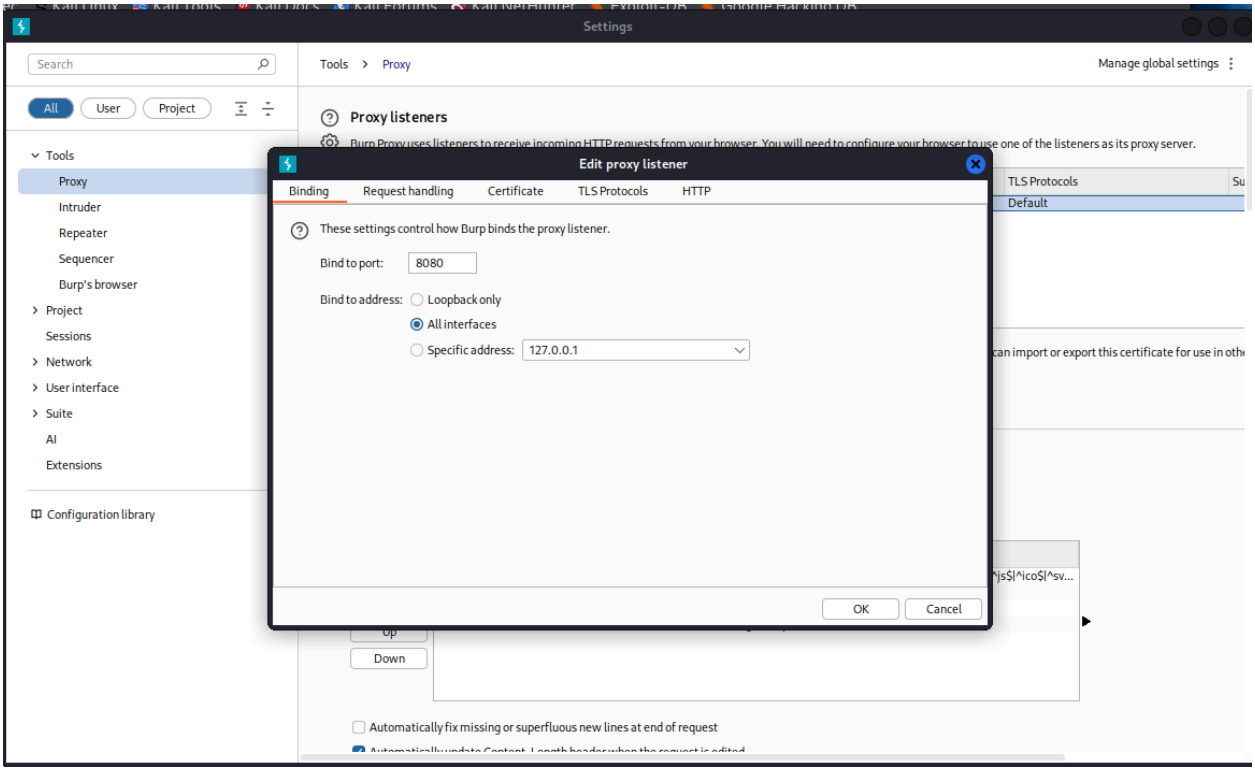
- Click `OK` to save settings.

Verify Proxy Configuration

- In Burp Suite, navigate to the Proxy tab and make sure `Intercept` is turned `ON`.
- Open a webpage in Firefox (e.g., `http://192.168.56.2`) and watch Burp capture the request.
- If Burp intercepts the HTTP request, the proxy is correctly set up.




Burp Suite Community Edition v2025.3.4 - Temporary Project
Burp Project Intruder Repeater View Help
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Settings
Intercept HTTP history WebSockets history Match and replace Proxy settings
Intercept on Forward Drop Open browser
Time Type Direction Method URL Status code Length
Intercept is on
Messages between Burp's browser and your target servers are held here. This enables you to analyze and modify these messages, before you forward them.
Learn more Open browser

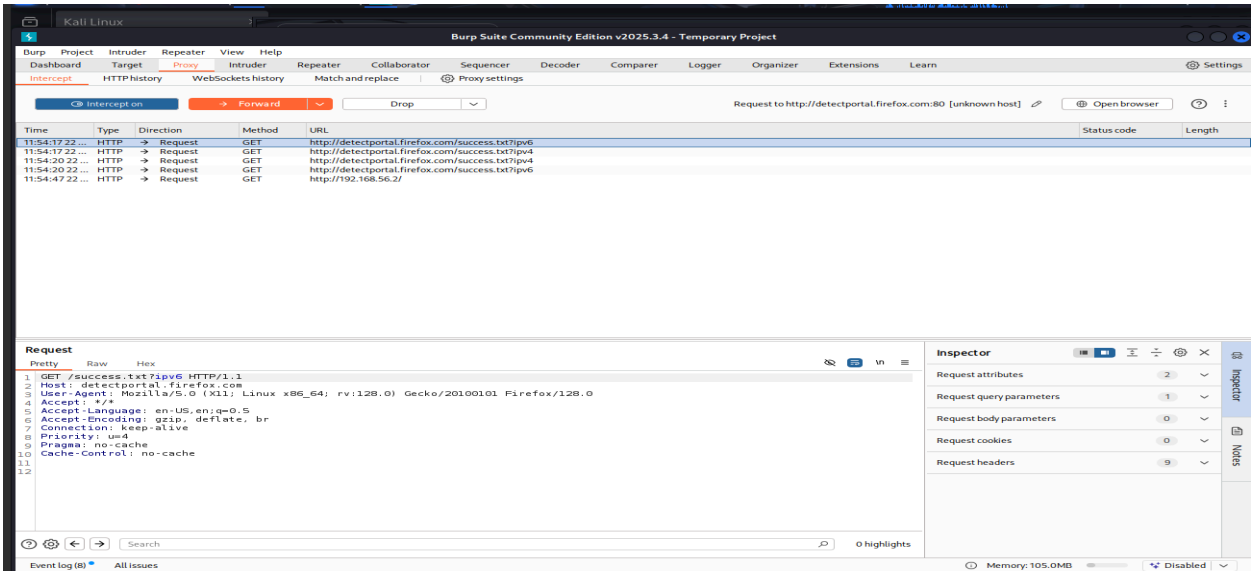

Settings
Search
All User Project
Tools
Proxy
Intruder
Repeater
Sequencer
Burp's browser
Project
Sessions
Network
User interface
Suite
AI
Extensions
Configuration library
Tools > Proxy
Manage global settings
Proxy listeners
Edit proxy listener
Binding Request handling Certificate TLS Protocols HTTP
These settings control how Burp binds the proxy listener.
Bind to port: 8080
Bind to address: Loopback only
All interfaces
Specific address: 127.0.0.1
TLS Protocols
Default
OK Cancel
Down
Automatically fix missing or superfluous new lines at end of request

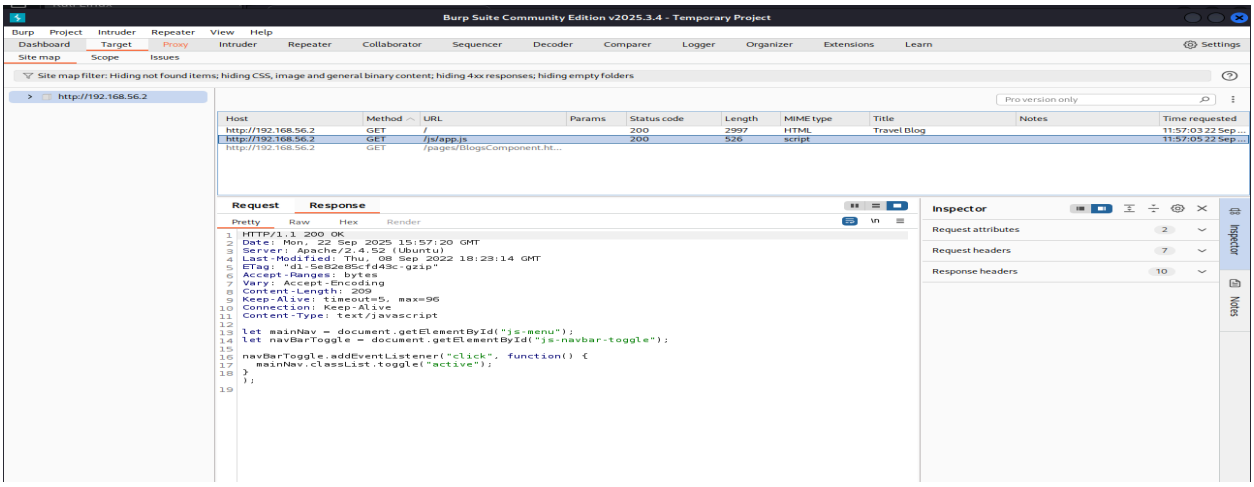
Notes for HTTPS Sites

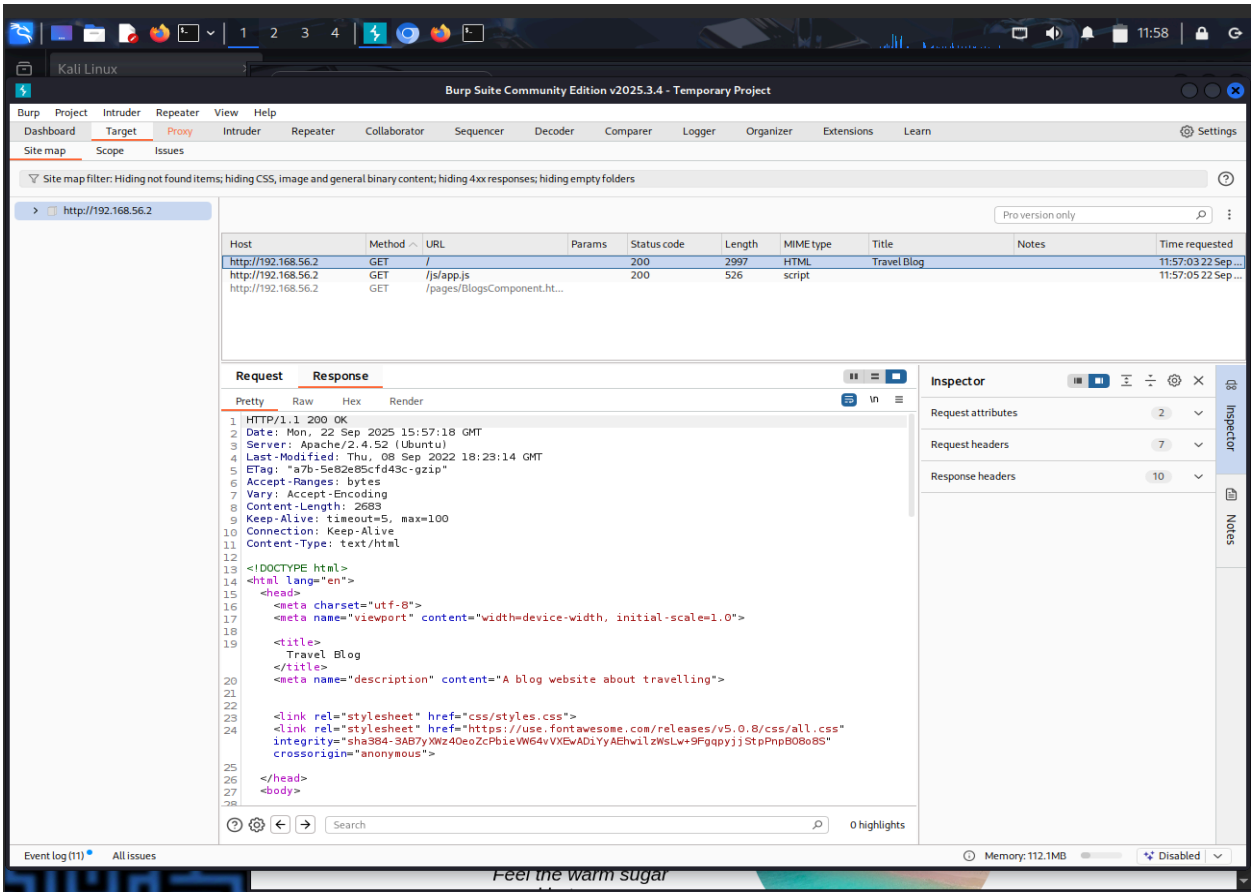
- For HTTPS traffic interception, install Burp's CA certificate in the browser to avoid SSL errors.
  Note: For our local vulnerable VM on HTTP, this is not needed.

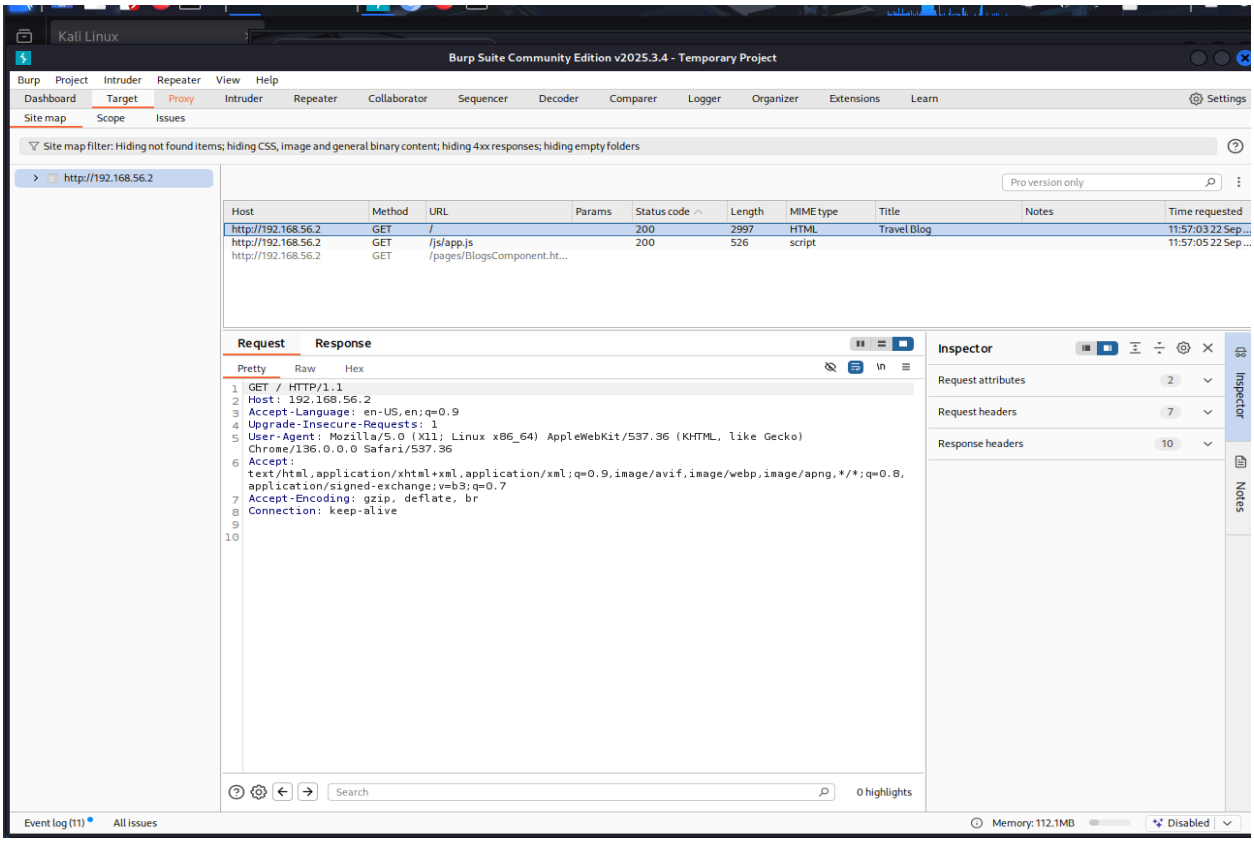
Intercept Traffic

- In Burp, go to `Proxy > Intercept tab` and ensure the intercept is ON.
- Browse to the target web application, for example `http://192.168.56.2`.
- Burp will pause HTTP requests and allow us to review and modify them before forwarding.




Burp Suite Community Edition v2025.3.4 - Temporary Project
Request
Response
Inspector


Kali Linux
Burp Suite Community Edition v2025.3.4 - Temporary Project
Request
Response
Inspector
Feel the warm sugar


Kali Linux
Burp Suite Community Edition v2025.3.4 - Temporary Project
Burp Project Intruder Repeater View Help
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Settings
Site map Scope Issues
Site map filter: Hiding not found items; hiding CSS, image and general binary content; hiding 4xx responses; hiding empty folders
http://192.168.56.2
Pro version only
Host Method URL Params Status code Length MIME type Title Notes Time requested
http://192.168.56.2 GET / 200 2997 HTML Travel Blog 11:57:03 22 Sep ...
http://192.168.56.2 GET /js/app.js 200 526 script 11:57:05 22 Sep ...
http://192.168.56.2 GET /pages/BlogsComponent.ht...
Request Response
Pretty Raw Hex
1 GET / HTTP/1.1
2 Host: 192.168.56.2
3 Accept-Language: en-US,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Connection: keep-alive
9
10
Search
0 highlights
Inspector
Request attributes 2
Request headers 7
Response headers 10
Inspector
Notes
Event log (11)
All issues
Memory: 112.1MB
Disabled

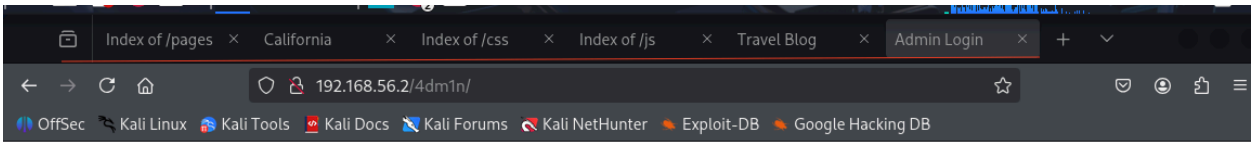
Manual Crawling

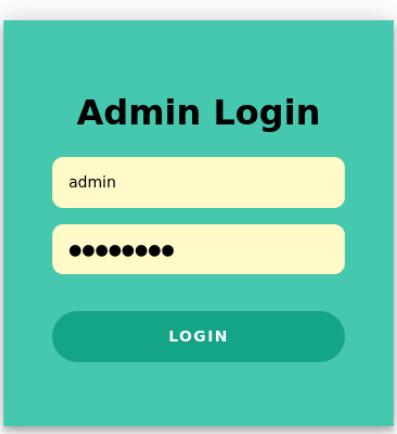
The Burp Suite Professional Edition comes with Crawler or Spider. And since we are using the Community edition - that feature is not available and so we will attempt manual crawling Techniques.

Browse the Application Like a User

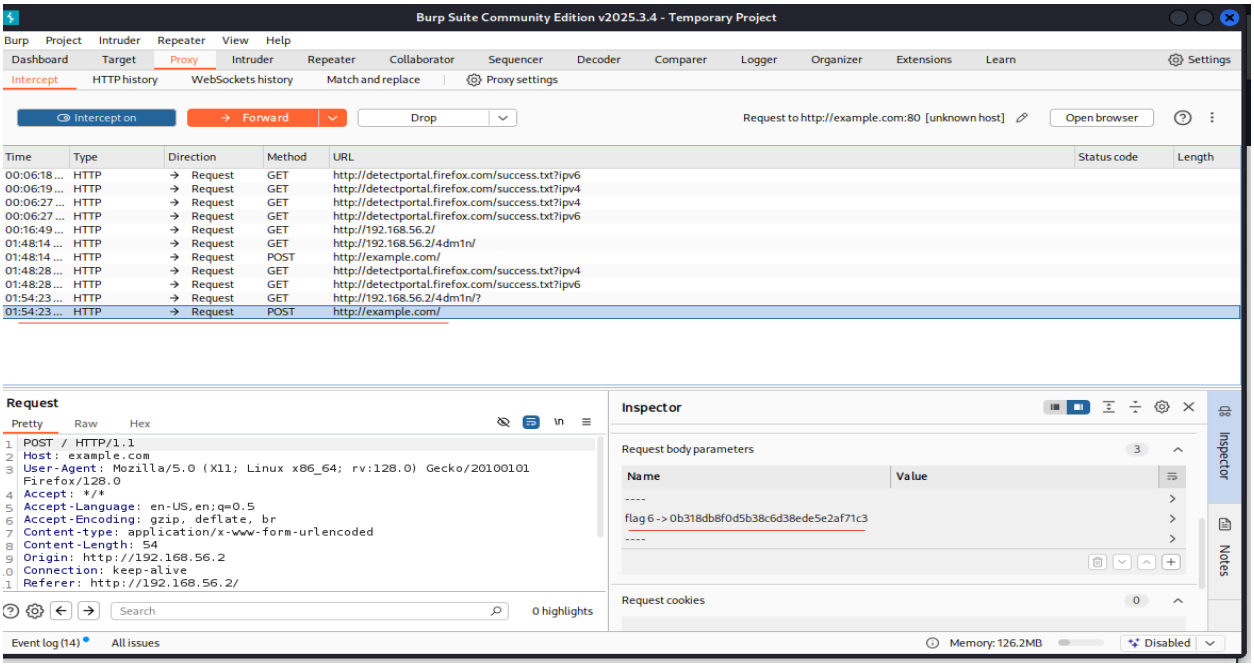
- With Burp running, visit all pages of the target vulnerable application in the browser.
- Manually click every link, menu item, and accessible feature.
- Fill out and submit every form with test data, trying different combinations.

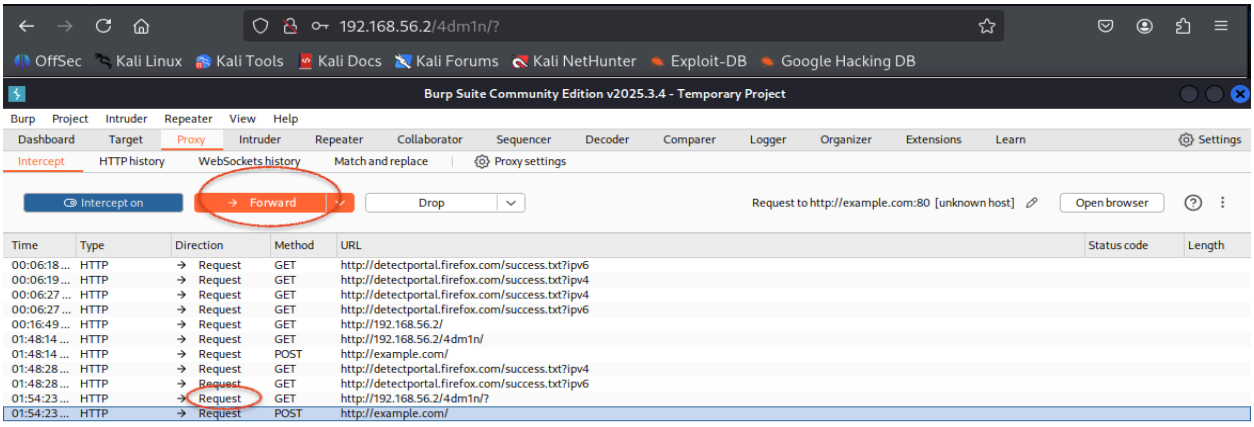
Admin Page

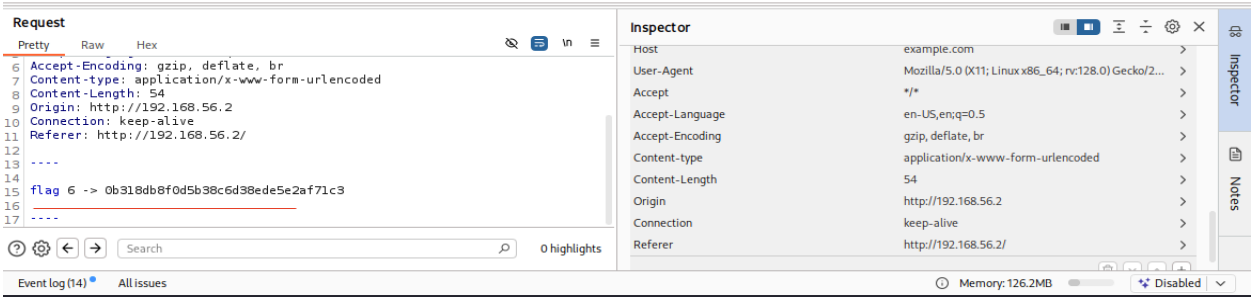


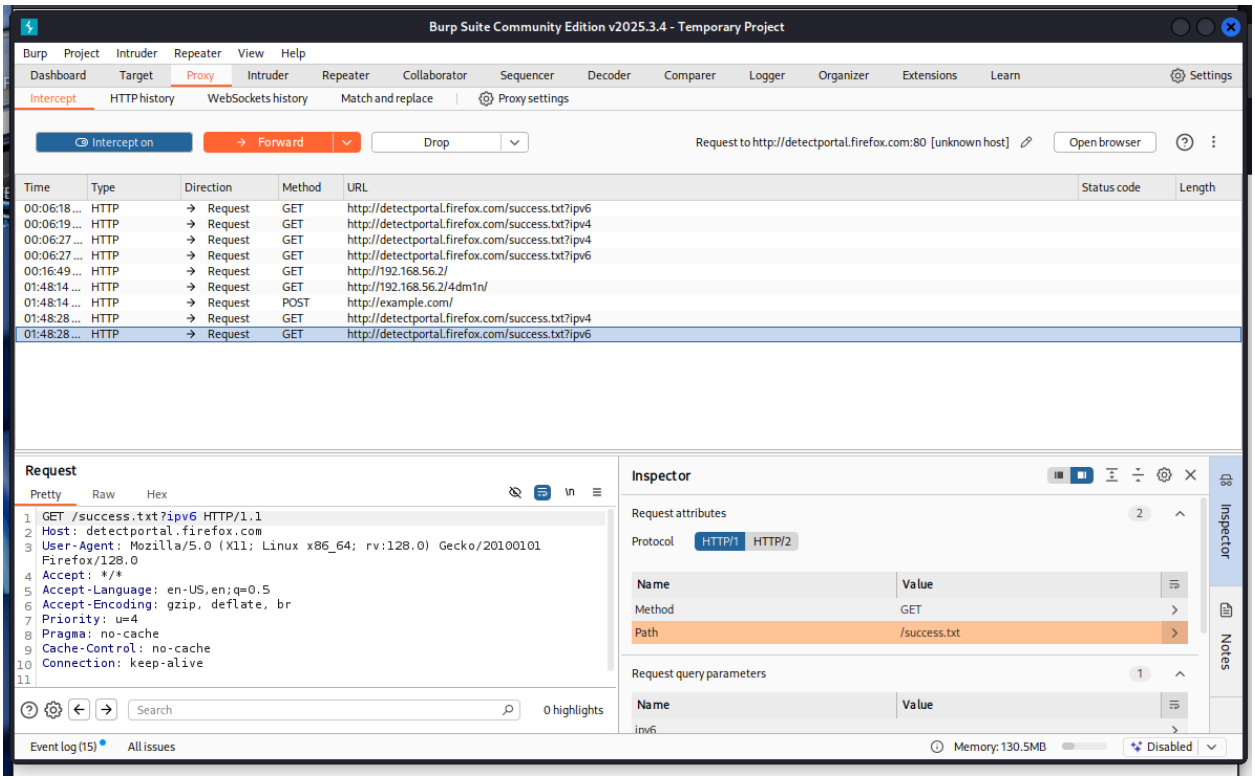


- In the Burp Suit → Proxy tab - Inspect details of each page one by one.
- Enter the url in browser: `http://192.168.56.2/4dm1n`


Burp Suite Community Edition v2025.3.4 - Temporary Project
Request
Inspector
POST / HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-type: application/x-www-form-urlencoded
Content-Length: 54
Origin: http://192.168.56.2
Connection: keep-alive
Referer: http://192.168.56.2/
Request body parameters
Name
Value
flag 6 -> 0b318db8f0d5b38c6d38ede5e2af71c3
Request cookies


192.168.56.2/4dm1n/?
Burp Suite Community Edition v2025.3.4 - Temporary Project
Intercept on
Forward
Drop
Request to http://example.com:80 [unknown host]
Open browser


Request
Inspector
Accept-Encoding: gzip, deflate, br
Content-type: application/x-www-form-urlencoded
Content-Length: 54
Origin: http://192.168.56.2
Connection: keep-alive
Referer: http://192.168.56.2/
flag 6 -> 0b318db8f0d5b38c6d38ede5e2af71c3
Host
User-Agent
Accept
Accept-Language
Accept-Encoding
Content-type
Content-Length
Origin
Connection
Referer

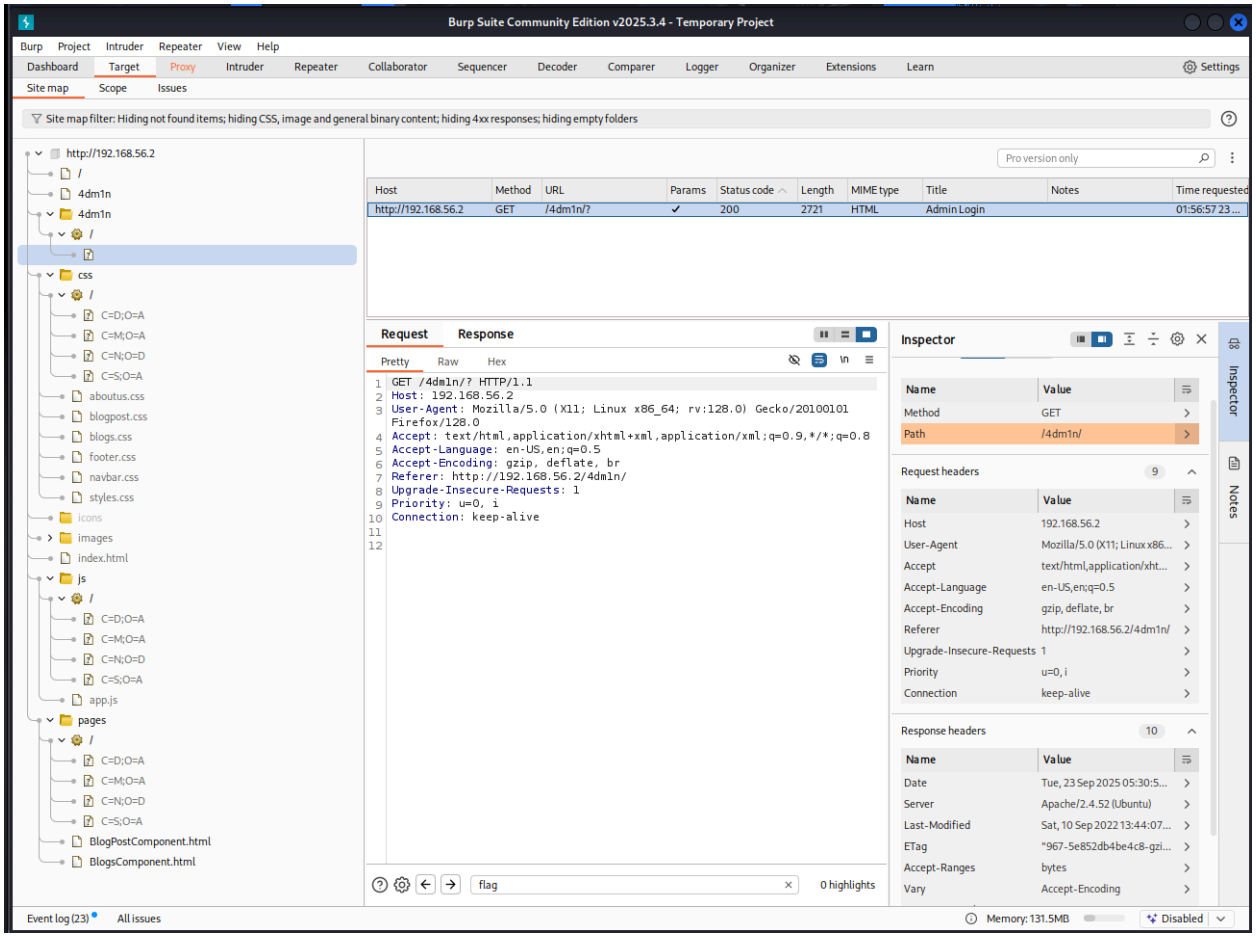


- Click on forward button
- Next the page admin page will open in the browser
- Once that opens enter the username password (guess it)
- Click on Login button on the admin web page
- Look at the Inspector section of the Burp suite
- We can find the flag 6 (possible sensitive information will be found here) as proven by the screenshot.
  `flag 6 -> 0b31bd8f0d5b38c6d38ede5e2af71c3`

Monitor the Site Map in Burp

- Go to the Target tab in Burp Suite.
- Under Site map, expand the target host (e.g., `http://192.168.56.2`).
- As we manually browse, new URLs, parameters, and forms will appear automatically in the tree.

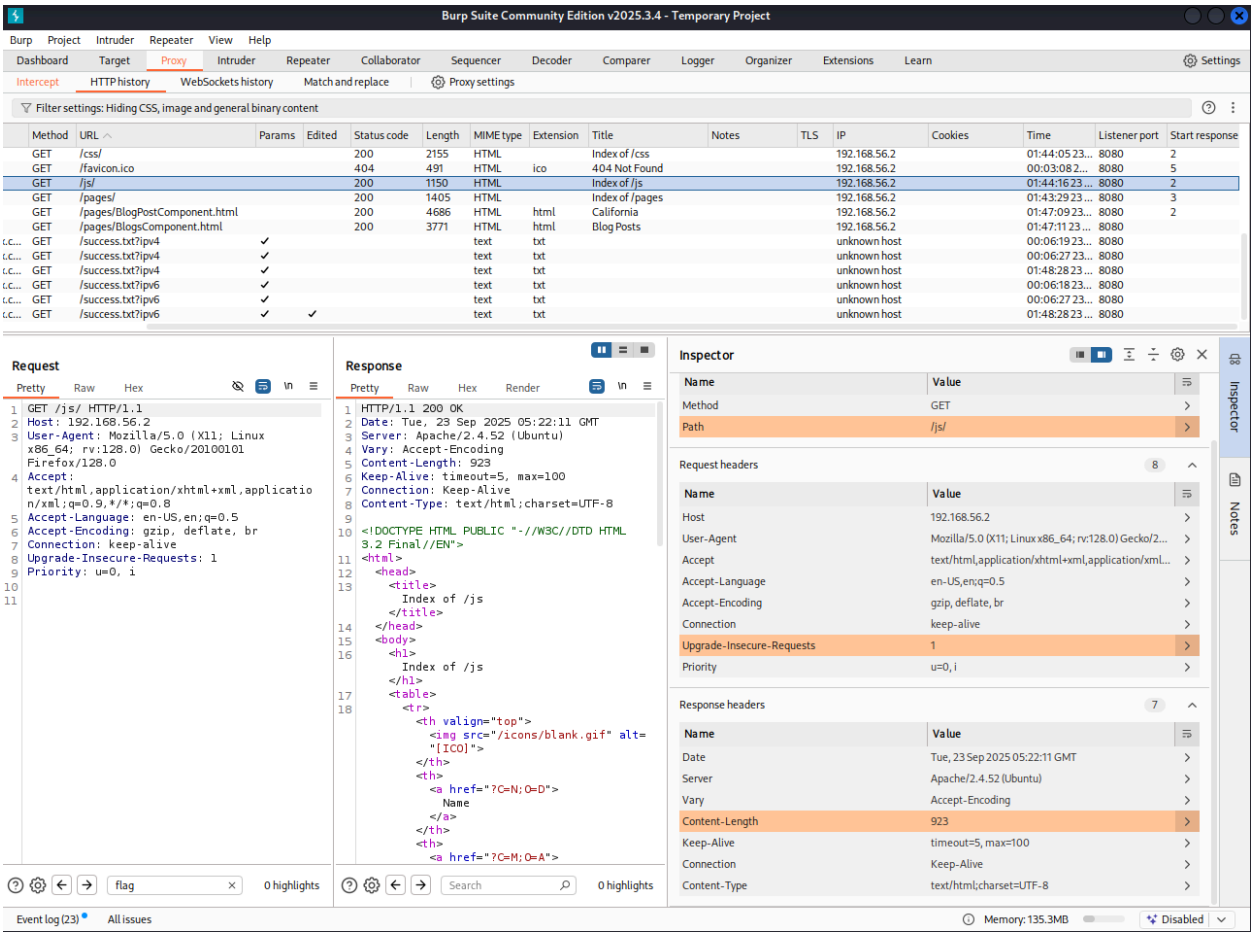
- Each page/component touch is captured for later review and



attack.

Use Burp's "Proxy history" for Deeper Review

- In Proxy > HTTP history, see every request/response captured by Burp, including details like parameters, cookies, headers, HTTP methods, and server responses.
- Send interesting requests to Burp's Repeater or Intruder for further testing.



Notes

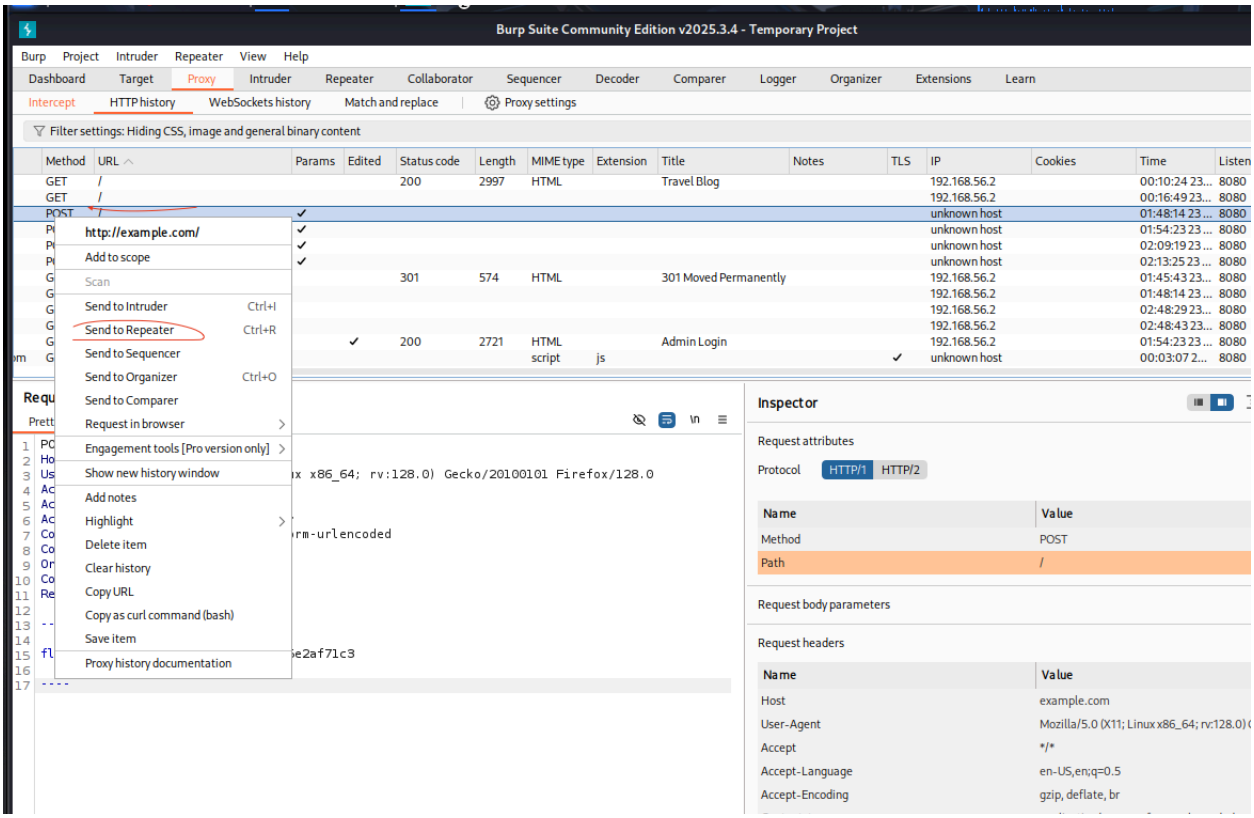
- Manual crawling is crucial for authentication-required or JavaScript-heavy apps where automated crawlers often miss things.
- Each dynamic element discovered becomes a potential attack vector for vulnerabilities (XSS, SQLi, CSRF, IDOR).

4adm1n

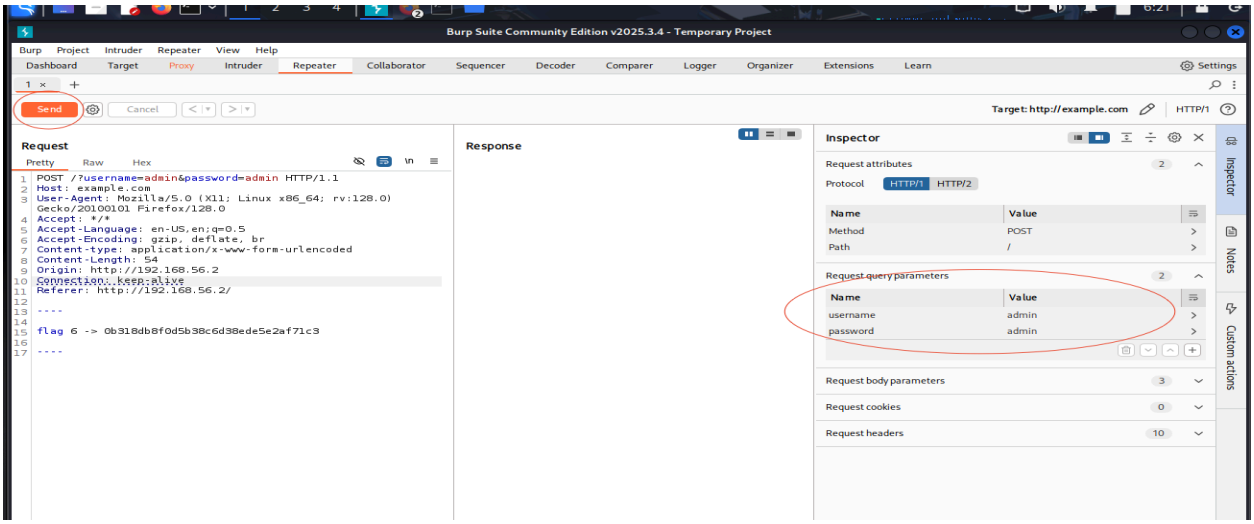
- Login form, POST captured
- test SQLi, weak/default creds, bypass
- Use Intruder/Repeater to fuzz parameters

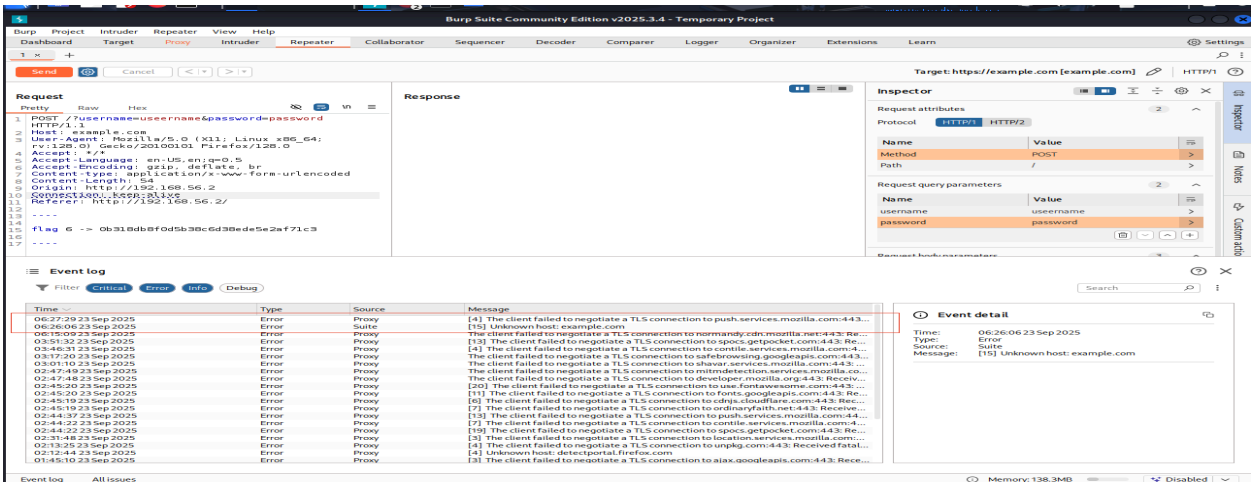
How to test?

- Send the Captured Request to Repeater
- In Burp's Proxy > HTTP history, right-click the POST request (the login attempt).
- Select Send to Repeater.
- Switch to the Repeater tab.



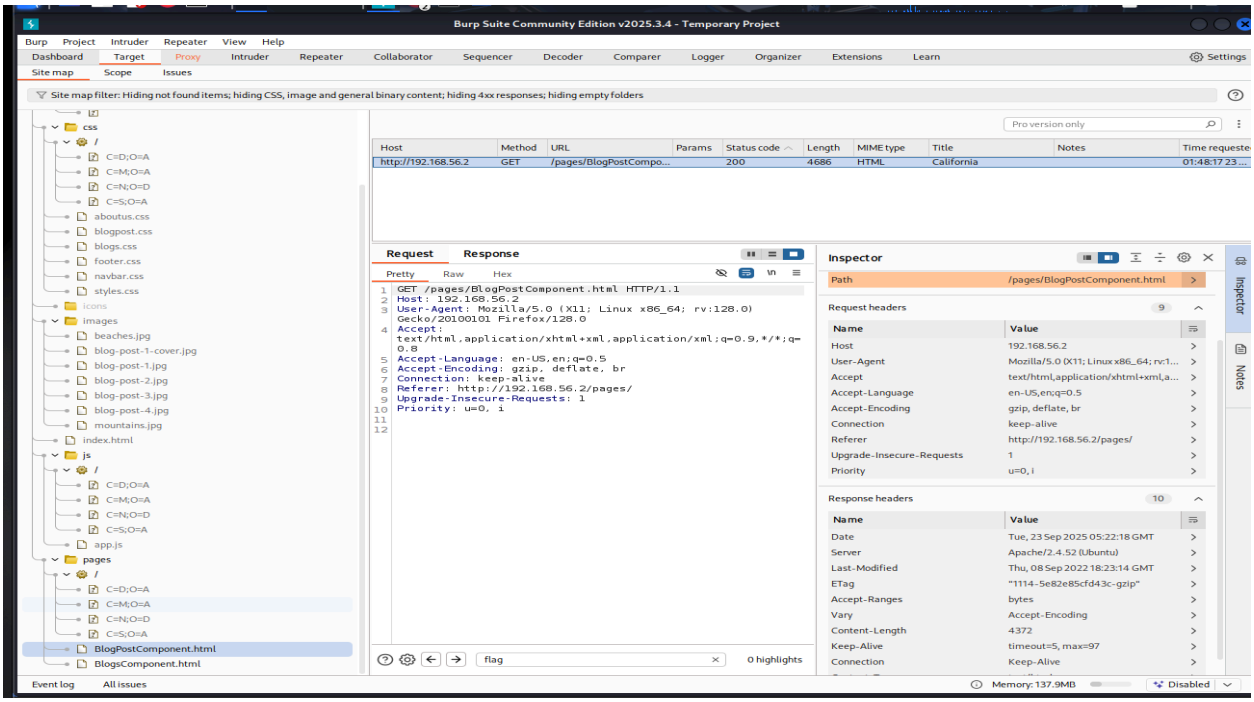
- In the repeater → inspector window → request query parameter → click add → enter the username and password –> click send.
- Check response. There is no response; rather when we open the event log we see the error that is generated. It shows an unknown host error.
- In Burp Suite, an "`unknown host`" error means that the target hostname cannot be resolved to an IP address, preventing the connection. So we have a dummy target. And so we cannot check sql injection or such attacks and we move to the next page.

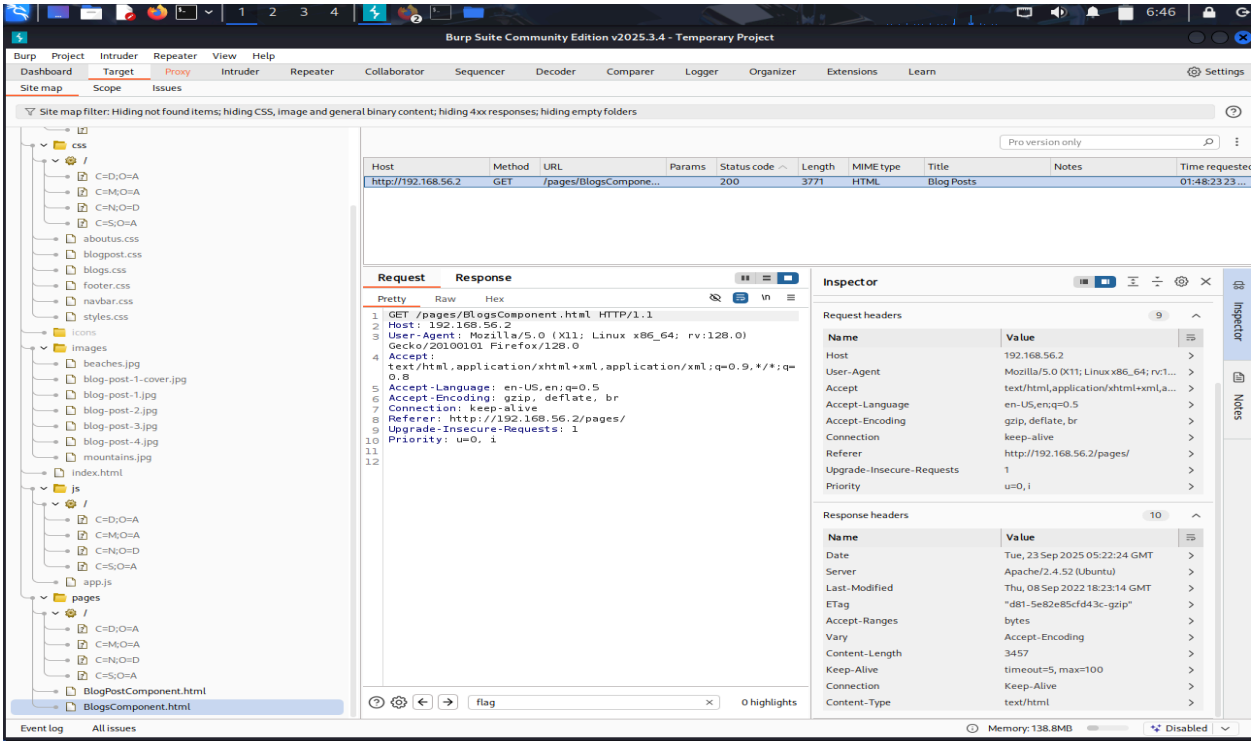




/pages/*.html

- Static blog pages with possible hidden info/comments
- Manual source/code review for flags, info


Burp Suite Community Edition v2025.3.4 - Temporary Project
Request
Response
Inspector
GET /pages/BlogPostComponent.html HTTP/1.1
Host: 192.168.56.2
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://192.168.56.2/pages/
Upgrade-Insecure-Requests: 1
Priority: u=0, i


Burp Suite Community Edition v2025.3.4 - Temporary Project
Request
Response
Inspector
GET /pages/BlogsComponent.html HTTP/1.1
Host: 192.168.56.2
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://192.168.56.2/pages/
Upgrade-Insecure-Requests: 1
Priority: u=0, i

Analysis of /pages/*.html Content

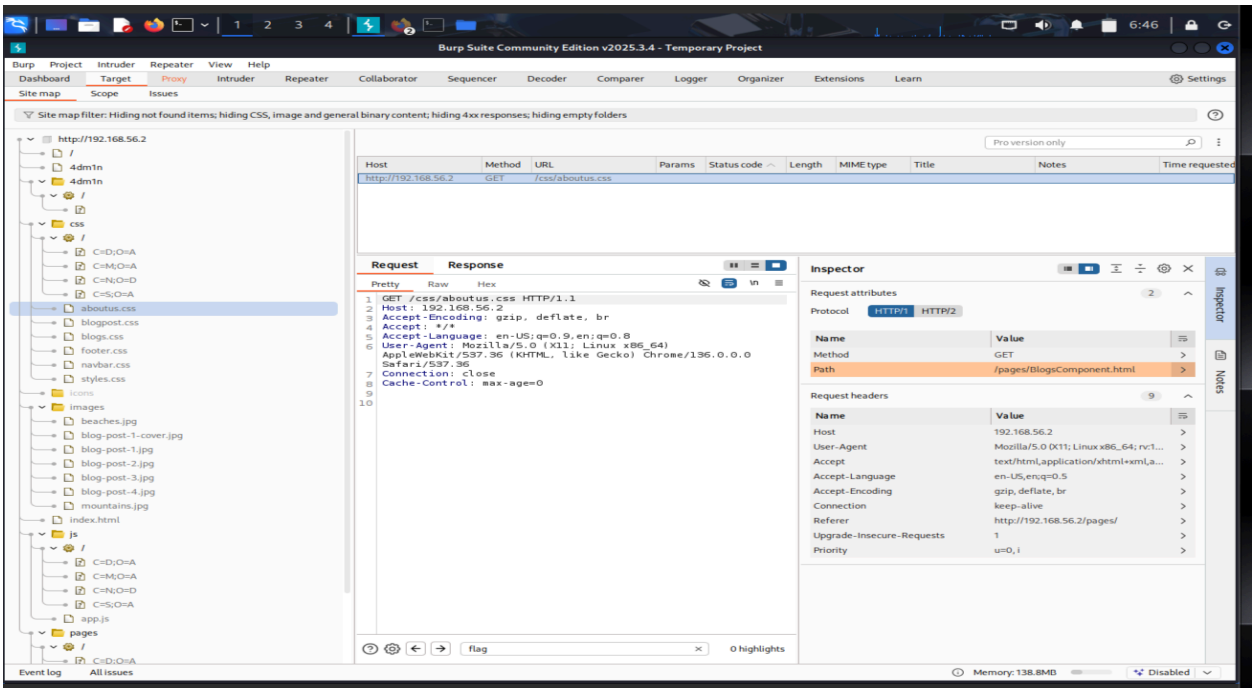
- The Burp requests show GETs to these HTML files, returning status 200 and presenting standard headers (e.g., Server: Apache/2.4.52 (Ubuntu)) and normal browser traffic.
- The responses are pure HTML; neither request or headers contain evidence of authentication challenge, sensitive cookies, nor backend API calls.

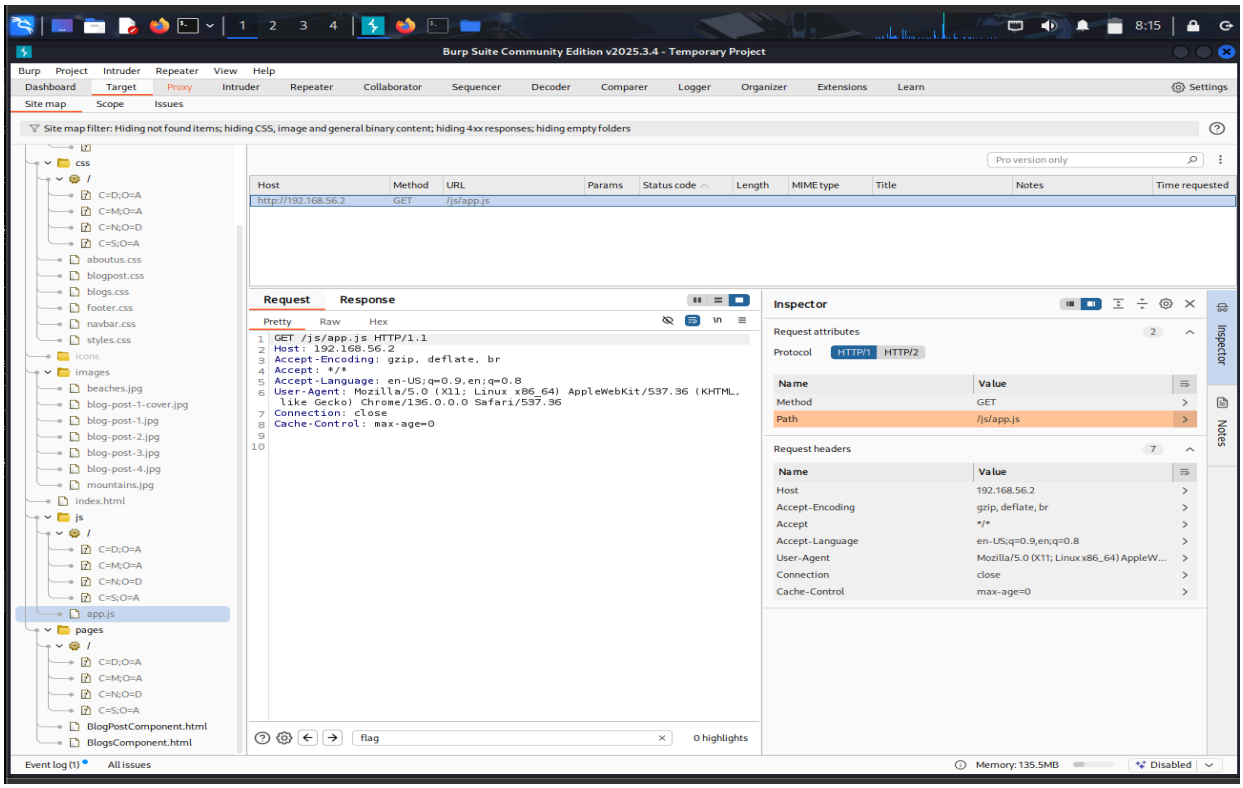
Hidden Information Assessment

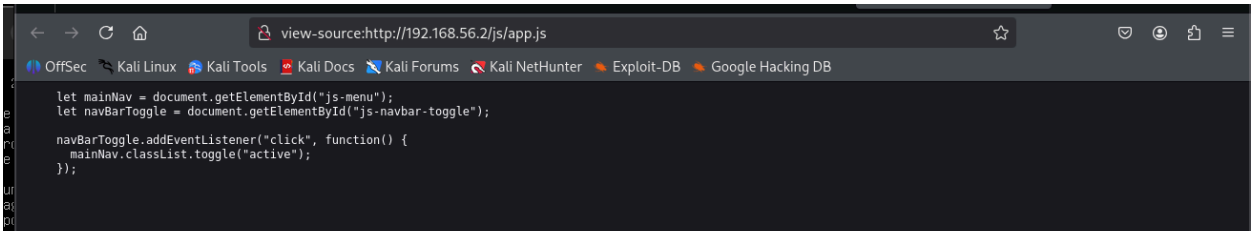
- No Indication of Sensitive Data in Headers: The HTTP responses for these requests do not return cookies (e.g., no PHPSESSID, JWT, etc.) and have normal content-type and server data.
- File Names and Paths: The file names themselves (component-style .html files) do not reveal user credentials, tokens, or config info. These appear to be static blog component pages typical of a beginner-level vulnerable app.
- No Extra Parameters or Query Strings: Requests do not include suspicious or interesting query strings which sometimes indicate dynamic/back-end interaction that might leak info.

`/css/, /js/`

- Public static files. JS can leak endpoints, functionality
- Inspecting for hidden functionality/endpoints


Burp Suite Community Edition v2025.3.4 - Temporary Project
Site map filter: Hiding not found items; hiding CSS, image and general binary content; hiding 4xx responses; hiding empty folders
http://192.168.56.2
/css/aboutus.css
Request
Response
GET /css/aboutus.css HTTP/1.1
Host: 192.168.56.2
Accept-Encoding: gzip, deflate, br
Accept: */*
Accept-Language: en-US;q=0.9,en;q=0.8
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36
Connection: close
Cache-Control: max-age=0
Inspector
Request attributes
Protocol
HTTP/1
HTTP/2
Name
Value
Method
GET
Path
/pages/BlogsComponent.html
Request headers
Host
192.168.56.2
User-Agent
Accept
Accept-Language
en-US,en;q=0.5
Accept-Encoding
gzip, deflate, br
Connection
keep-alive
Referer
http://192.168.56.2/pages/
Upgrade-Insecure-Requests
1
Priority
u=0, i
flag
0 highlights
Memory: 138.8MB
Disabled


Burp Suite Community Edition v2025.3.4 - Temporary Project
Site map filter: Hiding not found items; hiding CSS, image and general binary content; hiding 4xx responses; hiding empty folders
http://192.168.56.2
/js/app.js
Request
Response
GET /js/app.js HTTP/1.1
Host: 192.168.56.2
Accept-Encoding: gzip, deflate, br
Accept: */*
Accept-Language: en-US;q=0.9,en;q=0.8
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36
Connection: close
Cache-Control: max-age=0
Inspector
Request attributes
Protocol
HTTP/1
HTTP/2
Name
Value
Method
GET
Path
/js/app.js
Request headers
Host
192.168.56.2
Accept-Encoding
gzip, deflate, br
Accept
/
Accept-Language
en-US;q=0.9,en;q=0.8
User-Agent
Connection
close
Cache-Control
max-age=0
flag
0 highlights
Memory: 135.5MB
Disabled



Analysis of /css/ and /js/app.js

- CSS Files (/css/aboutus.css)
  - Request: Standard GET request to aboutus.css
  - Response: Status 200, normal CSS content-type headers
  - Finding: No sensitive information visible in the request/response headers or traffic patterns
- JavaScript File (/js/app.js)
  - Request: Standard GET request to app.js
  - Response: Status 200, standard JavaScript content-type
  - Finding: No sensitive information visible in the HTTP traffic itself
- Analysis of Js file code - It contains Frontend code for
  - Navigation Toggle: Simple menu toggle functionality for a responsive navigation bar
  - DOM Manipulation: Basic JavaScript to show/hide a mobile menu when clicked
  - No Sensitive Data: No hardcoded credentials, API endpoints, or admin functionality

OWASP ZAP

OWASP ZAP Introduction & Setup

What is OWASP ZAP?

- Free, open-source web application security scanner
- Similar to Burp Suite but completely free with full functionality
- Automated vulnerability scanning capabilities
- Great for finding OWASP Top 10 vulnerabilities

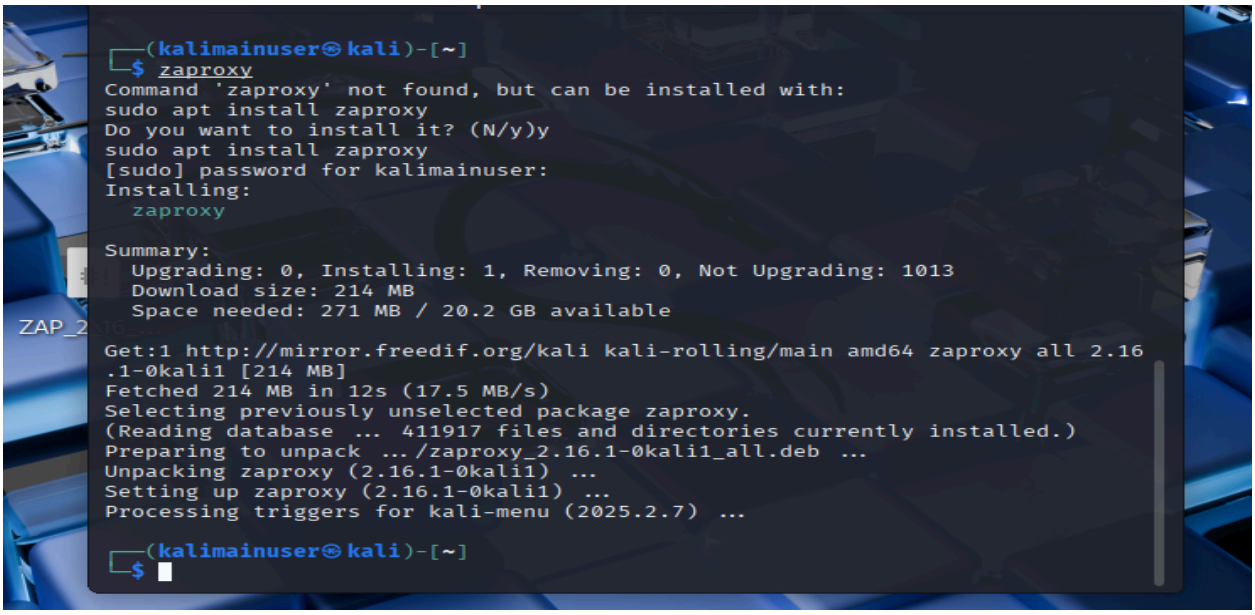
Why use ZAP after Burp?

- Compare results between two different tools
- ZAP might catch vulnerabilities Burp Community missed
- Provides automated scanning (unlike Burp Community's limited scanning)
- Good for comprehensive security assessment

ZAP Attack

Launch OWASP ZAP

`zaproxy`



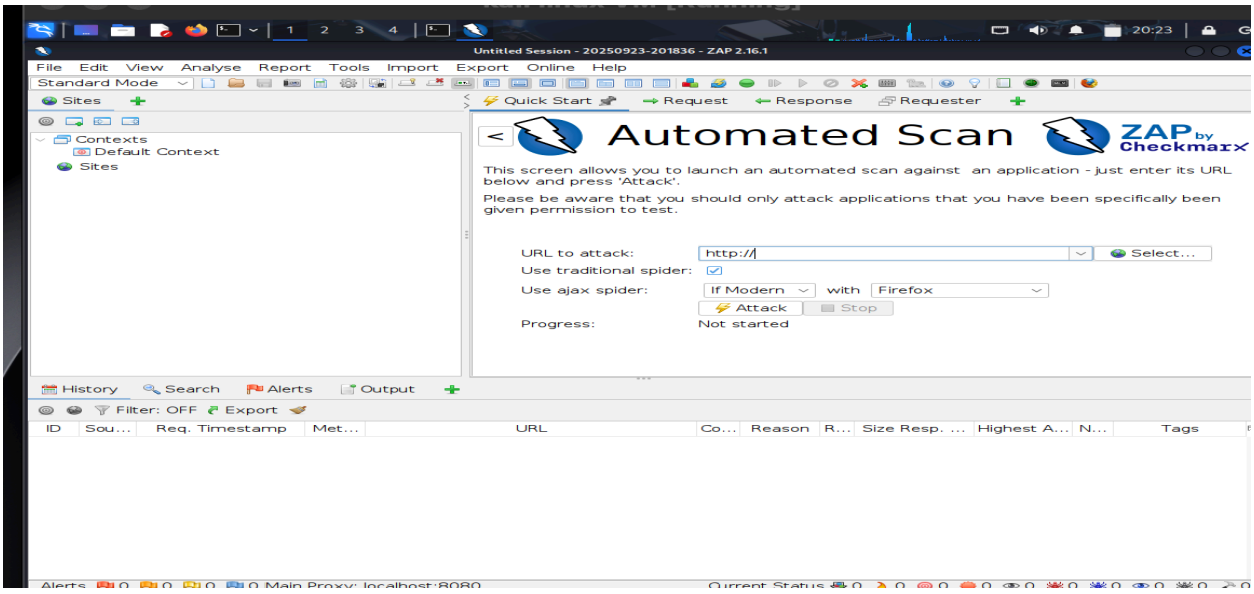
ZAP Starts:

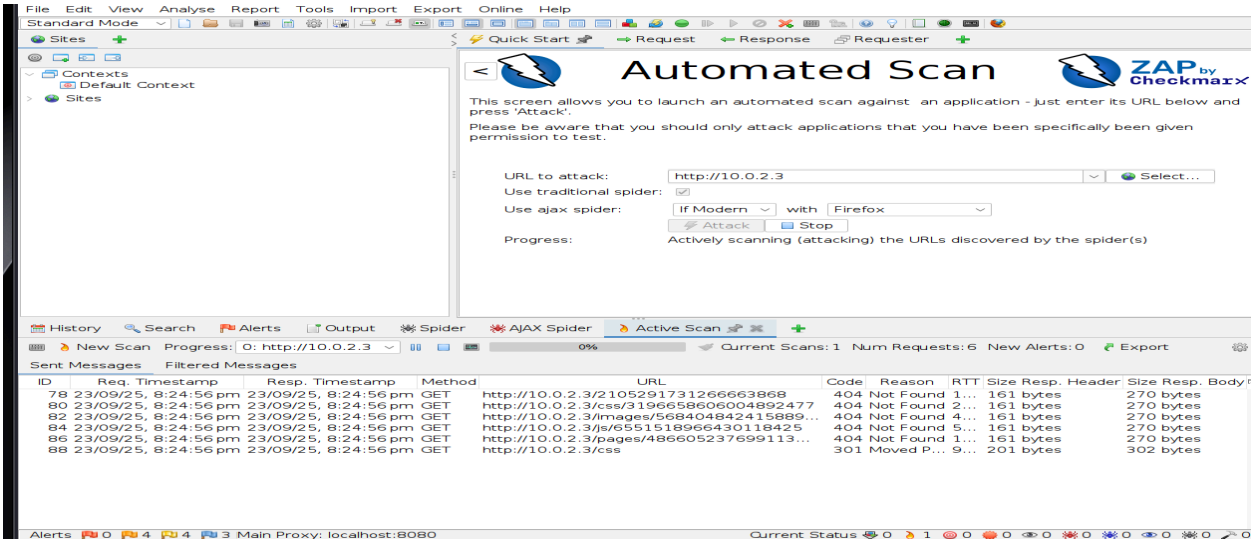
- Choose session type: Select "Persist Session" for saving results
- Configure proxy: ZAP will run on 127.0.0.1:8080 by default
- Set target URL: Enter the vulnerable VM's URL (e.g., http://[NEW_IP]/)

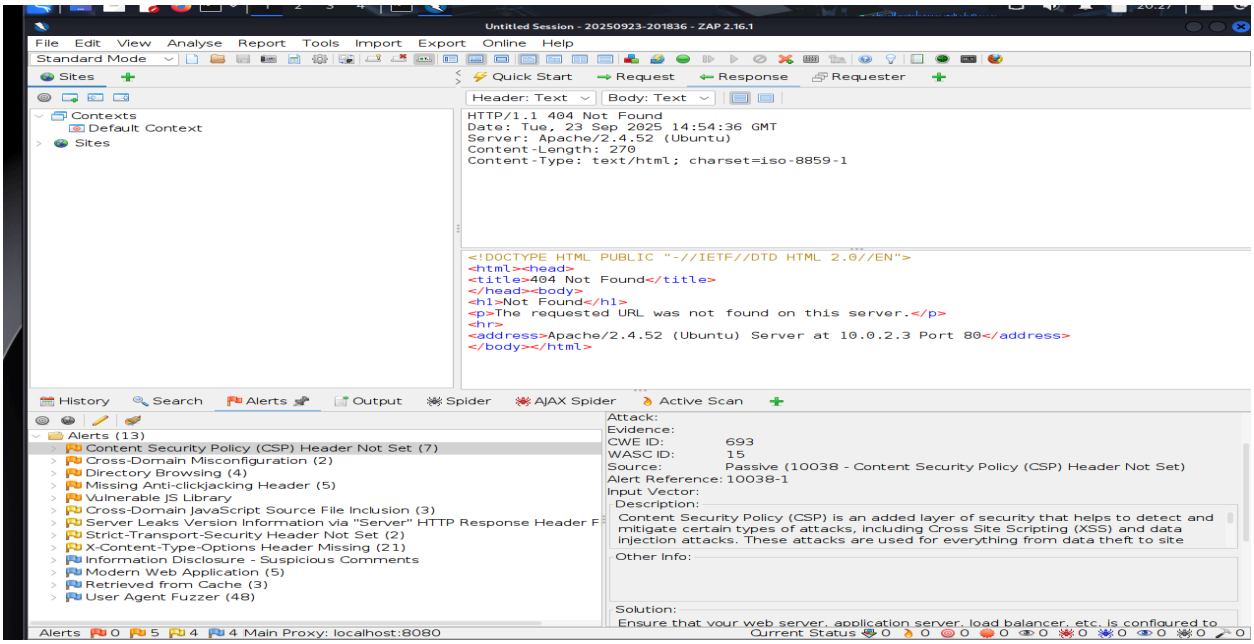
  Note: Due to technical problems needed to reinstall both VMs. So in upcoming screenshots we will use a new IP address for vulnerable VM.

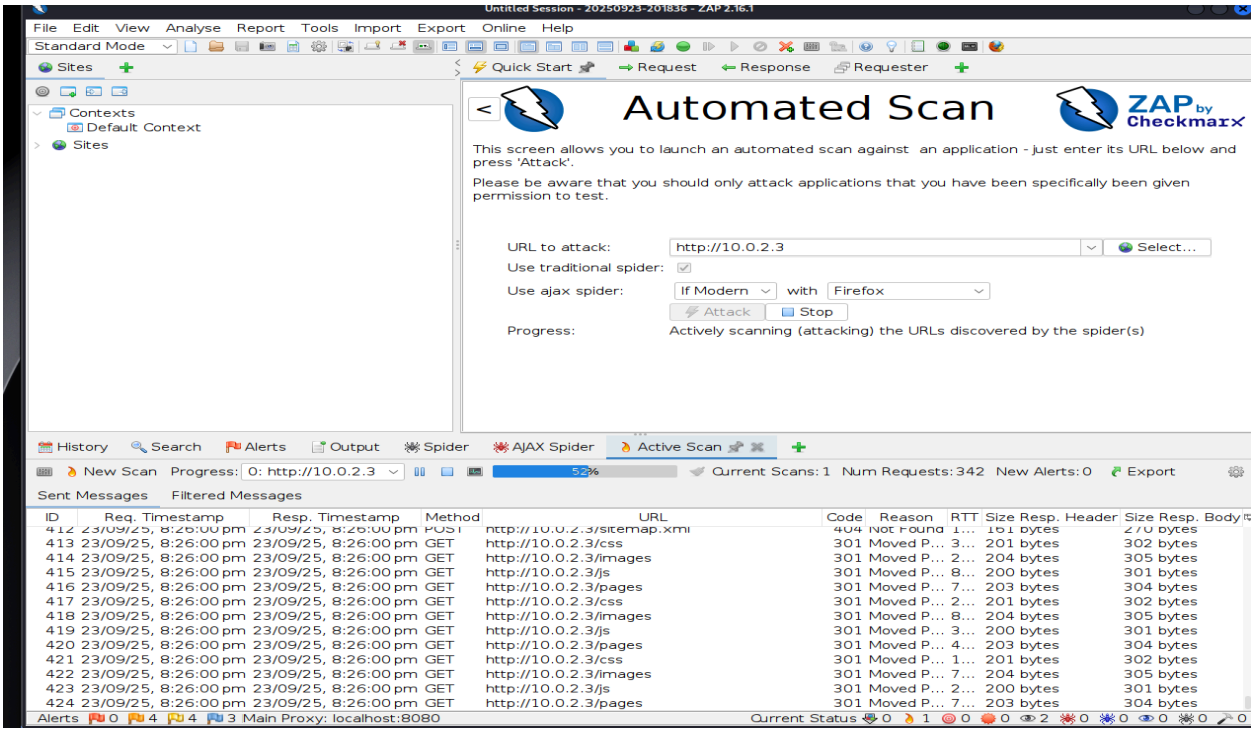
ZAP Automated Scan

- Zap → Automated Scan


Untitled Session - 20250923-201836 - ZAP 2.16.1
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Contexts
Default Context
Sites
Automated Scan
ZAP by Checkmarx
This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'.
Please be aware that you should only attack applications that you have been specifically been given permission to test.
URL to attack:
http://
Select...
Use traditional spider:
Use ajax spider:
If Modern
with
Firefox
Attack
Stop
Progress:
Not started
History
Search
Alerts
Output
Filter: OFF
Export
ID
Sou...
Req. Timestamp
Met...
URL
Co...
Reason
R...
Size Resp. ...
Highest A...
N...
Tags
Alerts 0 0 0 0 Main Proxy: localhost:8080
Current Status 0 0 0 0 0 0 0 0 0 0 0

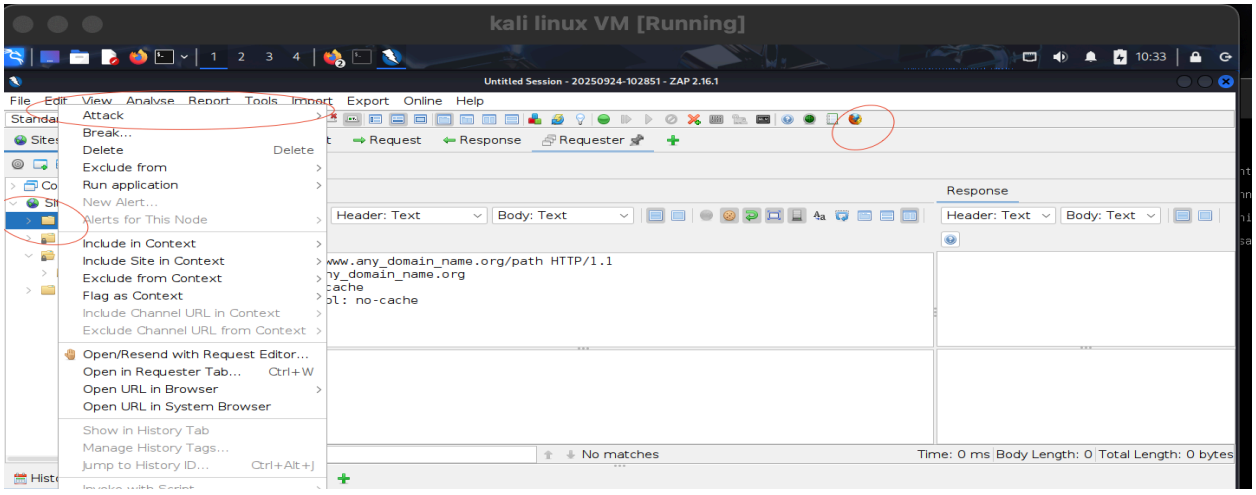

File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Contexts
Default Context
Sites
Automated Scan
ZAP by Checkmarx
This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'.
Please be aware that you should only attack applications that you have been specifically been given permission to test.
URL to attack:
http://10.0.2.3
Select...
Use traditional spider:
Use ajax spider:
If Modern
with
Firefox
Attack
Stop
Progress:
Actively scanning (attacking) the URLs discovered by the spider(s)
History
Search
Alerts
Output
Spider
AJAX Spider
Active Scan
New Scan
Progress: 0: http://10.0.2.3
0%
Current Scans: 1
Num Requests: 6
New Alerts: 0
Export
Sent Messages
Filtered Messages
ID Req. Timestamp Resp. Timestamp Method URL Code Reason RTT Size Resp. Header Size Resp. Body
78 23/09/25, 8:24:56 pm 23/09/25, 8:24:56 pm GET http://10.0.2.3/210529173126666386 8 404 Not Found 1... 161 bytes 270 bytes
80 23/09/25, 8:24:56 pm 23/09/25, 8:24:56 pm GET http://10.0.2.3/css/319665860600489247 7 404 Not Found 2... 161 bytes 270 bytes
82 23/09/25, 8:24:56 pm 23/09/25, 8:24:56 pm GET http://10.0.2.3/images/568404842415889... 404 Not Found 4... 161 bytes 270 bytes
84 23/09/25, 8:24:56 pm 23/09/25, 8:24:56 pm GET http://10.0.2.3/js/655151896643011842 5 404 Not Found 5... 161 bytes 270 bytes
86 23/09/25, 8:24:56 pm 23/09/25, 8:24:56 pm GET http://10.0.2.3/pages/486605237699113... 404 Not Found 1... 161 bytes 270 bytes
88 23/09/25, 8:24:56 pm 23/09/25, 8:24:56 pm GET http://10.0.2.3/css 301 Moved P... 9... 201 bytes 302 bytes
Alerts 0 4 4 3 Main Proxy: localhost:8080
Current Status 0 1 0 0 0 0 0 0 0 0 0

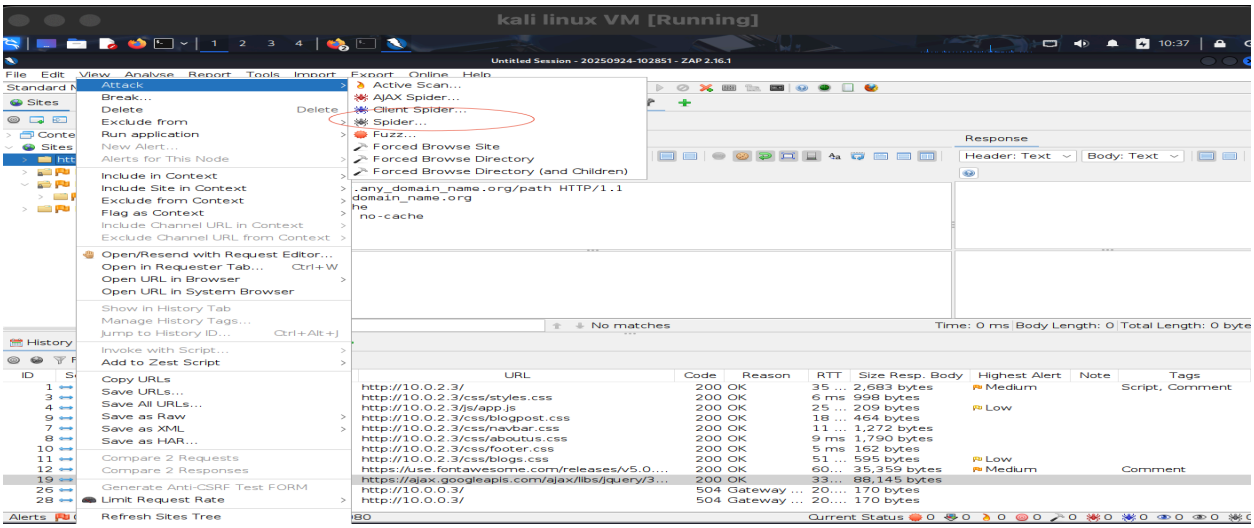

Untitled Session - 20250923-201836 - ZAP 2.16.1
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Header: Text
Body: Text
Contexts
Default Context
Sites
HTTP/1.1 404 Not Found
Date: Tue, 23 Sep 2025 14:54:36 GMT
Server: Apache/2.4.52 (Ubuntu)
Content-Length: 270
Content-Type: text/html; charset=iso-8859-1
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
The requested URL was not found on this server.
<hr>
<address>Apache/2.4.52 (Ubuntu) Server at 10.0.2.3 Port 80</address>
</body></html>
History
Search
Alerts
Output
Spider
AJAX Spider
Active Scan
Alerts (13)
Content Security Policy (CSP) Header Not Set (7)
Cross-Domain Misconfiguration (2)
Directory Browsing (4)
Missing Anti-clickjacking Header (5)
Vulnerable JS Library
Cross-Domain JavaScript Source File Inclusion (3)
Server Leaks Version Information via "Server" HTTP Response Header F
Strict-Transport-Security Header Not Set (2)
X-Content-Type-Options Header Missing (21)
Information Disclosure - Suspicious Comments
Modern Web Application (5)
Retrieved from Cache (3)
User Agent Fuzzer (48)
Attack:
Evidence:
CWE ID: 693
WASC ID: 15
Source: Passive (10038 - Content Security Policy (CSP) Header Not Set)
Alert Reference: 10038-1
Input Vector:
Description:
Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site
Other Info:
Solution:
Ensure that your web server, application server, load balancer, etc. is configured to
Alerts 0 5 4 4 Main Proxy: localhost:8080
Current Status

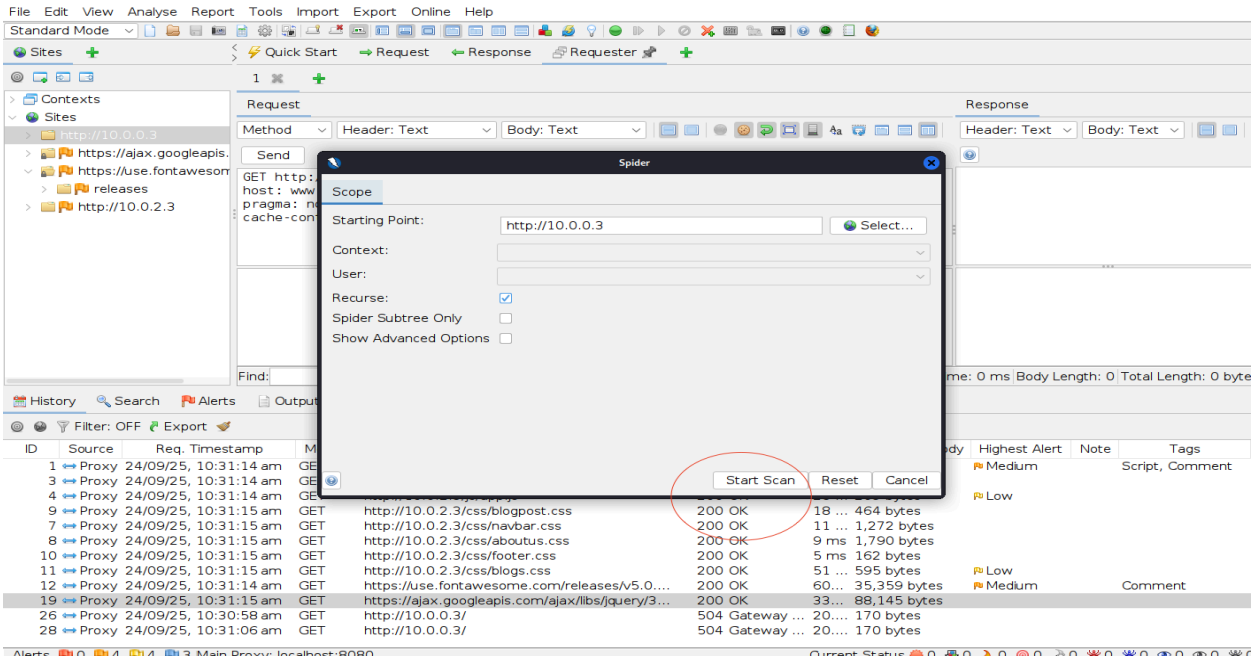

Untitled Session - 20250923-201836 - ZAP 2.16.1
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Contexts
Default Context
Sites
Automated Scan
ZAP by Checkmarx
This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'.
Please be aware that you should only attack applications that you have been specifically been given permission to test.
URL to attack: http://10.0.2.3
Select...
Use traditional spider:
Use ajax spider: If Modern with Firefox
Attack
Stop
Progress: Actively scanning (attacking) the URLs discovered by the spider(s)
History
Search
Alerts
Output
Spider
AJAX Spider
Active Scan
New Scan
Progress: 0: http://10.0.2.3
52%
Current Scans: 1
Num Requests: 342
New Alerts: 0
Export
Sent Messages
Filtered Messages
ID Req. Timestamp Resp. Timestamp Method URL Code Reason RTT Size Resp. Header Size Resp. Body
413 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/css 301 Moved P... 3... 201 bytes 302 bytes
414 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/images 301 Moved P... 2... 204 bytes 305 bytes
415 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/js 301 Moved P... 8... 200 bytes 301 bytes
416 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/pages 301 Moved P... 7... 203 bytes 304 bytes
417 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/css 301 Moved P... 2... 201 bytes 302 bytes
418 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/images 301 Moved P... 8... 204 bytes 305 bytes
419 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/js 301 Moved P... 3... 200 bytes 301 bytes
420 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/pages 301 Moved P... 4... 203 bytes 304 bytes
421 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/css 301 Moved P... 1... 201 bytes 302 bytes
422 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/images 301 Moved P... 7... 204 bytes 305 bytes
423 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/js 301 Moved P... 2... 200 bytes 301 bytes
424 23/09/25, 8:26:00 pm 23/09/25, 8:26:00 pm GET http://10.0.2.3/pages 301 Moved P... 7... 203 bytes 304 bytes
Alerts 0 4 4 3 Main Proxy: localhost:8080
Current Status

ZAP Spider Crawl

- Dashboard → firefox → http://10.0.2.3 (new IP of Vulnerable VM)
- Site Map → Right click on IP in the site map → Attack → Spider

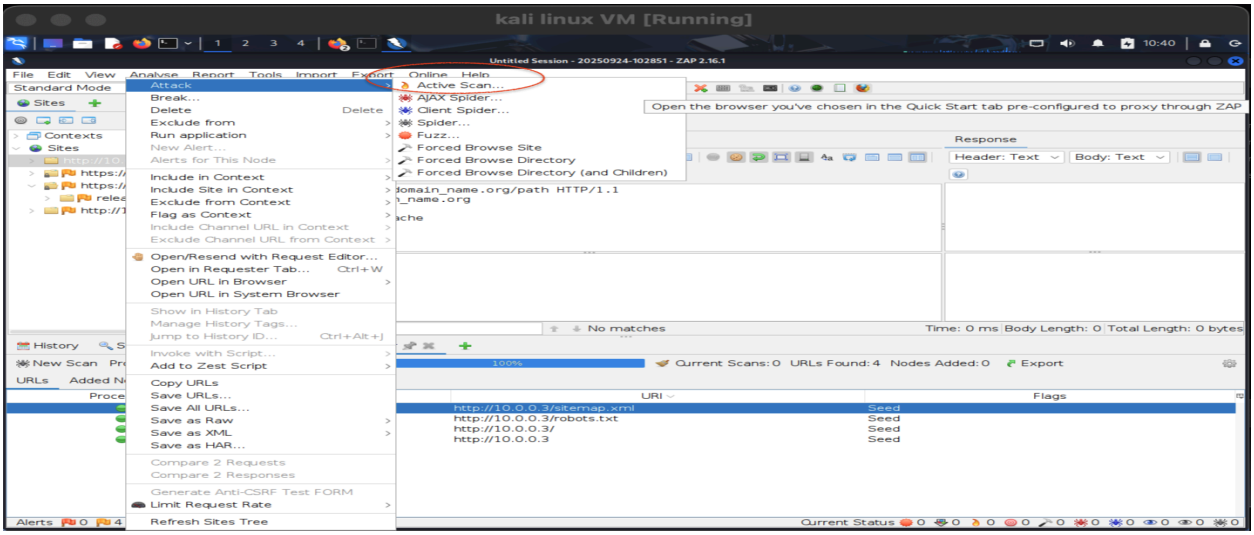


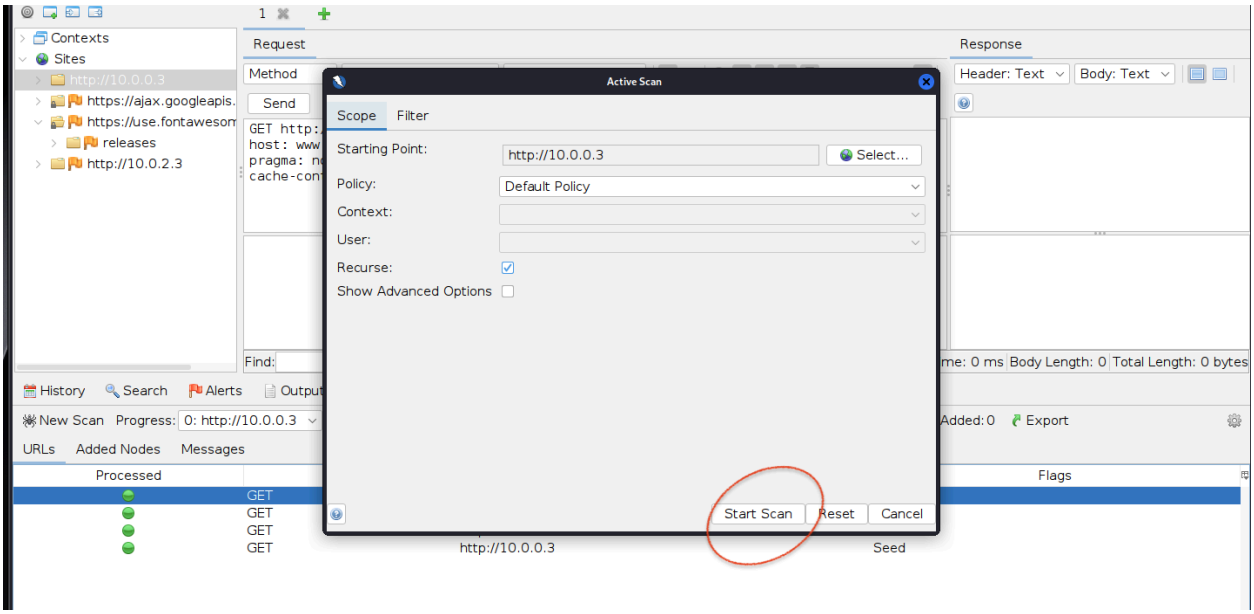

kali linux VM [Running]
Untitled Session - 20250924-102851 - ZAP 2.16.1
File Edit View Analyse Report Tools Import Export Online Help
Attack
Break...
Delete
Exclude from
Run application
New Alert...
Alerts for This Node
Include in Context
Include Site in Context
Exclude from Context
Flag as Context
Include Channel URL in Context
Exclude Channel URL from Context
Open/Resend with Request Editor...
Open in Requester Tab... Ctrl+W
Open URL in Browser
Open URL in System Browser
Show in History Tab
Manage History Tags...
Jump to History ID... Ctrl+Alt+J
Invoke with Script...
Add to Zest Script
Copy URLs
Save URLs...
Save All URLs...
Save as Raw
Save as XML
Save as HAR...
Compare 2 Requests
Compare 2 Responses
Generate Anti-CSRF Test FORM
Limit Request Rate
Refresh Sites Tree
Active Scan...
AJAX Spider...
Client Spider...
Spider...
Fuzz...
Forced Browse Site
Forced Browse Directory
Forced Browse Directory (and Children)
Response
Header: Text
Body: Text
No matches
Time: 0 ms Body Length: 0 Total Length: 0 byte
URL Code Reason RTT Size Resp. Body Highest Alert Note Tags
http://10.0.2.3/ 200 OK 35 ... 2,683 bytes Medium Script, Comment
http://10.0.2.3/css/styles.css 200 OK 6 ms 998 bytes
http://10.0.2.3/js/app.js 200 OK 25 ... 209 bytes Low
http://10.0.2.3/css/blogpost.css 200 OK 18 ... 464 bytes
http://10.0.2.3/css/navbar.css 200 OK 11 ... 1,272 bytes
http://10.0.2.3/css/aboutus.css 200 OK 9 ms 1,790 bytes
http://10.0.2.3/css/footer.css 200 OK 5 ms 162 bytes
http://10.0.2.3/css/blogs.css 200 OK 51 ... 595 bytes Low
https://use.fontawesome.com/releases/v5.0.... 200 OK 60... 35,359 bytes Medium Comment
https://ajax.googleapis.com/ajax/libs/jquery/3... 200 OK 33... 88,145 bytes
http://10.0.0.3/ 504 Gateway ... 20.... 170 bytes
http://10.0.0.3/ 504 Gateway ... 20.... 170 bytes
Alerts
Current Status


File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Contexts
Sites
http://10.0.0.3
https://ajax.googleapis.
https://use.fontawesom
releases
http://10.0.2.3
Method
Header: Text
Body: Text
Send
Spider
Scope
Starting Point:
http://10.0.0.3
Select...
Context:
User:
Recurse:
Spider Subtree Only
Show Advanced Options
Start Scan
Reset
Cancel
Find:
History
Search
Alerts
Filter: OFF
Export
ID Source Req. Timestamp M URL Code Reason RTT Size Resp. Body Highest Alert Note Tags
1 Proxy 24/09/25, 10:31:14 am GE Medium Script, Comment
3 Proxy 24/09/25, 10:31:14 am GE
4 Proxy 24/09/25, 10:31:14 am GE Low
9 Proxy 24/09/25, 10:31:15 am GET http://10.0.2.3/css/blogpost.css 200 OK 18 ... 464 bytes
7 Proxy 24/09/25, 10:31:15 am GET http://10.0.2.3/css/navbar.css 200 OK 11 ... 1,272 bytes
8 Proxy 24/09/25, 10:31:15 am GET http://10.0.2.3/css/aboutus.css 200 OK 9 ms 1,790 bytes
10 Proxy 24/09/25, 10:31:15 am GET http://10.0.2.3/css/footer.css 200 OK 5 ms 162 bytes
11 Proxy 24/09/25, 10:31:15 am GET http://10.0.2.3/css/blogs.css 200 OK 51 ... 595 bytes Low
12 Proxy 24/09/25, 10:31:14 am GET https://use.fontawesome.com/releases/v5.0.... 200 OK 60... 35,359 bytes Medium Comment
19 Proxy 24/09/25, 10:31:15 am GET https://ajax.googleapis.com/ajax/libs/jquery/3... 200 OK 33... 88,145 bytes
26 Proxy 24/09/25, 10:30:58 am GET http://10.0.0.3/ 504 Gateway ... 20.... 170 bytes
28 Proxy 24/09/25, 10:31:06 am GET http://10.0.0.3/ 504 Gateway ... 20.... 170 bytes

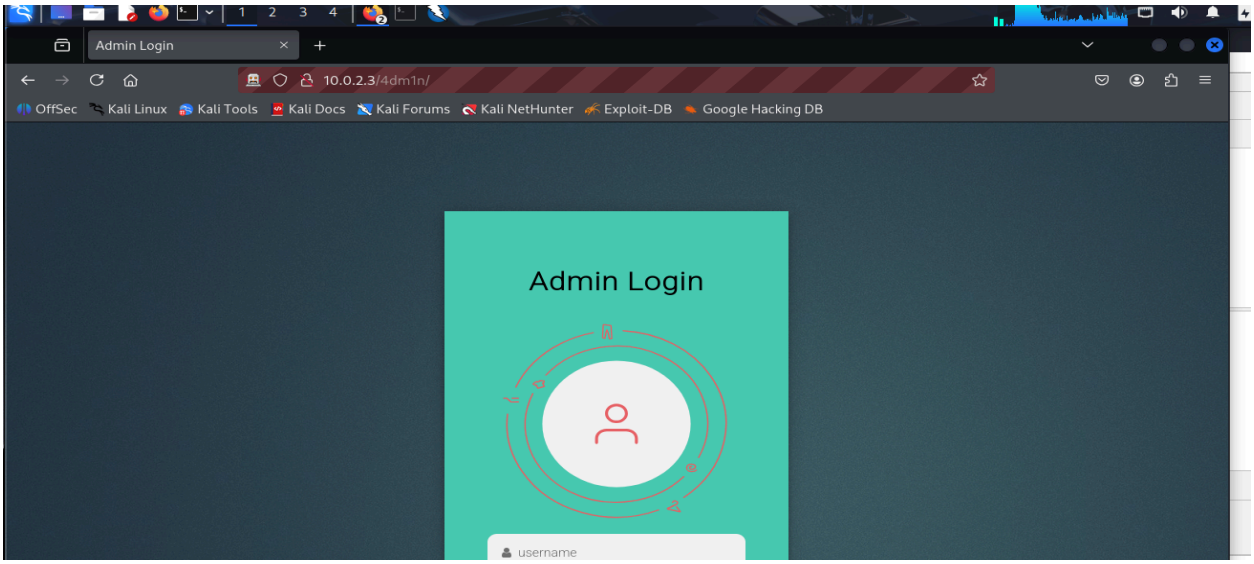
ZAP Active Scan

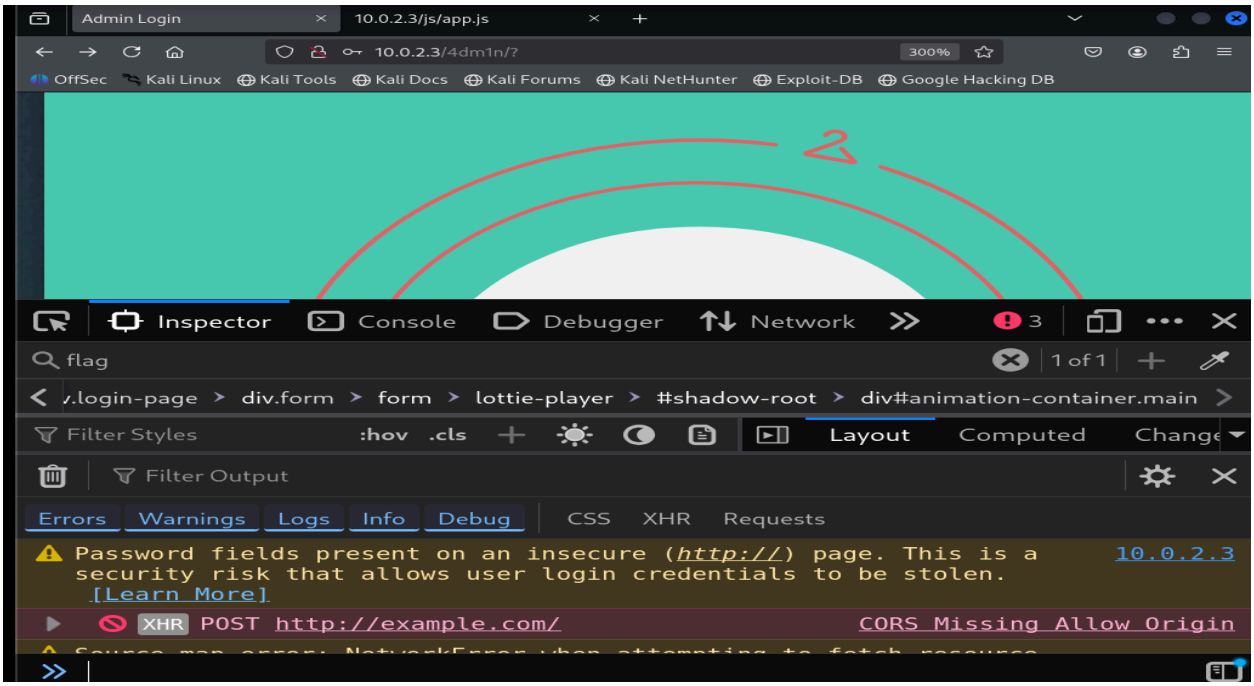
Site Map → Right Click on the IP Address of Site → Active Scan

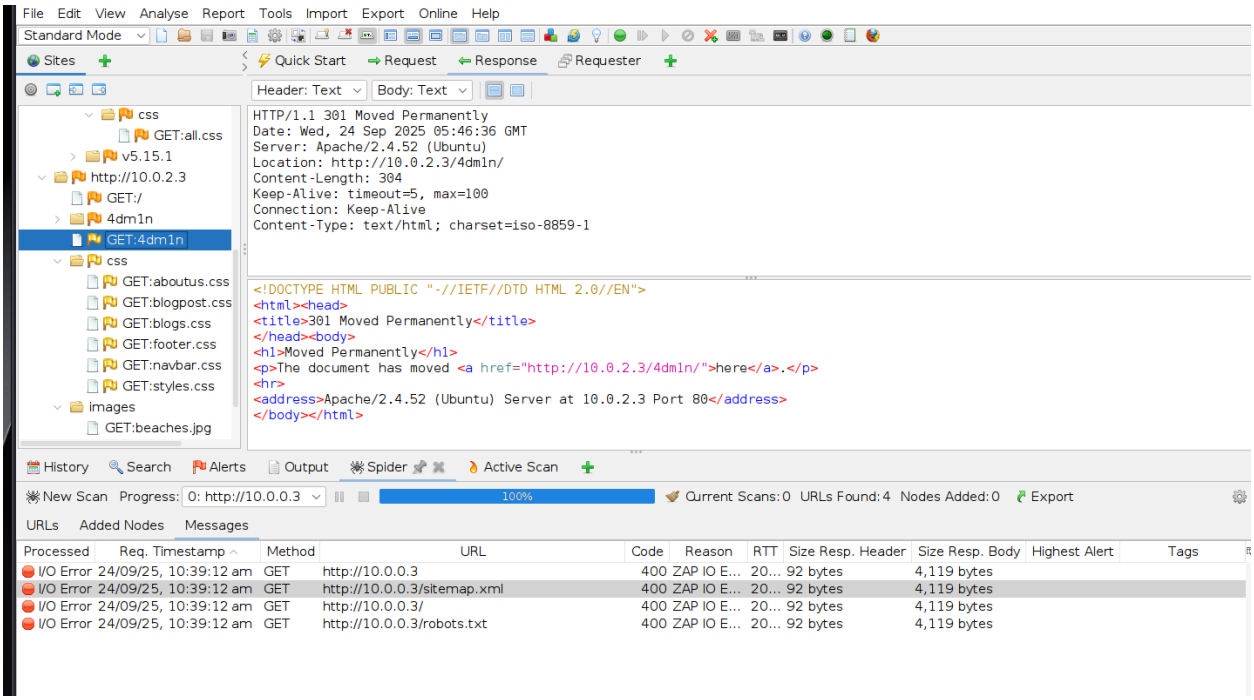


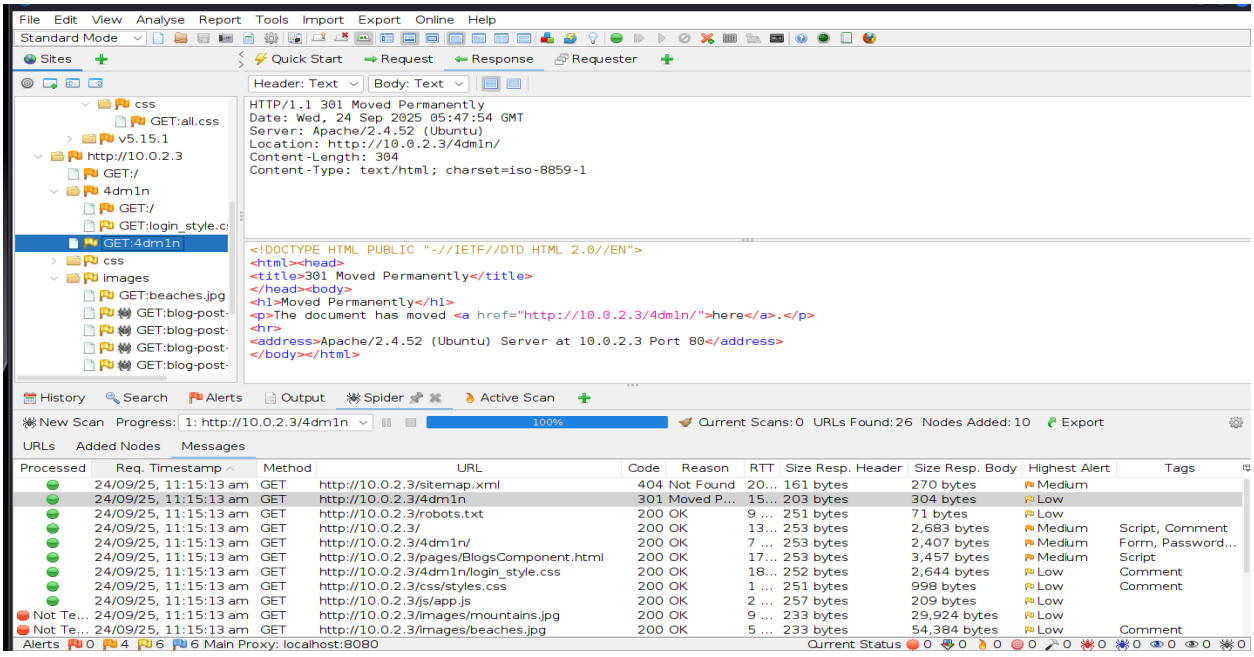


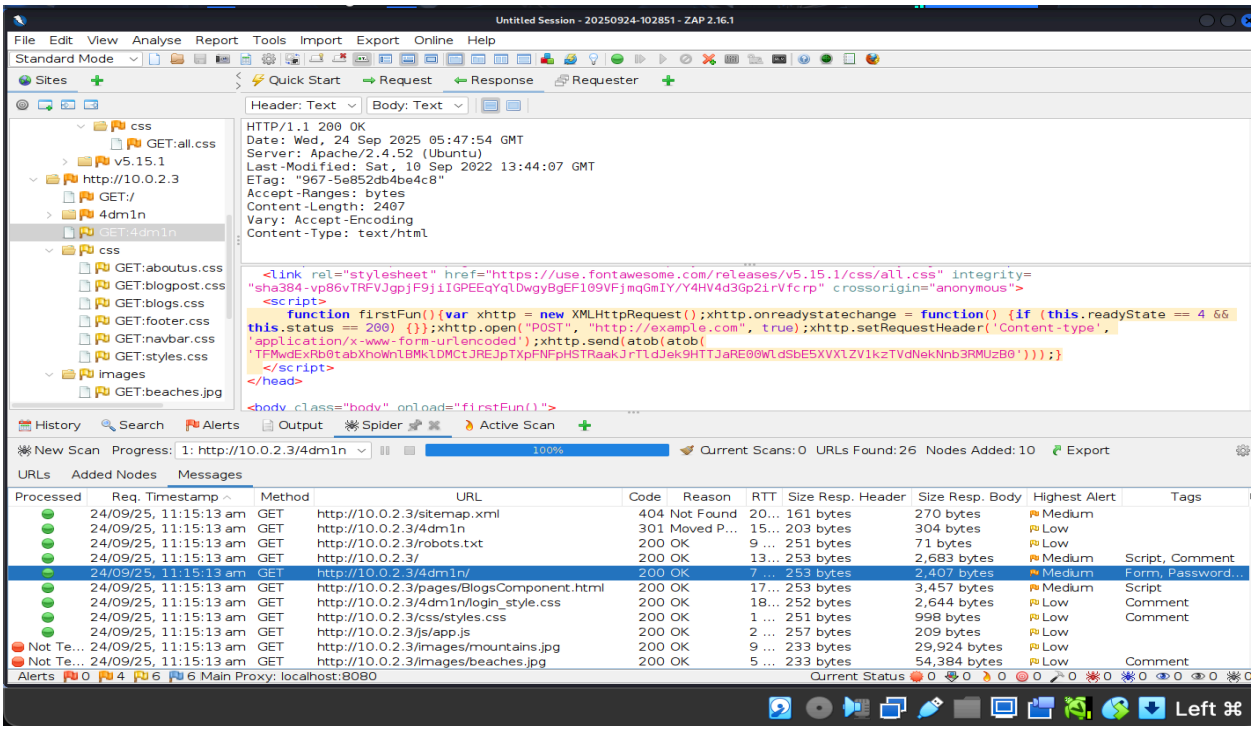
Now open the admin page from the safari browser option in ZAP. Enter the admin page url as shown in the screenshot.

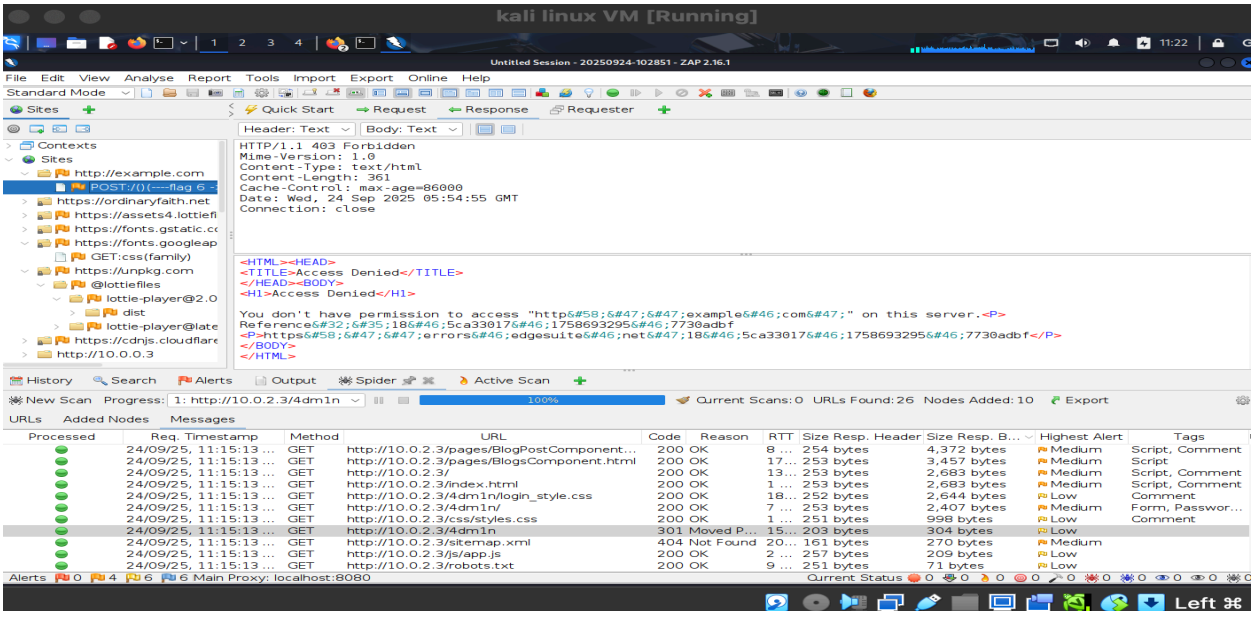

Admin Login
10.0.2.3/4dm1n/
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
Admin Login
username

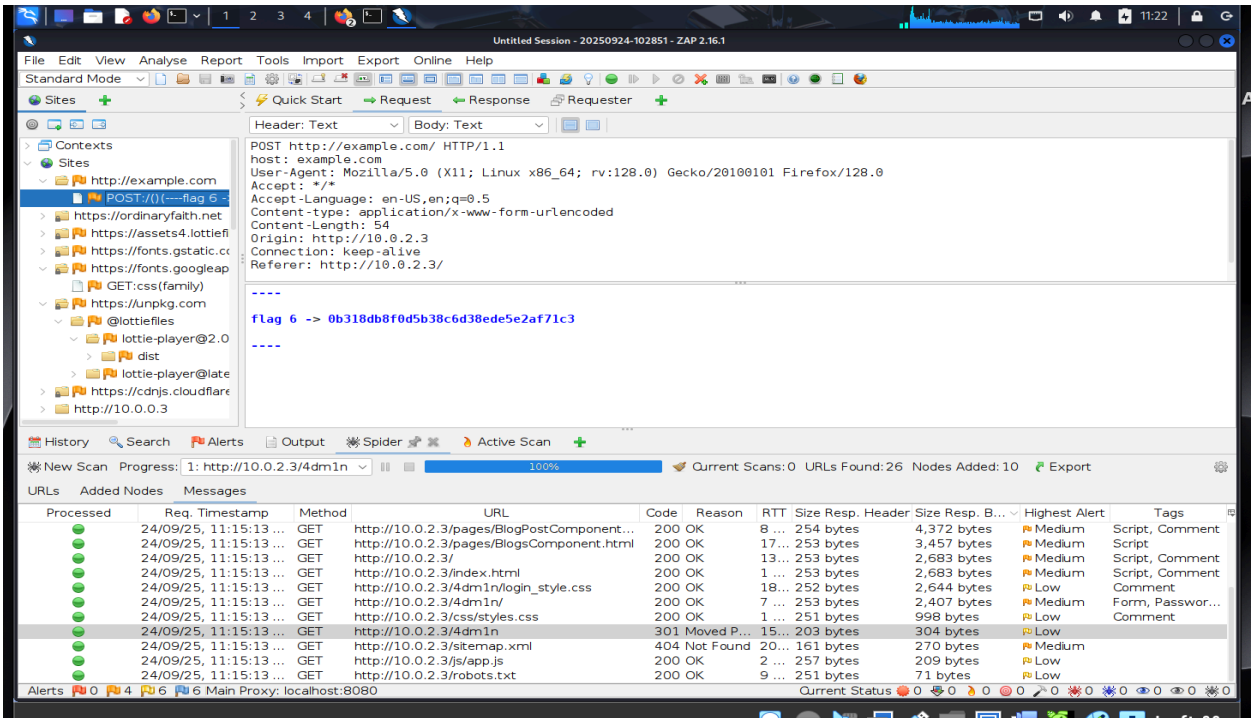

Admin Login
10.0.2.3/js/app.js
10.0.2.3/4dm1n/?
300%
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
Inspector
Console
Debugger
Network
flag
1 of 1
.login-page > div.form > form > lottie-player > #shadow-root > div#animation-container.main
Filter Styles
:hov .cls
Layout
Computed
Filter Output
Errors
Warnings
Logs
Info
Debug
CSS
XHR
Requests
Password fields present on an insecure (http://) page. This is a security risk that allows user login credentials to be stolen.
[Learn More]
10.0.2.3
XHR POST http://example.com/
CORS Missing Allow Origin


File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Header: Text
Body: Text
HTTP/1.1 301 Moved Permanently
Date: Wed, 24 Sep 2025 05:46:36 GMT
Server: Apache/2.4.52 (Ubuntu)
Location: http://10.0.2.3/4dm1n/
Content-Length: 304
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=iso-8859-1
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>301 Moved Permanently</title>
</head><body>
<h1>Moved Permanently</h1>
The document has moved here.
<hr>
<address>Apache/2.4.52 (Ubuntu) Server at 10.0.2.3 Port 80</address>
</body></html>
History
Search
Alerts
Output
Spider
Active Scan
New Scan
Progress: 0: http://10.0.0.3
100%
Current Scans: 0
URLs Found: 4
Nodes Added: 0
Export
URLs
Added Nodes
Messages
Processed
Req. Timestamp
Method
URL
Code
Reason
RTT
Size Resp. Header
Size Resp. Body
Highest Alert
Tags
I/O Error 24/09/25, 10:39:12 am GET http://10.0.0.3 400 ZAP IO E... 20... 92 bytes 4,119 bytes
I/O Error 24/09/25, 10:39:12 am GET http://10.0.0.3/sitemap.xml 400 ZAP IO E... 20... 92 bytes 4,119 bytes
I/O Error 24/09/25, 10:39:12 am GET http://10.0.0.3/ 400 ZAP IO E... 20... 92 bytes 4,119 bytes
I/O Error 24/09/25, 10:39:12 am GET http://10.0.0.3/robots.txt 400 ZAP IO E... 20... 92 bytes 4,119 bytes


File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Header: Text
Body: Text
HTTP/1.1 301 Moved Permanently
Date: Wed, 24 Sep 2025 05:47:54 GMT
Server: Apache/2.4.52 (Ubuntu)
Location: http://10.0.2.3/4dm1n/
Content-Length: 304
Content-Type: text/html; charset=iso-8859-1
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>301 Moved Permanently</title>
</head><body>
<h1>Moved Permanently</h1>
The document has moved here.
<hr>
<address>Apache/2.4.52 (Ubuntu) Server at 10.0.2.3 Port 80</address>
</body></html>
History
Search
Alerts
Output
Spider
Active Scan
New Scan
Progress: 1: http://10.0.2.3/4dm1n
100%
Current Scans: 0
URLs Found: 26
Nodes Added: 10
Export
URLs
Added Nodes
Messages
Processed
Req. Timestamp
Method
URL
Code
Reason
RTT
Size Resp. Header
Size Resp. Body
Highest Alert
Tags
24/09/25, 11:15:13 am GET http://10.0.2.3/sitemap.xml 404 Not Found 20... 161 bytes 270 bytes Medium
24/09/25, 11:15:13 am GET http://10.0.2.3/4dm1n 301 Moved P... 15... 203 bytes 304 bytes Low
24/09/25, 11:15:13 am GET http://10.0.2.3/robots.txt 200 OK 9 ... 251 bytes 71 bytes Low
24/09/25, 11:15:13 am GET http://10.0.2.3/ 200 OK 13... 253 bytes 2,683 bytes Medium Script, Comment
24/09/25, 11:15:13 am GET http://10.0.2.3/4dm1n/ 200 OK 7 ... 253 bytes 2,407 bytes Medium Form, Password...
24/09/25, 11:15:13 am GET http://10.0.2.3/pages/BlogsComponent.html 200 OK 17... 253 bytes 3,457 bytes Medium Script
24/09/25, 11:15:13 am GET http://10.0.2.3/4dm1n/login_style.css 200 OK 18... 252 bytes 2,644 bytes Low Comment
24/09/25, 11:15:13 am GET http://10.0.2.3/css/styles.css 200 OK 1 ... 251 bytes 998 bytes Low Comment
24/09/25, 11:15:13 am GET http://10.0.2.3/js/app.js 200 OK 2 ... 257 bytes 209 bytes Low
Not Te... 24/09/25, 11:15:13 am GET http://10.0.2.3/images/mountains.jpg 200 OK 9 ... 233 bytes 29,924 bytes Low
Not Te... 24/09/25, 11:15:13 am GET http://10.0.2.3/images/beaches.jpg 200 OK 5 ... 233 bytes 54,384 bytes Low Comment
Alerts 0 4 6 6 Main Proxy: localhost:8080
Current Status 0 0 0 0 0 0 0 0 0 0 0 0


Untitled Session - 20250924-102851 - ZAP 2.16.1
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Header: Text
Body: Text
HTTP/1.1 200 OK
Date: Wed, 24 Sep 2025 05:47:54 GMT
Server: Apache/2.4.52 (Ubuntu)
Last-Modified: Sat, 10 Sep 2022 13:44:07 GMT
ETag: "967-5e852db4be4c8"
Accept-Ranges: bytes
Content-Length: 2407
Vary: Accept-Encoding
Content-Type: text/html
<link rel="stylesheet" href="https://use.fontawesome.com/releases/v5.15.1/css/all.css" integrity="sha384-vp86vTRFVJgpjF9jiIGPEEqYqlDwgyBgEF109VFjmqGmIY/Y4HV4d3Gp2irVfcrp" crossorigin="anonymous">
<script>
function firstFun(){var xhttp = new XMLHttpRequest();xhttp.onreadystatechange = function() {if (this.readyState == 4 && this.status == 200) {}};xhttp.open("POST", "http://example.com", true);xhttp.setRequestHeader('Content-type', 'application/x-www-form-urlencoded');xhttp.send(atob(atob('TFMwdExRb0tabXhoWnlBMklDMCtJREJpTXpFNFpHSTRaakJrTldJek9HTTJaRE00WldSbE5XVXlZV1kzTVdNekNnb3RMUzB0')))};
</script>
</head>
<body class="body" onload="firstFun()">
History Search Alerts Output Spider Active Scan
New Scan Progress: 1: http://10.0.2.3/4dm1n 100% Current Scans: 0 URLs Found: 26 Nodes Added: 10 Export
URLs Added Nodes Messages
Processed Req. Timestamp Method URL Code Reason RTT Size Resp. Header Size Resp. Body Highest Alert Tags
24/09/25, 11:15:13 am GET http://10.0.2.3/sitemap.xml 404 Not Found 20... 161 bytes 270 bytes Medium
24/09/25, 11:15:13 am GET http://10.0.2.3/4dm1n 301 Moved P... 15... 203 bytes 304 bytes Low
24/09/25, 11:15:13 am GET http://10.0.2.3/robots.txt 200 OK 9 ... 251 bytes 71 bytes Low
24/09/25, 11:15:13 am GET http://10.0.2.3/ 200 OK 13... 253 bytes 2,683 bytes Medium Script, Comment
24/09/25, 11:15:13 am GET http://10.0.2.3/4dm1n/ 200 OK 7 ... 253 bytes 2,407 bytes Medium Form, Password...
24/09/25, 11:15:13 am GET http://10.0.2.3/pages/BlogsComponent.html 200 OK 17... 253 bytes 3,457 bytes Medium Script
24/09/25, 11:15:13 am GET http://10.0.2.3/4dm1n/login_style.css 200 OK 18... 252 bytes 2,644 bytes Low Comment
24/09/25, 11:15:13 am GET http://10.0.2.3/css/styles.css 200 OK 1 ... 251 bytes 998 bytes Low Comment
24/09/25, 11:15:13 am GET http://10.0.2.3/js/app.js 200 OK 2 ... 257 bytes 209 bytes Low
Not Te... 24/09/25, 11:15:13 am GET http://10.0.2.3/images/mountains.jpg 200 OK 9 ... 233 bytes 29,924 bytes Low
Not Te... 24/09/25, 11:15:13 am GET http://10.0.2.3/images/beaches.jpg 200 OK 5 ... 233 bytes 54,384 bytes Low Comment
Alerts 0 4 6 6 Main Proxy: localhost:8080 Current Status 0 0 0 0 0 0 0 0 0 0 0
Left ⌘


kali linux VM [Running]
Untitled Session - 20250924-102851 - ZAP 2.16.1
HTTP/1.1 403 Forbidden
Mime-Version: 1.0
Content-Type: text/html
Content-Length: 361
Cache-Control: max-age=86000
Date: Wed, 24 Sep 2025 05:54:55 GMT
Connection: close
<HTML><HEAD>
<TITLE>Access Denied</TITLE>
</HEAD><BODY>
<H1>Access Denied</H1>
You don't have permission to access "http://example.com/" on this server.<P>
Reference #18.5ca33017.1758693295.7730adbf
<P>https://errors.edgesuite.net/18.5ca33017.1758693295.7730adbf</P>
</BODY>
</HTML>
History Search Alerts Output Spider Active Scan
New Scan Progress: 1: http://10.0.2.3/4dm1n 100% Current Scans: 0 URLs Found: 26 Nodes Added: 10 Export
URLs Added Nodes Messages
Processed Req. Timestamp Method URL Code Reason RTT Size Resp. Header Size Resp. B... Highest Alert Tags
24/09/25, 11:15:13 ... GET http://10.0.2.3/pages/BlogPostComponent... 200 OK 8 ... 254 bytes 4,372 bytes Medium Script, Comment
24/09/25, 11:15:13 ... GET http://10.0.2.3/pages/BlogsComponent.html 200 OK 17... 253 bytes 3,457 bytes Medium Script
24/09/25, 11:15:13 ... GET http://10.0.2.3/ 200 OK 13... 253 bytes 2,683 bytes Medium Script, Comment
24/09/25, 11:15:13 ... GET http://10.0.2.3/index.html 200 OK 1 ... 253 bytes 2,683 bytes Medium Script, Comment
24/09/25, 11:15:13 ... GET http://10.0.2.3/4dm1n/login_style.css 200 OK 18... 252 bytes 2,644 bytes Low Comment
24/09/25, 11:15:13 ... GET http://10.0.2.3/4dm1n/ 200 OK 7 ... 253 bytes 2,407 bytes Medium Form, Passwor...
24/09/25, 11:15:13 ... GET http://10.0.2.3/css/styles.css 200 OK 1 ... 251 bytes 998 bytes Low Comment
24/09/25, 11:15:13 ... GET http://10.0.2.3/4dm1n 301 Moved P... 15... 203 bytes 304 bytes Low
24/09/25, 11:15:13 ... GET http://10.0.2.3/sitemap.xml 404 Not Found 20... 161 bytes 270 bytes Medium
24/09/25, 11:15:13 ... GET http://10.0.2.3/js/app.js 200 OK 2 ... 257 bytes 209 bytes Low
24/09/25, 11:15:13 ... GET http://10.0.2.3/robots.txt 200 OK 9 ... 251 bytes 71 bytes Low
Alerts 0 4 6 6 Main Proxy: localhost:8080 Current Status 0 0 0 0 0 0 0 0 0 0 0
Left ⌘


Untitled Session - 20250924-102851 - ZAP 2.16.1
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Header: Text
Body: Text
POST http://example.com/ HTTP/1.1
host: example.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Content-type: application/x-www-form-urlencoded
Content-Length: 54
Origin: http://10.0.2.3
Connection: keep-alive
Referer: http://10.0.2.3/
flag 6 -> 0b318db8f0d5b38c6d38ede5e2af71c3
History Search Alerts Output Spider Active Scan
New Scan Progress: 1: http://10.0.2.3/4dm1n 100% Current Scans: 0 URLs Found: 26 Nodes Added: 10 Export
URLs Added Nodes Messages
Processed Req. Timestamp Method URL Code Reason RTT Size Resp. Header Size Resp. B... Highest Alert Tags
24/09/25, 11:15:13 ... GET http://10.0.2.3/pages/BlogPostComponent... 200 OK 8 ... 254 bytes 4,372 bytes Medium Script, Comment
24/09/25, 11:15:13 ... GET http://10.0.2.3/pages/BlogsComponent.html 200 OK 17... 253 bytes 3,457 bytes Medium Script
24/09/25, 11:15:13 ... GET http://10.0.2.3/ 200 OK 13... 253 bytes 2,683 bytes Medium Script, Comment
24/09/25, 11:15:13 ... GET http://10.0.2.3/index.html 200 OK 1 ... 253 bytes 2,683 bytes Medium Script, Comment
24/09/25, 11:15:13 ... GET http://10.0.2.3/4dm1n/login_style.css 200 OK 18... 252 bytes 2,644 bytes Low Comment
24/09/25, 11:15:13 ... GET http://10.0.2.3/4dm1n/ 200 OK 7 ... 253 bytes 2,407 bytes Medium Form, Passwor...
24/09/25, 11:15:13 ... GET http://10.0.2.3/css/styles.css 200 OK 1 ... 251 bytes 998 bytes Low Comment
24/09/25, 11:15:13 ... GET http://10.0.2.3/4dm1n 301 Moved P... 15... 203 bytes 304 bytes Low
24/09/25, 11:15:13 ... GET http://10.0.2.3/sitemap.xml 404 Not Found 20... 161 bytes 270 bytes Medium
24/09/25, 11:15:13 ... GET http://10.0.2.3/js/app.js 200 OK 2 ... 257 bytes 209 bytes Low
24/09/25, 11:15:13 ... GET http://10.0.2.3/robots.txt 200 OK 9 ... 251 bytes 71 bytes Low
Alerts 0 4 6 6 Main Proxy: localhost:8080
Current Status

New Findings ZAP Discovered (Not Found in Burp):

Form with Password Field Flagged as "Medium" Risk

- Location: /4admin/ login form
- Risk Level: Medium
- Finding: "Form, Password..." vulnerability
- Why Burp Missed This: Burp's passive scanning didn't flag form security issues as prominently

Script/Comment Tags Flagged

- Multiple instances of "Script, Comment" findings
- Risk Level: Medium/Low
- Pages Affected: Multiple pages including blog components
- Issue: Potentially unsafe JavaScript comments or inline scripts

More Comprehensive Site Coverage

ZAP discovered and cataloged 26 URLs vs Burp's more limited manual crawling, including:

- /sitemap.xml (404 error)
- More blog component pages
- Better CSS/JS file enumeration

Error Page Analysis

- 404 errors properly flagged and analyzed
- Gateway timeout errors documented (504 errors)
- More systematic error handling analysis

Authentication Context Analysis

- ZAP shows authentication-related findings in the admin form
- Better analysis of login mechanisms
- Flag parameter detected in POST request (flag 6 -> 0b318db8f0d5b38c6d38ede5e2af71c3)

What This Reveals:

ZAP's Automated Advantages:

- Systematic Crawling: Found more pages automatically
- Form Analysis: Better at identifying form security issues and also great at application level security suggestions
- Comprehensive Scanning: More thorough passive + active scans
- Error Handling: Better documentation of error responses

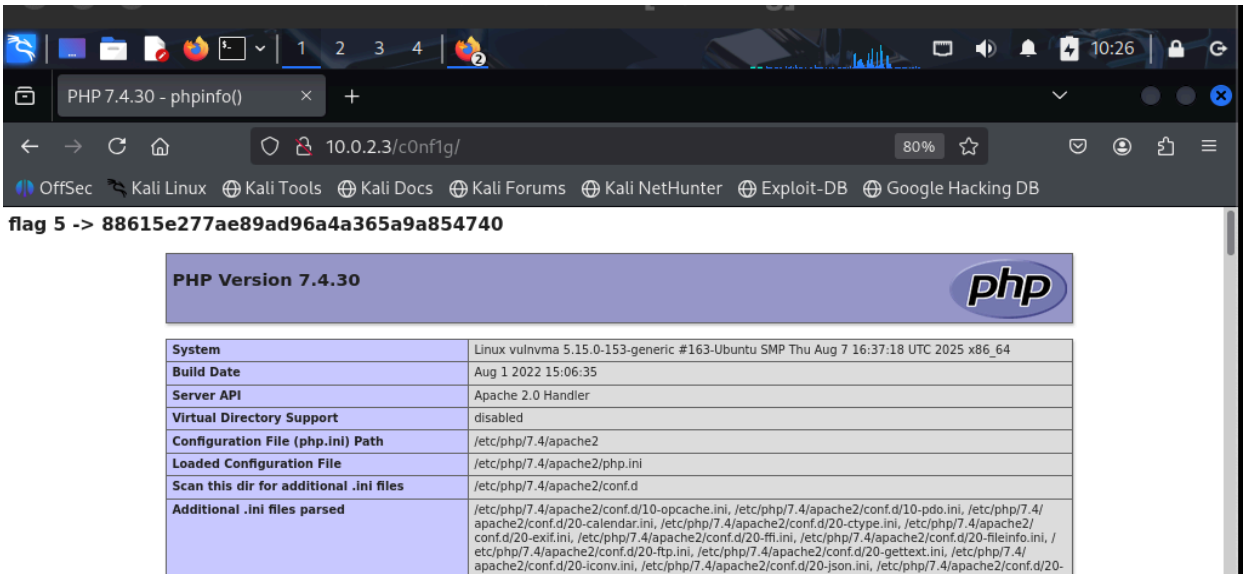
Burp's Manual Advantages (from earlier):

- Request Manipulation: Better for manual testing
- Detailed Inspection: More granular request/response analysis
- Custom Payloads: Manual injection testing capabilities

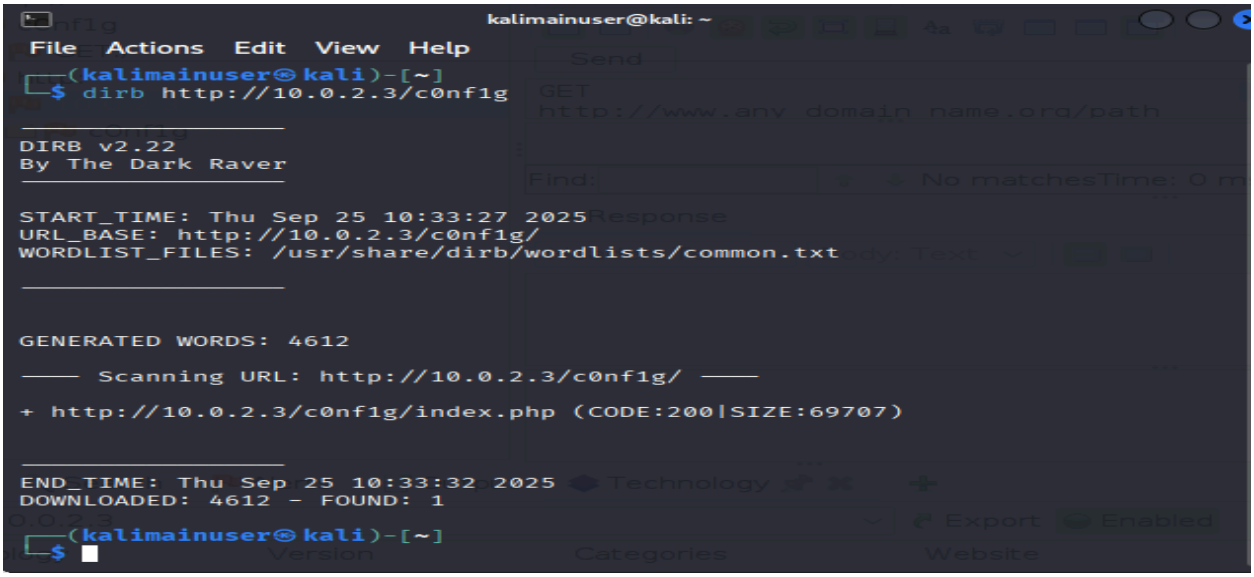
Most importantly, this taught us that we need to use a combination of tools in cyber security and must not rely on a single tool.

Flag 5

Since we found flags 1,2,3,4,6 - we needed to find Flag 5. It was after a lot of digging that we found the c0nf1g page which had flag 5.

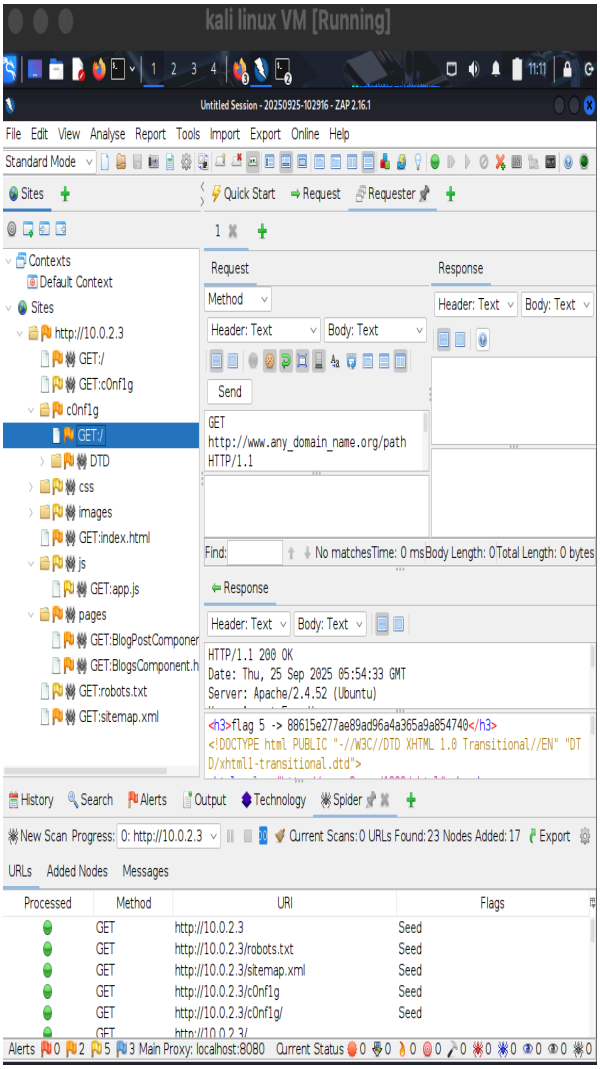


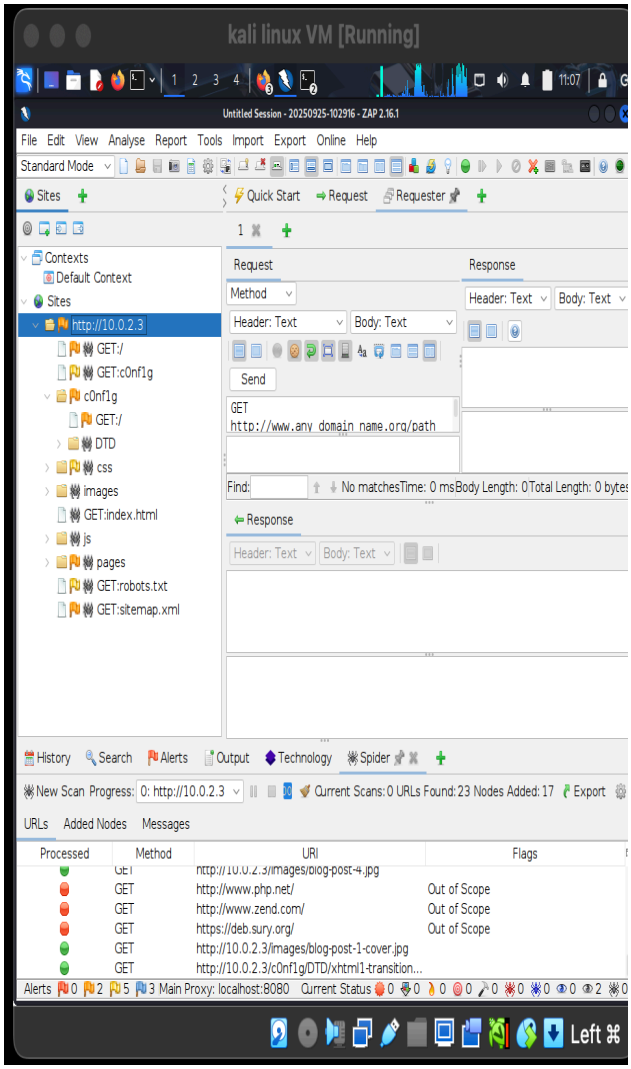
Lets run dirb against this url.



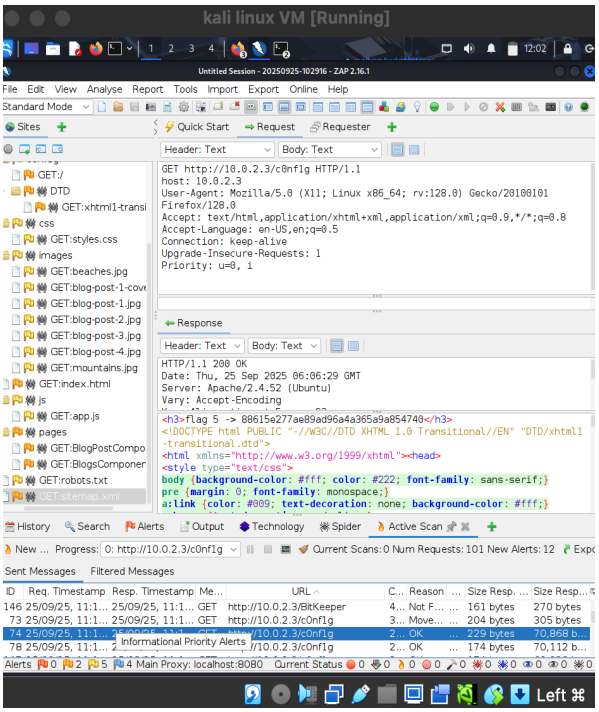
We found paths:
`/c0ng1g`
`/c0nf1g/index.php`

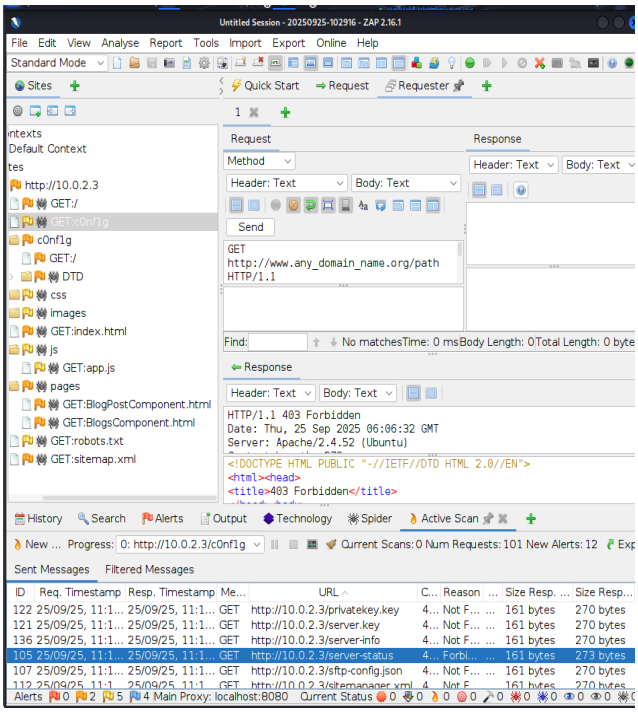
Next we use ZAP to perform spider scan this:

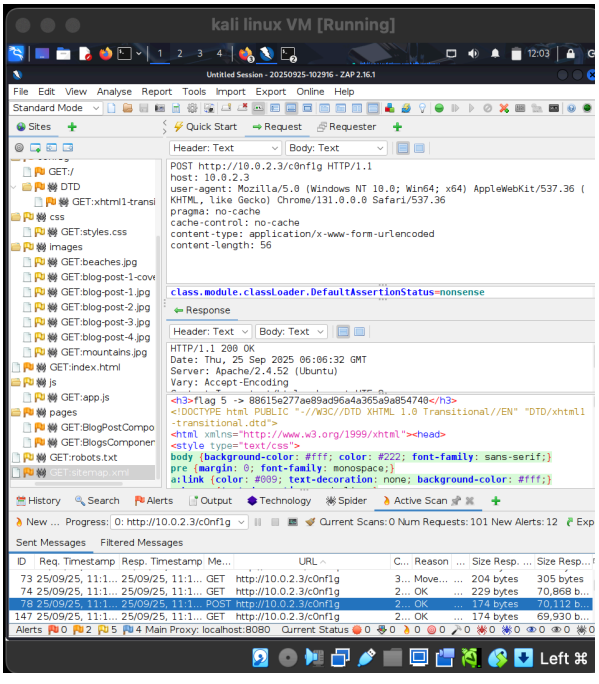


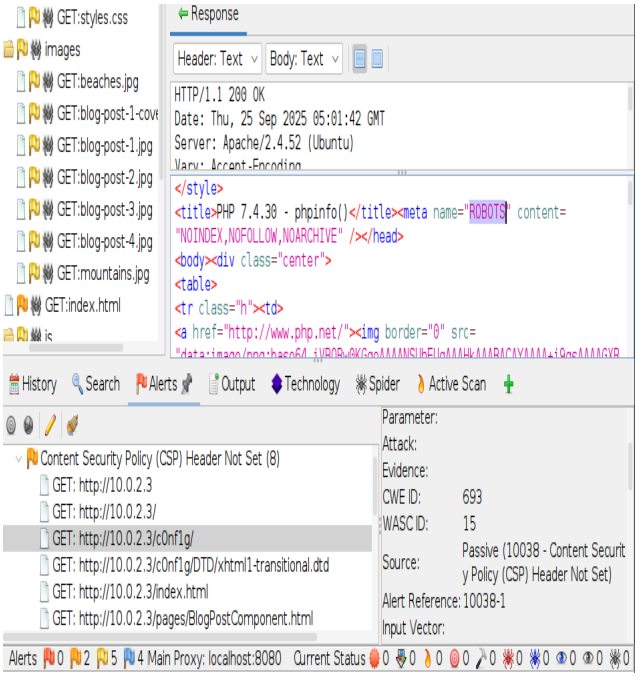


We also do active scan on this path as shown in screenshots.









But since there is nothing new here, we can move on with the next step.

XSS Injection Testing on /4dm1n Admin Login

Testing Approach

To assess the security of the admin login page, several common Cross-Site Scripting (XSS) payloads are injected into the input fields – primarily the username box. The payload <script>alert(1)</script> was used as a representative test case. Submissions were performed using the browser interface as well by capturing and modifying requests via developer tools and interception proxies.

Observation & Results

No XSS alert popups or script execution was observed in the browser after form submission, either in the login interface or in any error messages.

The server response remained unchanged regardless of input and did not reflect user-supplied data.

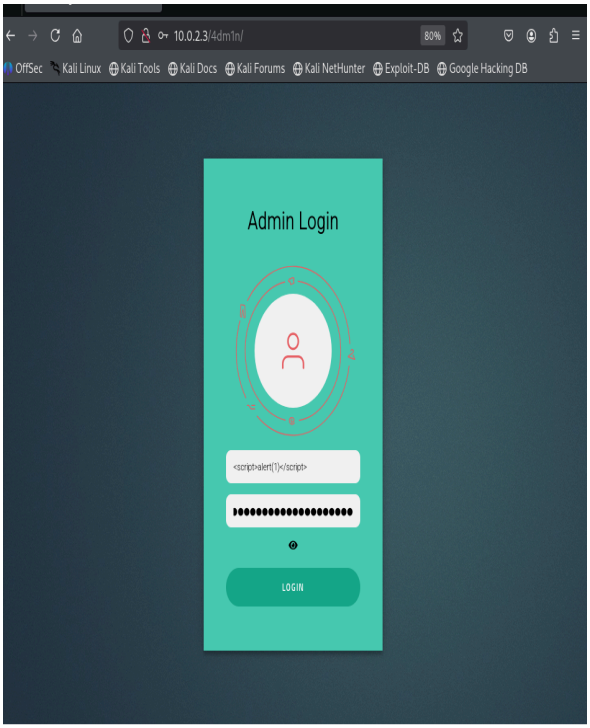
The developer/network tool and ZAP proxy captures show "Access Denied" (403/401 errors) or generic login failures, with no evidence that XSS input affected page output or application flow.

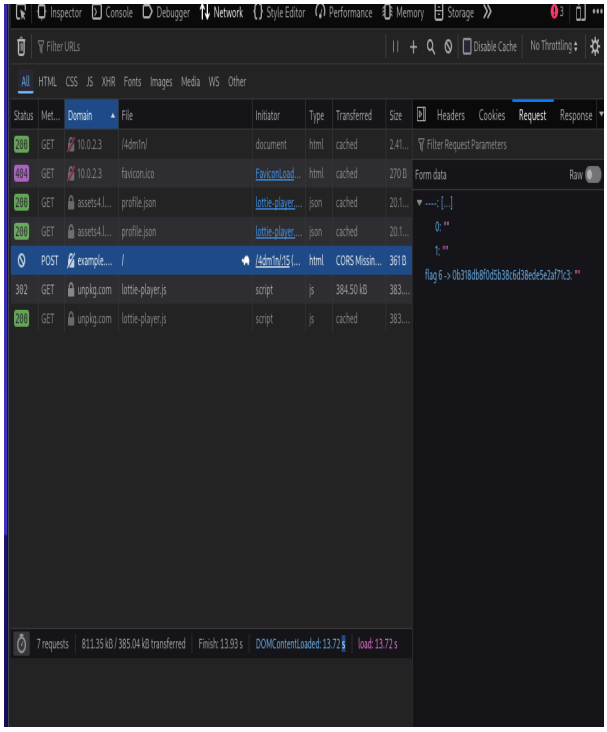
Conclusion

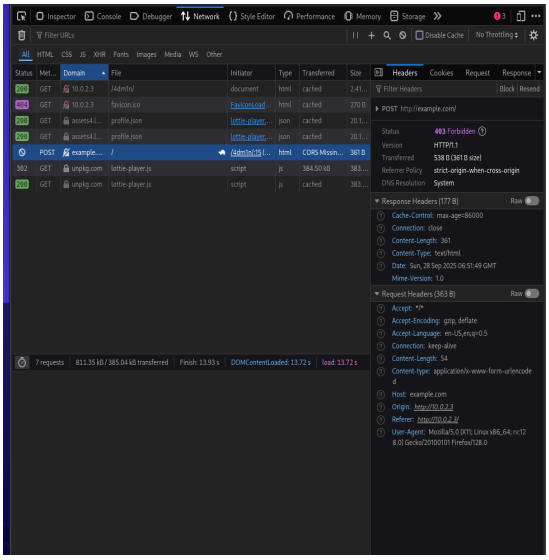
Based on this series of tests and the fact that there are no error messages regarding false inputs - we know that there is no validation of inputs, nor we are stopped from attempting failed login. Also since there is no request body except visible flag, with http protocol we safely deduce that that login request is simulated/fake and not processed at all.

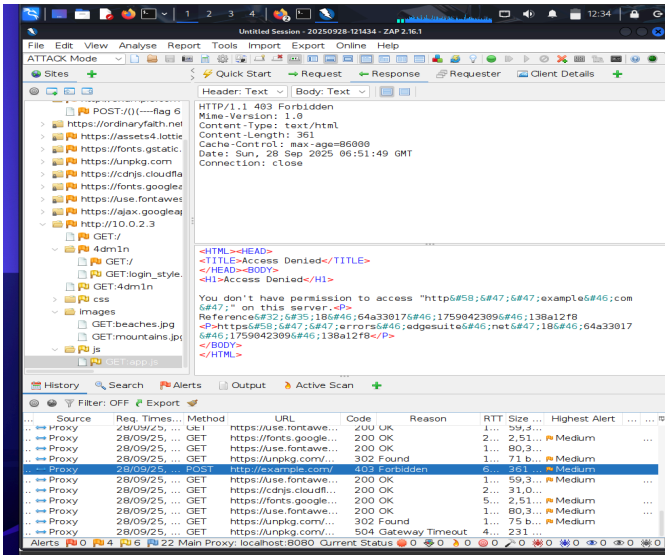
Evidence Screenshots

Screenshots of the attempted attacks and system/network responses are attached here as documentation for the assessment.









SQL Injection Testing on /4dm1n Page

Testing Approach

Form credentials and payloads are submitted via POST requests to the login endpoint (/4dm1n) as seen in the captured network traffic.

The request includes a login attempt, with crafted input designed to test for SQL injection, such as special characters, quotes, or typical bypass strings.

The application responds with an HTTP 403 Forbidden status, blocking further interaction and stating "Access Denied".

The proxy logs confirm that requests are routed to the admin page and that all attempts are denied with status codes (e.g., 301 Moved Permanently, 403 Forbidden).

Observations

No SQL error messages or flawed behavior were displayed in the browser or network logs. This suggests that input is either sanitized before reaching the database or the backend is simulated/fake and not processing queries at all.

The POST payload parameters (visible in the dev tools network view) do not appear to directly cause SQL syntax errors.

The presence of strict server controls and redirect rules (status codes 403, 301) indicates defensive backend handling.

The lack of database-specific error messages is a positive sign, demonstrating that the backend either uses error handling best practices or it is a simulation environment not fed by a real database.

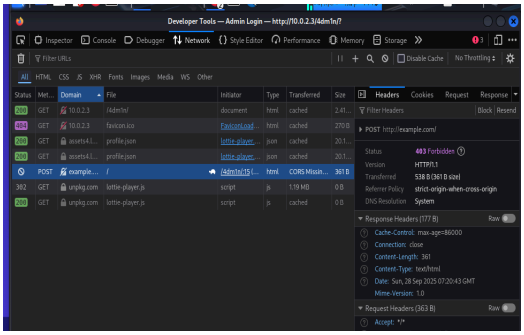
Conclusion

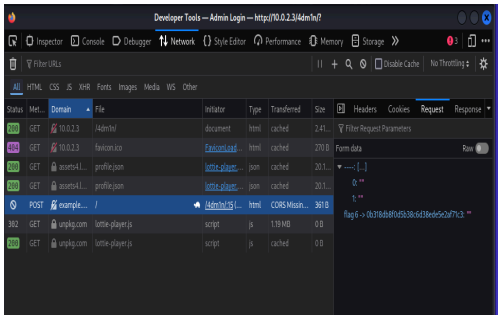
Despite submitting test payloads for SQL injection, the page did not reflect or leak sensitive database errors, nor did it allow bypass authentication via crafted input.

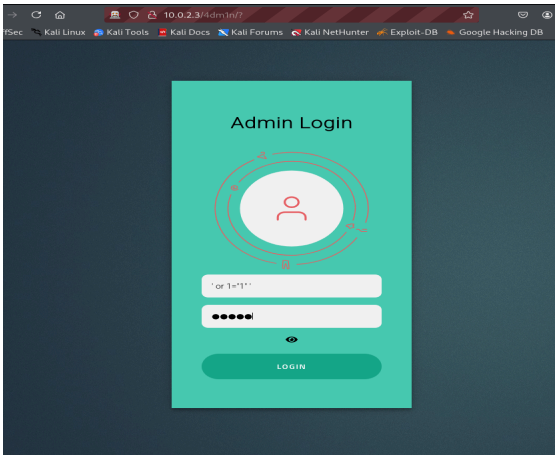
This simulated setup is either immune to SQL injection by virtue of non-existent backend logic or excellent backend error handling.

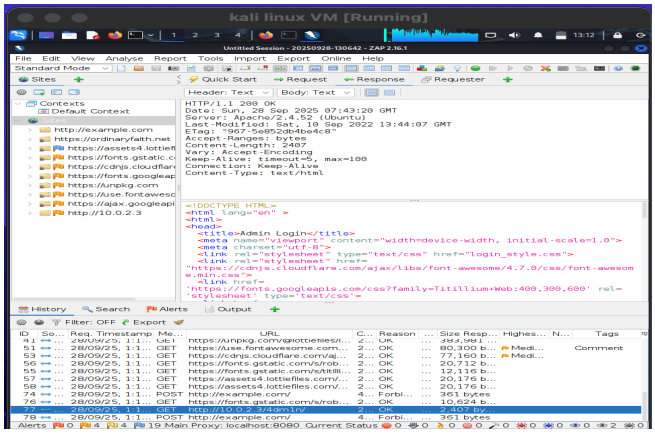
Evidence Screenshots

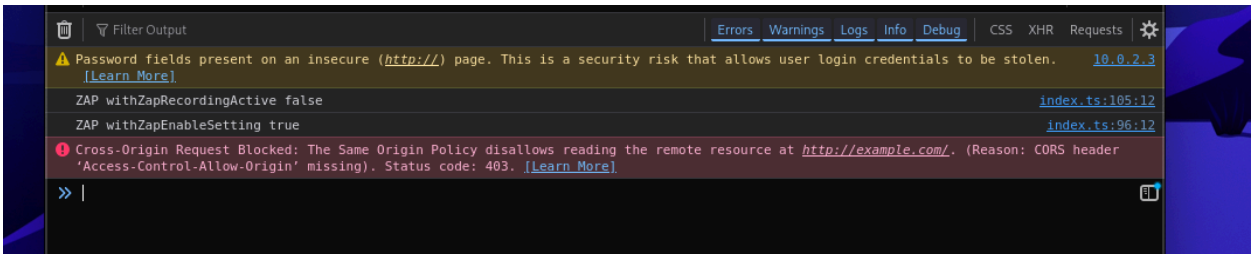
Screenshots of the attempted attacks and system/network responses are attached here as documentation for the assessment.

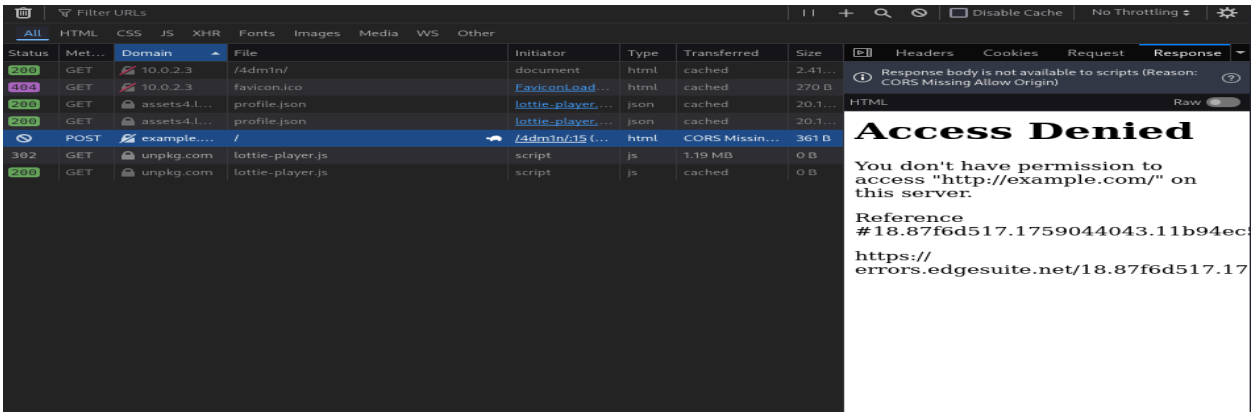


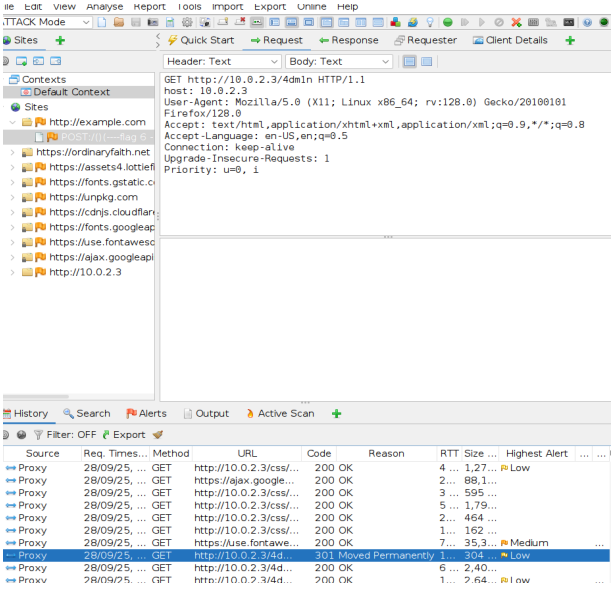


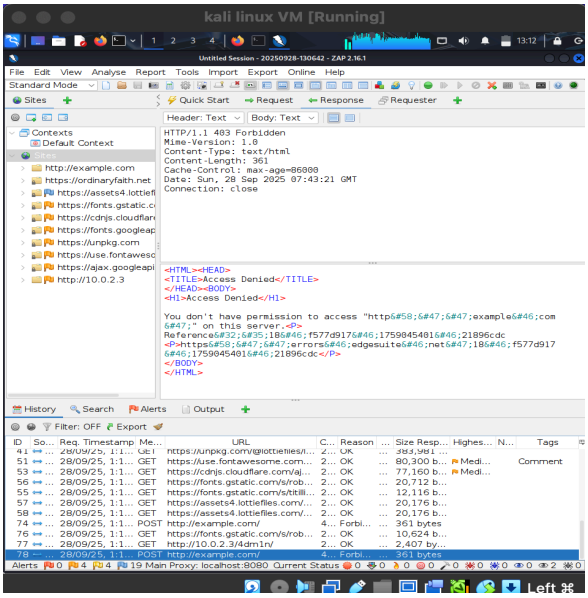





Filter Output
Errors
Warnings
Logs
Info
Debug
CSS
XHR
Requests
Password fields present on an insecure (http://) page. This is a security risk that allows user login credentials to be stolen.
[Learn More]
10.0.2.3
ZAP withZapRecordingActive false
index.ts:105:12
ZAP withZapEnableSetting true
index.ts:96:12
Cross-Origin Request Blocked: The Same Origin Policy disallows reading the remote resource at http://example.com/. (Reason: CORS header 'Access-Control-Allow-Origin' missing). Status code: 403. [Learn More]


Filter URLs
Disable Cache
No Throttling
All
HTML
CSS
JS
XHR
Fonts
Images
Media
WS
Other
Status
Met...
Domain
File
Initiator
Type
Transferred
Size
Headers
Cookies
Request
Response
Response body is not available to scripts (Reason: CORS Missing Allow Origin)
HTML
Raw
Access Denied
You don't have permission to access "http://example.com/" on this server.
Reference #18.87f6d517.1759044043.11b94ec
https://errors.edgesuite.net/18.87f6d517.17


Quick Start
Request
Response
Requester
Client Details
GET http://10.0.2.3/4dmln HTTP/1.1
host: 10.0.2.3
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i
History
Search
Alerts
Output
Active Scan


kali linux VM [Running]
HTTP/1.1 403 Forbidden
Mime-Version: 1.0
Content-Type: text/html
Content-Length: 361
Cache-Control: max-age=86000
Date: Sun, 28 Sep 2025 07:43:21 GMT
Connection: close
History
Search
Alerts
Output

Next Steps in Security Assessment

Verification and Validation

- Retesting the application after patches / fixes are applied by the development team.
- Use the same tools (Burp, ZAP, DIRB, manual testing) to confirm vulnerabilities have been fixed.
- Ensure no new issues have been introduced during remediation.

Knowledge Sharing and Improvement

- Discuss with developers and stakeholders about the lessons learned.
- Recommend improvements for secure development, regular vulnerability assessments, and awareness training.

Ongoing Monitoring and Hardening

- Propose to establish continuous monitoring for new vulnerabilities.
- Suggest periodic penetration testing and code reviews as part of the security lifecycle.
- These steps ensure our work directly leads to a more secure application and helps build a process for future security.

Summary: OWASP Vulnerabilities and Recommendations

OWASP Vulnerability	Files/ Url	Why	Recommendations
Security Misconfiguration: Directory Listing	`/css`, `/images`, `/js`, `/pages` directories allow directory listing	Directory listing lets attackers view all files in these folders, exposing resources that should be hidden, such as scripts, configuration files, or backups	Disable directory listing in the web server configuration. Use access control rules to protect sensitive directories (e.g., .htaccess for Apache)
Security Misconfiguration/ Inefficient Authorisation: Exposed FTP Service	FTP service allows anonymous login and listing/downloading of files	Anonymous login can expose sensitive files to unauthorized users and presents a vector for information leakage	Disable anonymous FTP logins and restrict file access with proper user authentication. Ensure FTP is disabled if not necessary
Cryptographic Failure: Login Over Plain HTTP	Admin login form (`/4dm1n`) sends credentials via HTTP (no HTTPS)	Credentials transmitted in plain text are susceptible to interception, allowing attackers to steal usernames and passwords	Enforce HTTPS for all login and sensitive pages. Redirect all HTTP traffic to HTTPS and use HTTP Strict Transport Security (HSTS)
Cross-Site Scripting–XSS	Admin login page (`/4dm1n`), vulnerable to input not being sanitized/escaped (tested with	Unsanitized user input can be injected and executed as scripts in the browser of other users, leading to data theft or session	Always sanitize and encode all user input before rendering it in the DOM. Implement a Content Security Policy (CSP) to mitigate the

	tools like Burp Suite and ZAP)	hijacking	impact of XSS.
Injection: SQL Injection	Admin login page or any input fields interacting with the backend.	If SQL queries use unsanitized user input, attackers can manipulate queries for authentication bypass or data exfiltration	Use parameterized statements and prepared queries for all database operations. Validate and sanitize all inputs server-side
Security Misconfiguration/Information Disclosure: Verbose Error Messages	`404` and `504` error pages, verbose server headers.	Exposes backend information (stack traces, detailed errors, server versions), which may assist attackers in crafting exploits	Customize error messages to be generic. Suppress stack traces and detailed backend information from end-users; log detailed errors only on the server.
Cryptographic Failures: Password Field in Insecure Context	Admin login page (`/4dm1n`), form uses HTTP and has a password field.	Transmitting passwords over HTTP exposes users to credential theft through network sniffing.	Always require password forms to use HTTPS and set secure flags; consider implementing Multi-Factor Authentication (MFA)
Security Misconfiguration/ Information Disclosure: Sensitive information in comments	HTML source for `blog` components contains hidden flags and comments.	Comments with sensitive information (such as flags, or potentially credentials in real apps) can be accessed by attackers via the source code	Remove any sensitive information from code comments before deploying to production. Minify or obfuscate files for deployment where appropriate
Security Misconfiguration: Missing Content Security Policy Header	HTTP header analysis (using ZAP and Burp Suite).	CSP mitigates many client-side attack vectors, especially XSS and data injection attacks.	Set a strong Content Security Policy header that restricts sources for scripts, styles, and other critical resources.

References

References for Documentation and Downloads

OWASP ZAP Proxy
https://www.zaproxy.org/

Burp Suite Community Edition
https://portswigger.net/burp

DIRB Web Content Scanner
https://tools.kali.org/web-applications/dirb

Kali Linux Distribution
https://www.kali.org/documentation/

References for Security Vulnerability and Recommendations

OWASP Application Security Verification Standard
https://owasp.org/www-project-application-security-verification-standard/

Sensitive info disclosure (exposed PHP info/flag)
https://www.php.net/manual/en/function.phpinfo.php#87803

Missing Content Security Policy (CSP) header
https://owasp.org/www-project-secure-headers/

Directory listing enabled (info disclosure risk)
https://httpd.apache.org/docs/current/misc/security_tips.html#directory

OWASP: Cross Site Scripting (XSS)
https://owasp.org/www-community/attacks/xss/

OWASP: SQL Injection
https://owasp.org/www-community/attacks/SQL_Injection